

Neural Network Fundamentals

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

What this chapter covers

- What a neural network computes (a differentiable parametric function)
- How losses are defined — MSE for regression, cross-entropy for classification
- How gradients are computed via backpropagation (chain rule on computation graphs)
- How optimisation updates parameters (gradient descent)
- Building a simple autoencoder and visualising a 2D latent space

1 — A Neural Network is a Differentiable Program

A neural network defines a **parametric function** $f_{\theta}(x)$ in $\mathbb{R}^k$, where x is the input, θ are the learnable parameters (weights and biases), and k is the output dimension. Training finds the parameters that minimise a loss function L over the data distribution:

$$\theta^* = \operatorname{argmin}_{\theta} E_{(x,y)} [L(f_{\theta}(x), y)]$$

The network is a *composition* of simple building blocks stacked into layers. Each layer applies a linear transformation followed by a nonlinearity. The key insight is that the entire pipeline is differentiable end-to-end, which is what makes gradient-based learning possible.

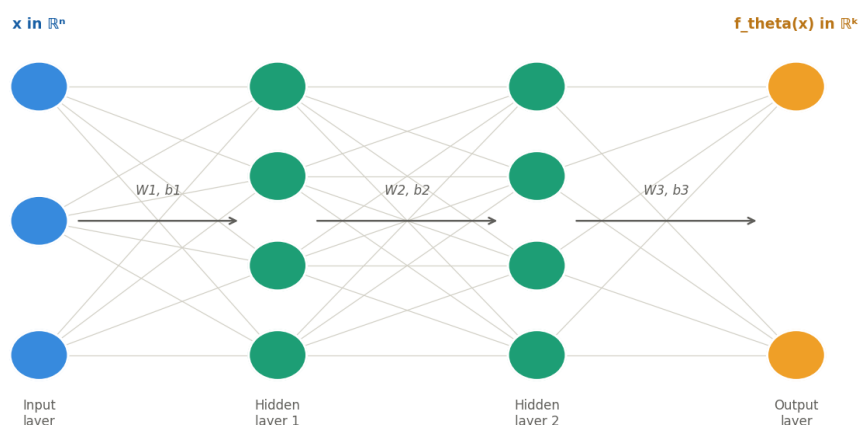


Figure 1 — A 2-hidden-layer MLP. Data flows left to right through learned weight matrices $W1, W2, W3$.

Common building blocks

Component	Formula / Role	When used
Linear layer	$h = Wx + b$	Every layer — core transformation
ReLU	$\max(0, x)$	Hidden layers — most common nonlinearity
Sigmoid	$1 / (1 + \exp(-x))$	Binary classification outputs
Tanh	$\tanh(x)$	Recurrent networks, normalised outputs
BatchNorm	Normalise per mini-batch	Deep networks — stabilises training
LayerNorm	Normalise per token/sample	Transformers — sequence models
Softmax	$\exp(x_i) / \sum \exp(x_j)$	Multi-class outputs — produces probabilities

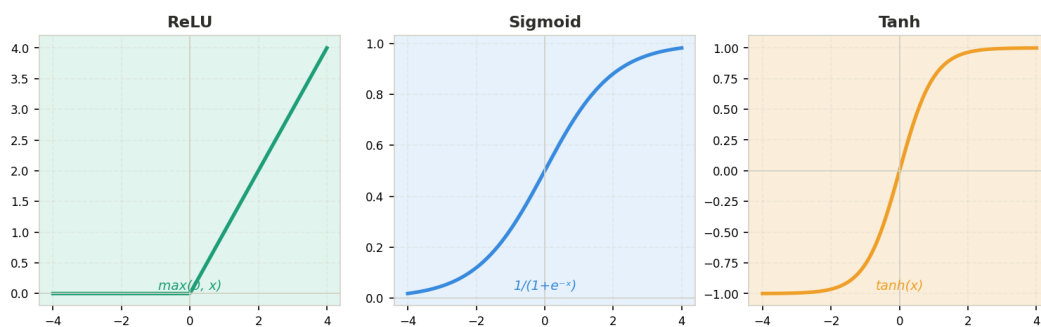


Figure 2 — The three most common nonlinearities. ReLU is the default for hidden layers; sigmoid and tanh appear in specific output/gating contexts.

2 — Loss Functions: What Are We Minimising?

The loss function measures how wrong the network's prediction is. The choice of loss encodes an assumption about the data-generating distribution.

Mean Squared Error (MSE) — for regression

MSE assumes the outputs are corrupted by Gaussian noise. Minimising MSE is equivalent to maximum-likelihood estimation under a Gaussian likelihood model:

$$L_{MSE} = (1/N) \sum_i (y_{\text{hat}} - y_i)^2$$

Cross-Entropy — for classification

Cross-entropy assumes outputs represent class probabilities (via softmax). Minimising it is equivalent to maximum-likelihood under a categorical distribution:

$$L_{CE} = -(1/N) \sum_i \sum_c y_{ic} \log p_{ic}$$

For *binary* classification with a sigmoid output: $L = -[y \log(p) + (1-y) \log(1-p)]$.

3 — Backpropagation: How Gradients Are Computed

Backpropagation computes $dL/d\theta$ for every parameter θ by applying the **chain rule** through the computational graph — from the loss backwards to each weight. This is the single most important algorithm in deep learning.

The chain rule in one line:

$$dL/d\theta = dL/da_L \times da_L/da_{L-1} \times \dots \times da_1/d\theta$$

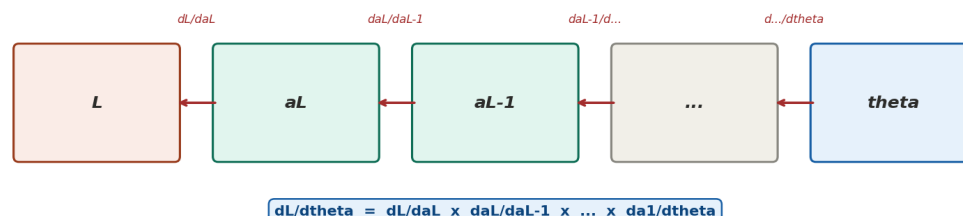


Figure 3 — The chain rule applied backwards through the computation graph. Each arrow is one local derivative; the product of all arrows gives $dL/d\theta$.

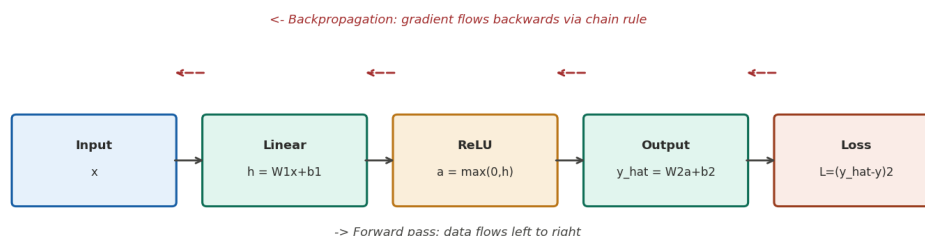


Figure 4 — Forward pass (solid arrows) and backward pass (dashed red arrows) through a 2-layer network. Gradients flow right to left, assigning blame to each parameter.

The ReLU gradient — the tricky part

$\text{ReLU}(x) = \max(0, x)$. Its derivative is: 1 if $x > 0$, and 0 if $x \leq 0$.

A neuron where $h \leq 0$ has zero gradient — it is "dead" for that input.

This is captured by the `relu_mask = (h > 0).float()` step in the lecture code.

Too many dead neurons can stall training — this motivates careful initialisation.

Manual backprop for a 2-layer network (exam-relevant)

Consider a 2-layer network: $h = W_1x + b_1$, $a = \text{ReLU}(h)$, $y_{\text{hat}} = W_2a + b_2$, $L = (y_{\text{hat}} - y)^2$. The gradients are computed as:

Step	Quantity	Formula
1	dL / dy_{hat}	$2(y_{\text{hat}} - y)$
2	dL / dW_2	$(dL/dy_{\text{hat}}) * a^T$

3	dL / da	$(dL/dy_{\hat{}}) * W2$
4	dL / dh	$(dL/da) \blacksquare 1[h > 0]$ (ReLU mask)
5	$dL / dW1$	$(dL/dh)^T * x^T$

4 — Gradient Descent and Optimisation

Once gradients are computed, parameters are updated in the direction that reduces the loss. Vanilla gradient descent: $\theta \leftarrow \theta - \eta * dL/d\theta$, where η (learning rate) controls step size.

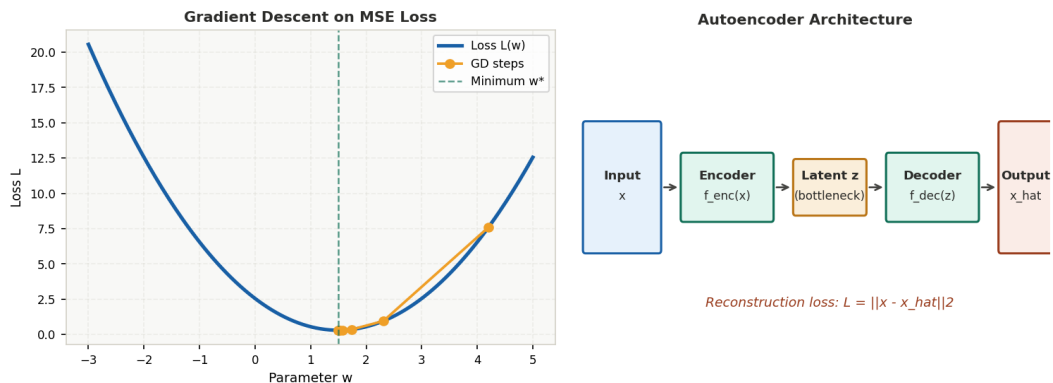


Figure 5 — Left: gradient descent converging on the MSE minimum. Right: the autoencoder architecture — encoder compresses x into bottleneck z ; decoder reconstructs $x_{\hat{}}$.

Optimiser	Key idea	When to use
SGD	Update using mini-batch gradient	Simple baselines
Momentum	Accumulate gradient direction	Smoother convergence
Adam	Adaptive per-parameter learning rates	Default for most deep learning
AdamW	Adam + weight decay decoupled	Transformers, fine-tuning

5 — Autoencoders: Gateway to Generative Models

An autoencoder learns to compress data into a low-dimensional latent space and then reconstruct it. It has two parts: an **encoder** $f_{\text{enc}}(x) \rightarrow z$, and a **decoder** $f_{\text{dec}}(z) \rightarrow x_{\hat{}}$. Training minimises reconstruction loss: $L = \|x - x_{\hat{}}\|^2$. This is the conceptual foundation for VAEs (Chapter 2).

Why autoencoders matter for this course

- They introduce the idea of a latent space — a compressed representation of the data.
- A standard (deterministic) autoencoder produces disconnected latent regions $\rightarrow$ cannot sample.
- A VAE (Chapter 2) fixes this by making the latent space probabilistic and structured.
- The encoder/decoder pattern appears in VAEs, diffusion (U-Net), VLMs, and flow models.

Sample Exam Questions

Q1 — What does backpropagation compute, and why is the chain rule central to it?

Backpropagation computes the gradient $dL/d\theta$ for every parameter in the network. The chain rule is central because the loss depends on the output through a composition of many functions. The chain rule allows us to decompose $dL/d\theta$ into a product of local derivatives, each easy to compute, and multiply them in a single backwards pass through the graph. Without the chain rule, computing gradients for deep networks would require finite differences — expensive and imprecise.

Q2 — A ReLU neuron has $h = -0.5$ for a given input. What is its gradient contribution?

Zero. Since $h \leq 0$, ReLU outputs 0, and its derivative is also 0 at this point. The gradient from upstream is multiplied by 0, so no gradient flows through this neuron for this input. This is called a "dead" neuron. If many neurons are dead, learning stalls — motivating initialisation strategies and alternative activations like Leaky ReLU.

Q3 — Explain the difference between MSE and cross-entropy. When should each be used?

MSE measures the squared difference between prediction and target. It is appropriate when the output is a continuous value and implicitly assumes Gaussian noise around the target. Cross-entropy measures the log-loss between predicted class probabilities and the true one-hot label. It is appropriate for classification because it assumes a categorical distribution and is much more sensitive to confident wrong predictions than MSE.

Q4 — Why can a standard (deterministic) autoencoder not be used as a generative model?

A deterministic autoencoder maps each input to a single point in latent space. These points can be arbitrarily distributed — there is no guarantee that the space between them contains meaningful representations. Sampling a random latent vector z and decoding it will typically produce unrealistic outputs. The model can reconstruct training examples but cannot define a proper generative distribution over the data. VAEs solve this by imposing a structured prior $p(z) = N(0, I)$ on the latent space via the KL term in the ELBO.

Key Concepts Summary

Key concepts

- Neural network = parametric differentiable function $f_{\theta}(x)$
- Loss function encodes the target distribution assumption (MSE $\rightarrow$ Gaussian, CE $\rightarrow$ Categorical)
- Backpropagation = chain rule applied backwards through the computation graph
- ReLU gradient = 1 if $h > 0$, else 0 (dead neuron problem)
- Gradient descent update: $\theta \leftarrow \theta - \eta * dL/d\theta$

- Autoencoder: encoder $\rightarrow$ bottleneck z $\rightarrow$ decoder; trained with reconstruction loss
- Deterministic autoencoders cannot generate — they produce disconnected latent spaces
- Adam is the default optimiser for most deep learning; AdamW for transformers

Next: Chapter 02 — Variational Autoencoders, where we add a probabilistic latent model to enable true generative sampling.

Variational Autoencoders (VAE)

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Learning Objectives (from lecture)

- Explain why VAEs are needed (AE limitations)
- Understand why ELBO is needed and the role of Jensen's inequality
- Understand the reparameterisation trick and why sampling fails in AE
- Understand entanglement/disentanglement in latent variable models
- Explain Beta-VAE, CVAE, and posterior collapse

1 — Why Deterministic Autoencoders Fail as Generative Models

A deterministic AE maps input to a fixed latent code $z = f_{\text{enc}}(x)$, then reconstructs $\hat{x} = f_{\text{dec}}(z)$. Training only minimises reconstruction loss — it imposes **no structure** on the latent space. This causes three problems:

The three failures of deterministic AEs

1. No probability model: the AE assigns no likelihood to data. It cannot answer "how probable is x ?"
2. Holes in latent space: codes $\{z = f_{\text{enc}}(x_i)\}$ can be scattered anywhere — empty regions between clusters.
3. Sampling fails: drawing $z \sim N(0, I)$ and decoding produces nonsense because the AE was never trained for that.

VAE fix: make z probabilistic and regularised toward $N(0, I)$ so random samples land in valid regions.

2 — The Probabilistic VAE Model

A VAE defines a proper generative model:

Prior: $p(z) = N(0, I)$ | Decoder: $p_{\text{theta}}(x|z)$ (neural network)

Joint: $p_{\text{theta}}(x, z) = p_{\text{theta}}(x|z) p(z)$

Marginal: $p_{\text{theta}}(x) = \int p_{\text{theta}}(x|z) p(z) dz$ [INTRACTABLE]

The marginal is intractable because the decoder is a nonlinear neural network — there is no closed form for the integral over a high-dimensional continuous z . This is the central computational problem VAEs must solve.

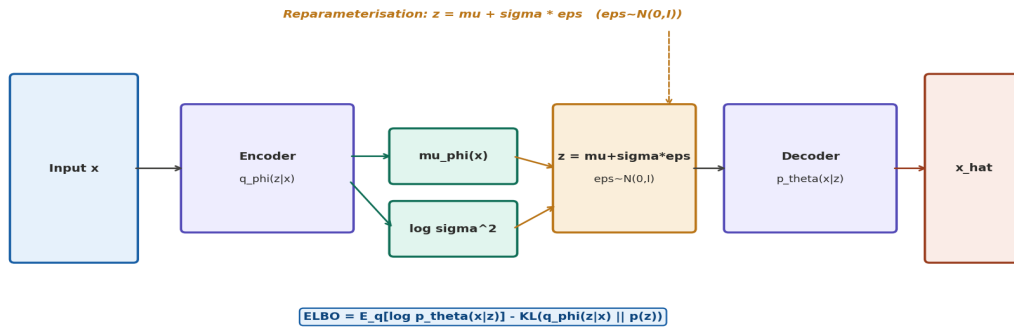


Figure 1 — VAE architecture. Encoder outputs $(\mu, \log \sigma^2)$; reparameterisation samples $z = \mu + \sigma \cdot \epsilon$; decoder reconstructs $x_{\hat{}}$. ELBO = reconstruction term - KL term.

3 — Jensen's Inequality and the ELBO Derivation (Lecture Section 2.7)

Because $\log p_{\theta}(x)$ is intractable, we derive a tractable lower bound using **Jensen's inequality**. This is the most important mathematical step in the VAE lecture.

Jensen's Inequality for concave functions

For any concave function f (like $\log$) and random variable Y : $f(E[Y]) \geq E[f(Y)]$

For $\log$ specifically: $\log(E[Y]) \geq E[\log Y]$

The inequality is tight when Y is constant (no randomness).

Applying Jensen to derive the ELBO:

$$\log p_{\theta}(x) = \log E_{q_{\phi}(z|x)}[p_{\theta}(x,z) / q_{\phi}(z|x)]$$

$$\geq E_{q_{\phi}(z|x)}[\log(p_{\theta}(x,z) / q_{\phi}(z|x))] \text{ (Jensen)}$$

$$= E_q[\log p_{\theta}(x|z)] - KL(q_{\phi}(z|x) || p(z)) = ELBO$$

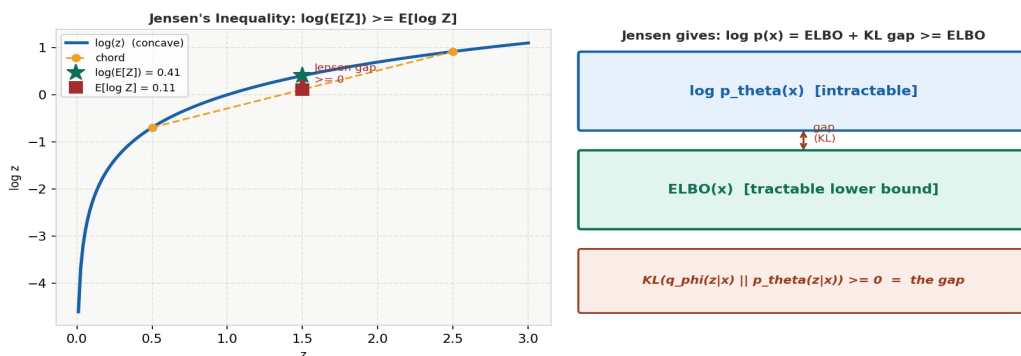


Figure 2 — Left: Jensen's inequality on the concave $\log$ function. The chord lies BELOW the curve, so $E[\log Y] \leq \log(E[Y])$. Right: ELBO is always $\leq \log p(x)$; gap = $KL(q || p_{\text{true_posterior}})$.

4 — The ELBO: Two Terms and Their Tension

$$ELBO(x) = E_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - KL(q_{\phi}(z|x) || p(z))$$

The two terms pull in opposite directions:

Term	Formula	Role	If missing
Reconstruction	$E_q[\log p_{\theta}(x z)]$	Decoder explains x from z	Blurry outputs; z not used
KL regularisation	$-KL(q_{\phi}(z x) \parallel p(z))$	Pulls q toward $N(0, I)$	Disconnected latent; sampling fails

The bound becomes exact when $q_{\phi}(z|x) = p_{\theta}(z|x)$ (zero KL gap). Maximising the ELBO simultaneously improves the decoder AND the inference model q .

5 — Analytical KL for Gaussians (Lecture Section 2.8)

When $q_{\phi}(z|x) = N(\mu, \sigma^2)$ and $p(z) = N(0, I)$, the KL has a closed form — no sampling needed. This is why Gaussians are special for VAEs:

$$KL(N(\mu, \sigma^2) \parallel N(0, I)) = 0.5 * \sum(\sigma^2 + \mu^2 - 1 - \log(\sigma^2))$$

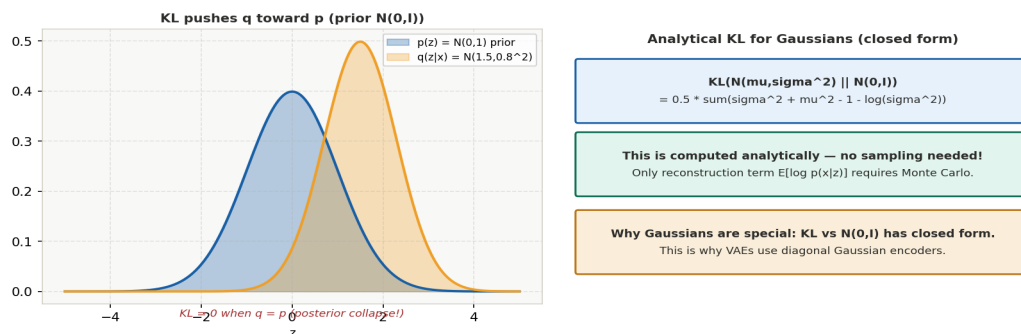


Figure 3 — Left: KL pushes q (posterior) toward p (prior $N(0, I)$). Posterior collapse = q becomes p for all x . Right: analytical KL formula for Gaussians — only reconstruction term needs Monte Carlo sampling.

6 — Reparameterisation Trick (Lecture Section 2.11)

The ELBO expectation requires gradients with respect to ϕ (encoder parameters), but the sampling step $z \sim q_{\phi}(z|x)$ breaks backpropagation. The trick rewrites sampling as a deterministic transformation:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) * \epsilon, \epsilon \sim N(0, I)$$

Now z is a differentiable function of (μ, σ) . Gradients flow back to the encoder. The randomness is in ϵ , which has no learnable parameters.

Why Gaussians are uniquely useful for reparameterisation

Sampling from $N(\mu, \sigma^2) = \mu + \sigma * N(0, 1)$ — trivial to reparameterise.

KL vs $N(0, I)$ has a closed form — no sampling needed for the regularisation term.

Both properties together make end-to-end training efficient and exact.

Other distributions often lack one or both properties.

7 — Entanglement, Disentanglement, Beta-VAE, CVAE (Lecture Sections 2.14–2.18)

Disentanglement means each latent dimension corresponds to a separate, interpretable factor of variation. Standard VAEs do NOT guarantee this — the ELBO objective does not force factor separation.

Method	Modification	Effect	Tradeoff
Standard VAE	ELBO as-is	No disentanglement guarantee	Good reconstruction
Beta-VAE ($\beta > 1$)	Weight KL by β	Stronger compression → factor separation	Worse reconstruction
CVAE	Condition on y : $q(z x, y)$, $p(x z, y)$	z models residual variation only	Requires labels y
Posterior collapse	$q(z x) \approx p(z)$ for all x	z carries no information about x	Decoder ignores z

Inductive bias and disentanglement (Lecture 2.15)

Disentanglement is mostly a function of inductive bias — the structure we impose on the model.

Weak inductive bias: the model is free to represent any latent structure, including entangled.

Beta-VAE increases pressure: compressed codes are forced to be more factorial.

Neither approach reliably produces human-interpretable factors without strong supervision.

Sample Exam Questions — with Model Answers

Q1 — Why is the ELBO called a lower bound on $\log p_{\theta}(x)$? When does it become exact?

The ELBO is a lower bound because $\log p_{\theta}(x) = \text{ELBO} + \text{KL}(q_{\phi}(z|x) \parallel p_{\theta}(z|x))$. Since $\text{KL} \geq 0$ always, $\text{ELBO} \leq \log p_{\theta}(x)$. The bound becomes exact when $q_{\phi}(z|x) = p_{\theta}(z|x)$ — i.e., the encoder perfectly matches the true posterior and the KL gap is zero. This shows VAE training simultaneously improves the decoder and tightens the inference model.

Q2 — Explain the role of Jensen's inequality in deriving the ELBO.

Jensen's inequality states that for a concave function f : $f(E[Y]) \geq E[f(Y)]$. Applied to $\log$: $\log(E_{q}[p(x, z)/q(z|x)]) \geq E_q[\log(p(x, z)/q(z|x))]$. The left side is $\log p(x)$ (intractable). The right side is the ELBO (tractable). Jensen guarantees the ELBO is always a lower bound — so maximising it is a valid surrogate.

Q3 — Explain the reparameterisation trick. Why is it more than a coding convenience?

The trick rewrites $z = \mu + \sigma * \epsilon$ ($\epsilon \sim N(0, I)$), separating randomness (ϵ) from learnable parameters (μ, σ). This makes z a differentiable function of the encoder outputs, so gradients flow through the sampling step. Without it, the gradient of the ELBO with respect to ϕ would require differentiating through a stochastic node — impossible with standard backpropagation. The trick is the key innovation that makes VAEs trainable end-to-end.

Q4 — Why does a standard VAE not guarantee disentangled representations?

The ELBO rewards reconstruction quality and latent regularity (toward $N(0, I)$), but neither pressure forces separate latent dimensions to correspond to separate semantic factors. The model is free to entangle multiple factors if that optimises the ELBO. Beta-VAE addresses this by upweighting the KL term (forcing more compressed codes), and CVAE conditions on known factors to reduce the burden on z . Both require extra inductive bias — disentanglement does not emerge from the ELBO alone.

Q5 — What is posterior collapse? What causes it and what are the consequences?

Posterior collapse occurs when $q_\phi(z|x) \approx p(z) = N(0, I)$ for all inputs x . The latent variable z carries virtually no information about x — the encoder ignores the input. Causes: too-powerful decoder (can reconstruct without z), or too-strong KL pressure early in training. Consequence: the KL term is trivially minimised, but reconstruction degrades because z provides no input-specific signal. The model fails its intended purpose as a structured generative model.

Key Concepts Summary

- AE fails as generative model: no $p(x)$, disconnected latent space, sampling from $N(0, I)$ produces nonsense
- VAE: encoder $q_\phi(z|x) = N(\mu, \sigma^2)$; decoder $p_\theta(x|z)$; marginal $p(x) = \int p_\theta(x|z) q_\phi(z|x) dz$ [INTRACTABLE]
- Jensen's inequality: $\log(E[Y]) \geq E[\log Y]$ (concave) $\rightarrow$ gives ELBO lower bound
- ELBO = $E_q[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$: reconstruction - KL
- ELBO exact when $q_\phi(z|x) = p_\theta(z|x)$: zero gap, perfect inference
- Analytical KL for Gaussians: $0.5 * \sum(\sigma^2 + \mu^2 - 1 - \log \sigma^2)$
- Reparameterisation: $z = \mu + \sigma * \epsilon$, $\epsilon \sim N(0, I)$ — differentiable, enables backprop
- Gaussians are special: KL has closed form AND easy reparameterisation
- Beta-VAE: $L = \text{reconstruction} - \beta * \text{KL}$; $\beta > 1 \rightarrow$ compression $\rightarrow$ disentanglement (but worse recon)
- CVAE: $q(z|x, y)$, $p(x|z, y)$ — z models residual variation; conditioning reduces entanglement
- Posterior collapse: $q(z|x) \approx p(z) \rightarrow z$ carries no info $\rightarrow$ decoder ignores latent variable

Diffusion Models

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Learning Objectives (from lecture)

- Review what a Markov chain is and its key properties
- Understand the basic idea behind diffusion generative processes
- Know the various families of diffusion processes
- Understand the forward diffusion process and why the marginal $q(x_{t|x_0})$ matters
- Understand why modelling the reverse process is hard
- Understand how a neural network is trained for the backwards process
- Understand why sinusoidal encoding is required for time input
- Understand why U-Net is an appropriate architecture

1 — Markov Chains: The Foundation (Lecture Review)

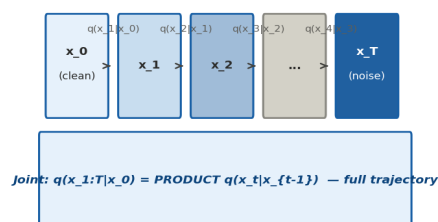
A Markov chain is a sequence of random variables $x_0, x_1, x_2, \dots$ satisfying the **Markov property**: the future depends only on the present, not the past.

$$P(x_t | x_{\{t-1\}}, x_{\{t-2\}}, \dots, x_0) = P(x_t | x_{\{t-1\}})$$

A Markov chain is fully specified by an initial distribution $P(x_0)$ and a transition distribution $P(x_{t+1} | x_t)$. The joint distribution factorises as a product of transitions — this tractability is what makes diffusion models practical.

$$P(x_{0:T}) = P(x_0) * \text{PRODUCT}_{\{t=1\}^T} P(x_t | x_{\{t-1\}})$$

Markov Chain: joint factorises as product of transitions



Trajectory vs Marginal — key distinction

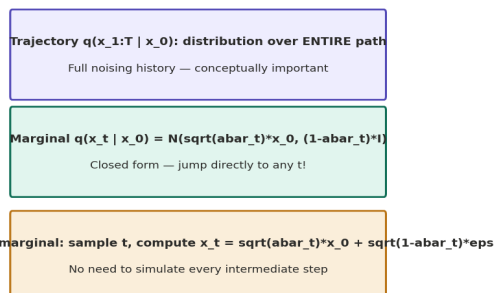


Figure 1 — Left: Markov chain for diffusion — each step depends only on the previous. Joint = product of transitions. Right: Trajectory $q(x_{1:T}|x_0)$ vs marginal $q(x_t|x_0)$ — key distinction.

2 — The Forward Process: Controlled Corruption

The forward process adds Gaussian noise step by step, gradually destroying the data. It is a Markov chain with Gaussian transitions:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{\alpha_t} * x_{t-1}, (1 - \alpha_t) * I)$$

where $\alpha_t = 1 - \beta_t$ and β_t is the noise schedule. The forward process is **not learned** — it is designed by the practitioner. After T steps, the distribution approaches $N(0, I)$.

The closed-form marginal — why it matters for training

$$q(x_t | x_0) = N(x_t; \sqrt{\alpha_{\bar{t}}} * x_0, (1 - \alpha_{\bar{t}}) * I)$$

$$\text{where } \alpha_{\bar{t}} = \text{PRODUCT}_{s=1}^t \alpha_s$$

$$\text{Equivalently: } x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \text{eps}, \text{eps} \sim N(0, I)$$

Why the marginal is critical for training

Training requires (x_0, x_t) pairs. The trajectory needs all intermediate steps.

The marginal $q(x_t | x_0)$ lets us jump directly to any t — no chain simulation needed.

Training loop: sample $t \sim \text{Uniform}(1, T)$, sample $\text{eps} \sim N(0, I)$, compute $x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \text{eps}$.

This is one of the most important practical simplifications in DDPM-style training.

3 — Why the Reverse Process Must Be Learned

In principle, the reverse $q(x_{t-1} | x_t)$ would let us generate by running the chain backwards. But this reverse conditional depends on the unknown data distribution:

$$q(x_{t-1} | x_t) \text{ is INTRACTABLE — requires knowing } p_{\text{data}}$$

Even though $q(x_t | x_{t-1})$ is simple and Gaussian, the reverse is not. Bayes' rule introduces terms involving the marginals $q(x_t)$ and $q(x_{t-1})$, which are intractable integrals over the data distribution. This is why diffusion models learn a parametric approximation $p_{\theta}(x_{t-1} | x_t)$.

Training objective: predict the noise

The network $\text{eps}_{\theta}(x_t, t)$ is trained to predict the noise eps added at step t :

$$L_{\text{simple}} = E_{\{x_0, \text{eps}, t\}} [\| \text{eps} - \text{eps}_{\theta}(x_t, t) \|^2]$$

This is not arbitrary — it emerges from the ELBO under Gaussian forward noise and is equivalent to approximate score estimation at each noise level.

4 — Why Sinusoidal Timestep Embeddings? (Lecture Learning Objective)

The denoising task is fundamentally different at different noise levels. The network must know the current timestep t to know how much signal vs noise is present. Sinusoidal embeddings convert t into a high-dimensional vector:

$$PE(t, 2i) = \sin(t / 10000^{(2i/d)})$$

$$PE(t, 2i+1) = \cos(t / 10000^{(2i/d)})$$

Why sinusoidal for time embedding?

Each timestep gets a unique, continuous high-dimensional fingerprint.

Relative time relationships are encoded: nearby t values $\rightarrow$ similar embeddings.

No learned parameters needed — works for any T .

Injected at every level of the U-Net via FiLM: $\text{feature_map} = \text{feature_map} + \text{time_bias}(t)$.

5 — The U-Net Architecture for Diffusion

Denosing requires both global context (to understand the scene) and fine spatial detail (to reconstruct textures and edges). U-Nets provide this through:

U-Net component	Purpose	Why it matters for diffusion
Downsampling path	Extract multi-scale context	Global scene understanding
Upsampling path	Reconstruct spatial detail	Fine-grained denoising
Skip connections	Preserve spatial info across scales	Prevents information loss at deep layers
Timestep injection	FiLM conditioning at every level	Model knows noise level at all scales
Self-attention in mid	Global coherence	Ensures long-range consistency

6 — Diffusion Model Families (Lecture Table)

Diffusion Model Families (from lecture)				
Family	Forward process	What is learned	Sampling	Notes
DDPM	Gaussian VP Markov chain	$\epsilon_{\theta}(x_t, t)$ noise	Stochastic ~1000 steps	Canonical; ELBO motivates loss
DDIM	Same as DDPM	Same network	Deterministic 10-100 steps	No retraining needed
Score-SDE	SDE (VP/VE/sub-VP)	Score $s(x_t) = \nabla \log p_t$	Reverse SDE solvers	Unifies many diffusions
LDM	Same in latent z	Same as DDPM in z	DDPM/DDIM in latent	Stable Diffusion; faster
Guided	Same + condition	Same + classifier grad	Steered reverse chain	Classifier or CFG guidance
Consistency	Direct from diffusion	$f_{\theta}(x_t, t)$ map	Very few steps	Distilled from teacher
Flow Matching	ODE (deterministic)	Velocity $v_{\theta}(x_t, t)$	ODE integration	No stochastic term

Figure 2 — All diffusion model families from the lecture. DDPM is canonical; DDIM uses same network with deterministic sampling; Score-SDE unifies via continuous-time SDE; LDM runs in latent space; Flow Matching is the ODE variant.

7 — DDPM vs DDIM: Training and Sampling Are Separate

The DDPM training objective and sampling procedure are independent choices. DDIM keeps the same trained model but uses a deterministic trajectory:

Property	DDPM	DDIM
Training	MSE noise prediction	Identical — same network
Sampling	Stochastic: noise added each step	Deterministic: fixed trajectory
Steps	~1000 for quality	10-100 steps (same quality)
Diversity	Higher	Lower
Speed	Slow	Much faster

Sample Exam Questions — with Model Answers

Q1 — What is the conceptual difference between $q(x_{1:T|x_0})$ and $q(x_{t|x_0})$?

$q(x_{1:T|x_0})$ is the trajectory distribution — the probability over the entire noising sequence. It is a joint distribution over all T intermediate states. $q(x_{t|x_0})$ is the marginal — the distribution of the noisy state at one specific time t , averaging over all intermediate steps. The marginal matters for training because it has a closed form: $q(x_{t|x_0}) = N(\sqrt{\alpha_t}x_0, (1-\alpha_t)\sigma^2)$. This lets us sample training pairs (x_0, x_t) directly at any t without simulating the full chain.

Q2 — Why cannot we use the true reverse $q(x_{t-1}|x_t)$ in practice?

The true reverse conditional requires the data marginals $q(x_t)$ and $q(x_{t-1})$. By Bayes rule: $q(x_{t-1}|x_t) = q(x_{t|x_{t-1}})q(x_{t-1}) / q(x_t)$. Both marginals require integrating over the data distribution, which is unknown. So while the forward transition $q(x_{t|x_{t-1}})$ is simple Gaussian, the reverse is intractable for real data. This is why the reverse process must be learned.

Q3 — Why does predicting noise epsilon make sense as a training target?

Under the forward marginal $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon$, the only unknown given (x_t, t) is the noise realisation ϵ . Predicting ϵ also approximates the score: $\nabla_{x_t} \log q(x_{t|x_0}) = -\epsilon/\sqrt{1-\alpha_t}$. So the noise predictor implicitly learns the score field. The simple MSE loss emerges from the ELBO under fixed variance — it is principled, not an engineering trick.

Q4 — Why must the denoising network know timestep t ?

At small t , x_t retains strong semantic signal and denoising should be gentle. At large t , x_t is nearly pure noise and denoising requires global, uncertain inference. Without t , the network faces an ambiguous mapping — the same x_t value could correspond to very different noise levels. Conditioning on t resolves this by specifying the current SNR. The timestep is part of the state of the reverse problem, not a bookkeeping variable. Sinusoidal embeddings provide a unique fingerprint for each t .

Q5 — Why are diffusion models described as both latent-variable and score-based models?

As a latent-variable model: the trajectory $x_{1:T}$ are latent variables mediating generation. The full model $p_\theta(x_{0:T})$ is a joint distribution; the ELBO provides the training objective. As a

score-based model: the trained network ϵ_θ approximates the score $\nabla_x \log p_t(x)$ at each noise level, guiding samples from noise toward data. These are complementary descriptions of the same process — one emphasises graphical structure, the other emphasises geometric denoising directions.

Key Concepts Summary

- Markov property: $P(x_{t|x_0}, \dots, x_{t-1}) = P(x_{t|x_{t-1}})$ — future depends only on present
- Joint factorises: $P(x_{0:T}) = P(x_0) * \text{PRODUCT } P(x_{t|x_{t-1}})$
- Forward process: adds Gaussian noise, NOT learned, designer choice
- Marginal: $q(x_{t|x_0}) = N(\sqrt{\alpha_t}x_0, (1-\alpha_t)I)$ — CLOSED FORM — key for training
- Training: sample t , sample $\epsilon \sim N(0, I)$, compute x_t directly, predict ϵ via MSE
- Reverse intractable: requires unknown data marginals — must learn $p_\theta(x_{t-1}|x_t)$
- DDPM loss: $L = E[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$ — noise prediction = approximate score
- Sinusoidal timestep embedding: unique fingerprint per t ; injected at every U-Net level
- U-Net: downsampling (context) + upsampling (detail) + skip connections (spatial info)
- DDPM: stochastic, ~1000 steps. DDIM: deterministic, 10-100 steps, same trained model.
- Diffusion families: DDPM, DDIM, Score-SDE, LDM, Guided, Consistency, Flow Matching

Advanced Diffusion Concepts

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Learning Objectives (from lecture)

- Understand the score function and its geometric meaning
- Explain classifier guidance from Bayes rule
- Explain classifier-free guidance (CFG) and why condition dropout is essential
- Understand why noise-aware classifiers are needed
- Understand FID and its limitations
- Understand the guidance tradeoff (realism vs diversity vs controllability)

1 — The Score Function: What the Network Really Learns

The **score function** is the gradient of the log-density:

$$s_t(x) = \nabla_x \log p_t(x)$$

The score points toward the direction of steepest increase in log-probability — toward regions of higher density. For a Gaussian it points toward the mean. For a multimodal distribution, it points toward the locally nearest mode.

The denoising-score identity connects noise prediction to score estimation:

$$\nabla_{x_t} \log p_t(x_t) \approx -\epsilon_{\theta}(x_t, t) / \sqrt{1 - \bar{\alpha}_t}$$

So even a noise-predicting network implicitly estimates the score field. The reverse process can be written as drift (denoising) + stochastic term:

$$x_{t-1} = x_t + \beta_t * s_t(x_t) + \sqrt{\beta_t} * z, z \sim N(0, I)$$

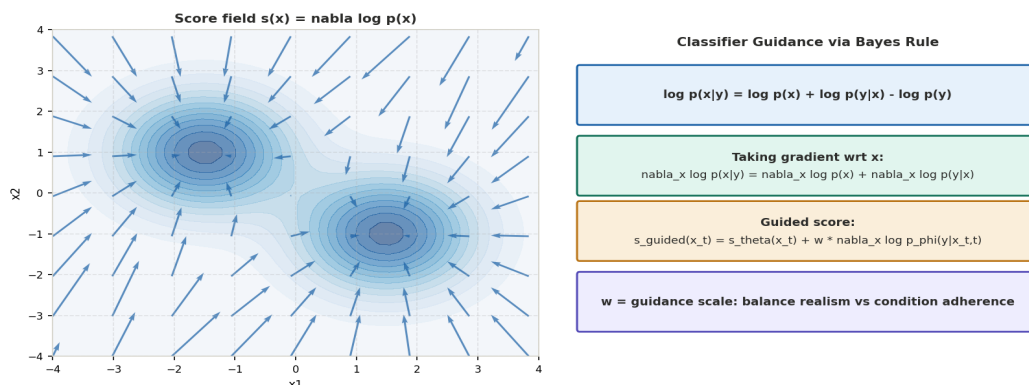


Figure 1 — Left: score field of a bimodal density — arrows point toward higher density. Right: classifier guidance derivation from Bayes rule — the guided score adds the classifier gradient.

2 — Classifier Guidance

To generate samples from $p(x|y)$, we use Bayes rule to derive a guided score:

$$\nabla_x \log p(x|y) = \nabla_x \log p(x) + \nabla_x \log p(y|x)$$

$$s_{\text{guided}}(x_t) = s_{\text{theta}}(x_t) + w * \nabla_{\{x_t\}} \log p_{\text{phi}}(y|x_t, t)$$

Why noise-aware classifiers are essential

During the reverse process, the classifier is queried on NOISY states x_t , not clean images.

A standard classifier trained on clean images produces meaningless gradients at high noise levels.

A noise-aware classifier is trained on (x_t, t) pairs from the same forward diffusion process.

This ensures its gradients are meaningful throughout the entire reverse trajectory.

3 — Classifier-Free Guidance (CFG)

CFG trains a single model for both conditional and unconditional generation by randomly dropping the condition during training (replacing with a null token). At sampling time:

$$\text{eps_cfg} = (1+w) * \text{eps_theta}(x_t, t, y) - w * \text{eps_theta}(x_t, t, \text{null})$$

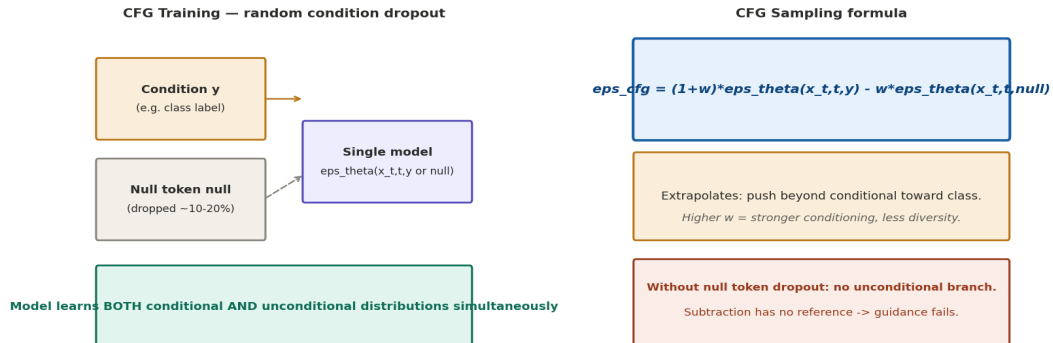


Figure 2 — CFG training (left): random condition dropout forces model to learn both branches. Sampling (right): guided formula extrapolates beyond conditional. Without null-token training, the unconditional branch does not exist and guidance fails.

CFG vs Classifier Guidance

Classifier guidance: needs a separate noise-aware classifier — extra training, extra complexity.

CFG: one model, condition dropout during training — dominant approach in modern systems.

Both share the tradeoff: higher w -> stronger conditioning -> less diversity -> possible artifacts.

Random dropout is essential: without it, the model has no unconditional branch to subtract.

4 — FID: Evaluation of Generative Models

$$FID = ||\mu_r - \mu_g||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{0.5})$$

FID measures how well generated images match real images in Inception feature space, comparing both means (location) and covariances (spread). Lower FID = better.

FID limitation	Why it matters
Depends on Inception features	Domain mismatch inflates FID unfairly (medical, satellite)
Assumes Gaussian in feature space	Feature distributions may be non-Gaussian
Sensitive to sample size	Small sample = noisy estimate
Misses controllability	A model can have good FID but poor prompt alignment
Misses diversity	A model can have good FID by memorising training data

Sample Exam Questions — with Model Answers

Q1 — What is the theoretical basis of classifier guidance?

Classifier guidance follows from Bayes rule: $\log p(x|y) = \log p(x) + \log p(y|x) - \log p(y)$. Taking gradients w.r.t. x : $\nabla_x \log p(x|y) = \nabla_x \log p(x) + \nabla_x \log p(y|x)$. The first term is the unconditional score from the diffusion model; the second comes from the classifier. Adding them with scale w produces a guided score balancing realism and alignment.

Q2 — Why must the classifier in classifier-guided diffusion be noise-aware?

The reverse chain queries the classifier on noisy states x_t , not clean data. A classifier trained only on clean images produces weak, unstable, or meaningless gradients when evaluated on heavily corrupted inputs at large t . A noise-aware classifier is trained on (x_t, t) pairs from the same forward diffusion process, making its gradients meaningful across all noise levels and consistent with the actual reverse trajectory.

Q3 — Why is random condition dropping essential for CFG?

CFG requires the model to produce both conditional $\text{eps}_{\theta}(x_t, t, y)$ and unconditional $\text{eps}_{\theta}(x_t, t, \text{null})$ predictions. The guided formula subtracts the unconditional from the conditional. Without dropout, the model never sees the null token and has no unconditional branch — the subtraction has no meaningful reference and guidance completely fails.

Q4 — Why does stronger guidance reduce diversity?

Guidance amplifies the component of the reverse update favouring the desired condition. As w increases, the sampler is pushed toward condition-consistent regions, suppressing trajectories that

do not strongly support the condition. The output distribution becomes more concentrated — better alignment but less diversity and potential artifacts if guidance overwhelms the learned unconditional structure.

Q5 — What does FID measure and what are its three main limitations?

FID measures the Frechet distance between real and generated samples in Inception feature space, comparing mean and covariance of feature distributions. Lower = generated samples match real statistics. Three limitations: (1) depends on Inception extractor — domain mismatch inflates FID unfairly; (2) assumes Gaussian distributions in feature space — may not hold; (3) sensitive to sample size and numerical estimation. Also does not capture controllability or diversity.

Key Concepts Summary

- Score: $s_t(x) = \nabla_x \log p_t(x)$ — direction toward higher density
- Denoising-score identity: $\nabla \log p_t \approx -\epsilon_{\theta}(x_t, t) / \sqrt{1 - \bar{\alpha}_t}$
- Classifier guidance (Bayes): $s_{\text{guided}} = s_{\text{uncond}} + w \cdot \nabla \log p_{\phi}(y|x_t, t)$
- Noise-aware classifier: trained on (x_t, t) pairs — essential for stable guidance
- CFG: one model, condition dropped w/ null token during training
- CFG formula: $\epsilon_{\text{cfg}} = (1+w) \cdot \epsilon_{\text{cond}} - w \cdot \epsilon_{\text{uncond}}$
- Null token dropout is essential — without it, no unconditional branch exists
- Guidance tradeoff: higher $w \rightarrow$ better condition adherence $\rightarrow$ less diversity $\rightarrow$ artifacts
- FID: Frechet distance in Inception space (lower=better) — necessary but insufficient
- FID limitations: feature extractor, Gaussian assumption, sample size, misses controllability

Flow Matching: SDE Connection & Consistency Models

Additional content from flashcards — ECE786 Spring 2026

What this addendum adds (flashcard gaps from original PDF)

- Continuous-time SDE form: $dx = f(x,t)dt + g(t)dW$ — the general framework underlying diffusion.
- Variance-Preserving (VP) property: $a(t)^2 + s(t)^2 = 1$ — what it means and why it matters.
- DDPM as a finite-step discretisation of the VP-SDE.
- Log-density evolution along a trajectory via the continuity equation.
- Consistency model stop-gradient loss, boundary condition, log-SNR sampling, trivial failure mode.
- Teacher distillation for consistency models.

A — Continuous-Time SDEs and the Connection to Diffusion

The general continuous-time stochastic differential equation (SDE) framework unifies diffusion, flow matching, and score-based models:

$$dx = f(x,t) dt + g(t) dW_t$$

where $f(x,t)$ is the drift term (deterministic direction) and $g(t)dW_t$ is the diffusion term (stochastic noise, Wiener process). Setting $g(t)=0$ recovers an ODE — this is exactly the flow matching case.

Variance-Preserving (VP) SDEs

A VP-SDE enforces that signal and noise variances sum to 1 at every t :

$$a(t)^2 + s(t)^2 = 1 \text{ for all } t \text{ in } [0, T]$$

This gives the marginal: $x_t = a(t)x_0 + s(t)\epsilon$, $\epsilon \sim N(0, I)$. The VP property ensures that if x_0 has unit variance, x_t also has unit variance at every t — the total "energy" is preserved even as signal is replaced by noise.

DDPM as VP-SDE discretisation

DDPM is a finite-step discretisation of the continuous VP-SDE. The α -bar schedule in DDPM corresponds to $a(t)^2$, and $(1 - \alpha\text{-bar})$ corresponds to $s(t)^2$. The continuous SDE viewpoint explains why DDPM can be sampled deterministically (DDIM) — one simply drops the stochastic term in the

reverse SDE.

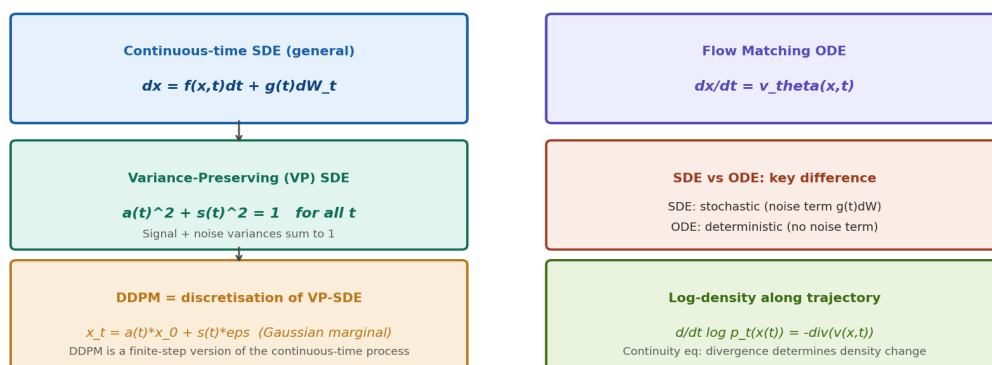


Figure A1 — SDE framework hierarchy. General SDE (top left) → VP-SDE (middle, $a^2+s^2=1$) → DDPM discretisation (bottom). Flow matching ODE (top right) has no stochastic term. Log-density evolution (bottom right): $d/dt \log p_t = -\text{div}(v)$.

Log-density along a trajectory (flashcard question)

Along a particle trajectory $x(t)$, the log-density evolves as:

$$d/dt \log p_t(x(t)) = -\text{div}(v(x(t), t)) = -\text{nabla} \cdot v$$

Positive divergence: probability mass flows outward → density decreases.

Negative divergence: mass converges → density increases.

This is the continuity equation in log-density form, not just probability flux.

B — Consistency Models: Full Flashcard Coverage

Consistency models learn a map $f_{\theta}(x_t, t)$ that is approximately constant along the forward noising trajectory — mapping any noisy state directly to the corresponding clean sample x_0 .

Consistency condition: $f_{\theta}(x_s, s) = f_{\theta}(x_t, t)$ for any $s < t$ on the same trajectory

Mapping target: f_{θ} maps any $(x_t, t) \rightarrow$ canonical x_0 . This enables one-step or few-step generation.

Training details (from flashcards)

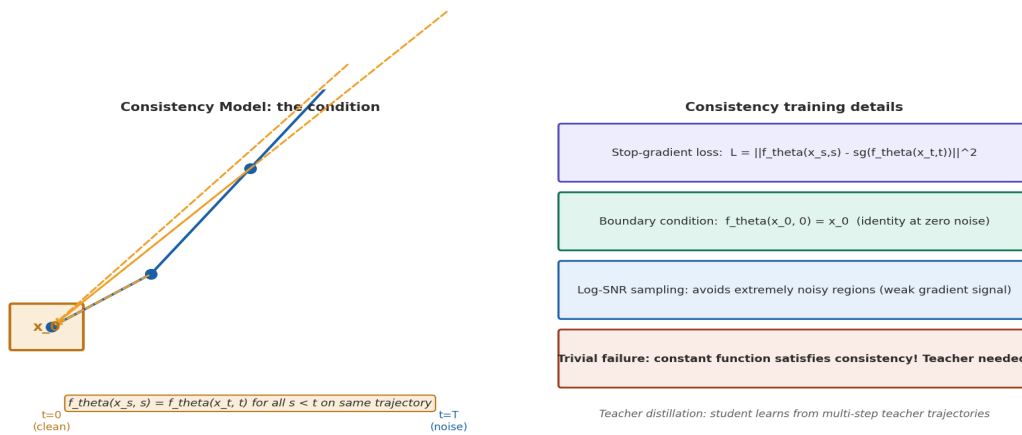


Figure A2 — Left: consistency condition — all noisy states on the same trajectory map to the same x_0 (orange dashed arrows). Right: four training details from flashcards — stop-gradient loss, boundary condition, log-SNR sampling, trivial failure.

Detail	Formula / explanation	Why it matters
Stop-gradient loss	$L = \ f_{\theta}(x_s, s) - \text{sg}(f_{\theta}(x_t, t))\ ^2$	Prevents gradient through target side — stabilises training
Boundary condition	$f_{\theta}(x_0, 0) = x_0$	Anchors model at zero noise — must be identity there
Log-SNR sampling	Sample t in log-SNR space, not linear	Avoids oversampling high-noise regions where gradient signal is too weak
Trivial failure	Constant function satisfies consistency	Model can collapse without teacher or other constraint
Teacher distillation	Student matches teacher's multi-step trajectories	Teacher provides non-trivial anchor to prevent collapse
Teacher posterior	$p_{\theta}(x_0 x_t, t)$: model's estimate of clean state	Used to implement teacher denoise step for consistency distillation

Consistency distillation vs consistency training

Consistency training: train from scratch to satisfy the consistency condition — harder, requires careful design.

Consistency distillation: student learns to match a pretrained diffusion teacher — easier, teacher provides meaningful anchor.

Teacher denoise step: sample clean candidate x_0 from teacher posterior $p_{\theta}(x_0|x_t, t)$, then re-noise to time s .

This gives: (x_s, s) and (x_t, t) pairs from the same trajectory for the consistency loss.

Key concepts

- Continuous-time SDE: $dx = f(x, t)dt + g(t)dW_t$ (general framework)
- VP-SDE property: $a(t)^2 + s(t)^2 = 1$ — signal + noise variance always sum to 1
- DDPM = finite-step discretisation of VP-SDE
- Flow matching = SDE with $g(t)=0$ (no noise term) $\rightarrow$ deterministic ODE
- Log-density along trajectory: $d/dt \log p_t(x(t)) = -\text{div}(v(x, t))$
- Consistency: $f_{\theta}(x_s, s) = f_{\theta}(x_t, t) \rightarrow$ maps any noisy state to x_0
- Boundary: $f_{\theta}(x_0, 0) = x_0$ — identity at zero noise (must be enforced)
- Stop-gradient loss: $L = \|f_{\theta}(x_s, s) - \text{sg}(f_{\theta}(x_t, t))\|^2$

- Log-SNR sampling: avoids extreme noise where gradient signal vanishes
- Trivial failure: constant function satisfies consistency without learning anything useful
- Consistency distillation: student matches pretrained diffusion teacher trajectories

This addendum should be read alongside the Chapter 05 main PDF.

Flow Matching

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Core idea

- Flow matching reframes generation as ODE-based transport: particles flow from noise to data.
- Learns a time-dependent velocity field $v_{\theta}(x, t)$ via supervised regression on known bridge velocities.
- Unlike diffusion, the training target is a known velocity — not an unknown density gradient.
- The continuity equation shows flow matching is fundamentally about transporting probability mass.
- OT couplings, latent FM, and consistency models are important extensions in this family.

1 — The Core Idea: ODE-Based Transport

Flow matching learns a time-dependent velocity field $v_{\theta}(x, t)$ which defines an Ordinary Differential Equation (ODE):

$$dx/dt = v_{\theta}(x, t), t \text{ in } [0, 1]$$

Particles initialised at $x(0) \sim p_0$ (typically Gaussian noise) follow the learned dynamics so that $x(1) \sim p_{\text{data}}$. Generation is integrating this ODE from noise to data. Think of v_{θ} as a "wind field" — at each location x and time t , it tells the particle which direction to move and how fast.

The training objective

A bridge (interpolation path) between $x_0 \sim p_0$ and $x_1 \sim p_{\text{data}}$ is defined. For the simplest **linear bridge**:

$$x_t = (1 - t) * x_0 + t * x_1$$

The target velocity is known analytically:

$$u_t = dx_t/dt = x_1 - x_0$$

Training minimises:

$$L(\theta) = E_{\{x_0, x_1, t\}} [\| v_{\theta}(x_t, t) - u_t \|^2]$$

The elegance: the training target u_t is known — it is the instantaneous velocity along the chosen bridge. No score estimation required. No intractable integrals.

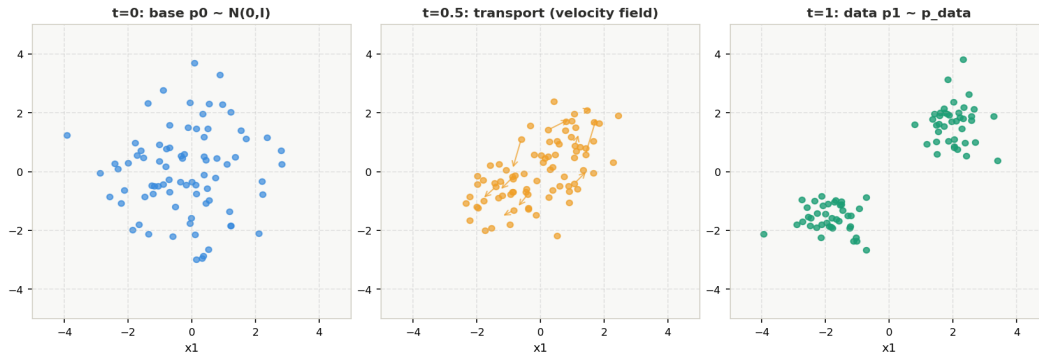


Figure 1 — Flow matching transport. Left: particles at $t=0$ sampled from noise. Middle: particles transported by learned velocity field at $t=0.5$ (arrows show velocity). Right: particles arrive at data distribution at $t=1$.

The continuity equation — why this is mass transport, not just regression

If $p_t(x)$ is the density of particles at time t , and particles move with velocity $v(x,t)$, then:
 $\frac{dp_t}{dt} + \text{div}(p_t * v) = 0$ — the continuity equation.

Probability mass is conserved: it is transported through space, never created or destroyed.

Flow matching is fundamentally about moving a distribution, not just fitting point-to-point vectors.

2 — Flow Matching vs Diffusion

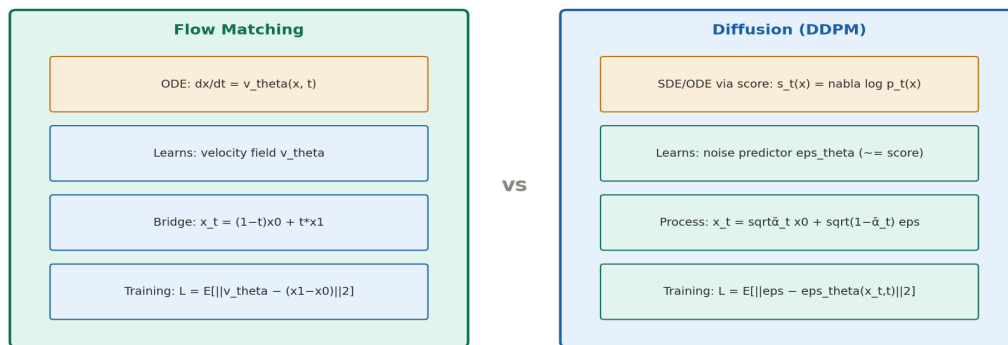


Figure 2 — Flow matching (left) vs diffusion (right). Key difference: flow matching learns a velocity field via direct regression; diffusion learns a noise predictor that approximates the score. Euler integration vs stochastic/deterministic reverse chain.

Aspect	Flow Matching	Diffusion (DDPM/DDIM)
Learned object	Velocity field $v_\theta(x,t)$	Noise predictor $\text{eps}_\theta(x_t, t) \approx \text{score}$
Training target	Known bridge velocity u_t	Unknown score (via noise proxy)
Forward process	Deterministic bridge	Stochastic Markov chain
Sampling	Euler ODE integration	DDPM: stochastic / DDIM: deterministic
Steps needed	Few (ODE solver)	Many (DDPM) or few (DDIM)

Aspect	Flow Matching	Diffusion (DDPM/DDIM)
Score estimation	Not required	Implicit via noise prediction

Sampling via Euler integration

Once v_{θ} is trained, sample by integrating the ODE numerically:

$$x_{n+1} = x_n + \Delta t * v_{\theta}(x_n, t_n)$$

This approximates $x(1) = x(0) + \int_0^1 v_{\theta}(x(t), t) dt$ via a Riemann sum. More sophisticated ODE solvers (Runge-Kutta etc.) can be used for better accuracy with fewer steps.

3 — Optimal Transport Couplings

In naive flow matching, x_0 and x_1 are paired independently (random coupling). This can create unnecessarily long or crossing trajectories, making the velocity field harder to learn. **OT-based flow matching** instead finds the coupling π^* that minimises expected transport cost:

$$\pi^* = \operatorname{argmin}_{\pi \in \operatorname{PROD}(p_0, p_1)} E_{\{(x_0, x_1) \sim \pi\}} [\|x_0 - x_1\|_2]$$

Importantly: the bridge formula stays the same (linear). OT only changes *which* x_0 is paired with which x_1 . Better pairings $\rightarrow$ shorter, more coherent paths $\rightarrow$ simpler velocity field $\rightarrow$ easier learning.

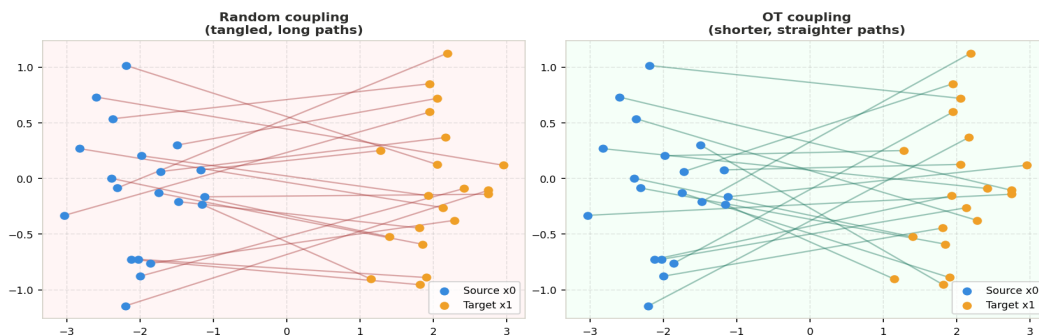


Figure 3 — Random coupling (left): tangled, long transport paths — complex velocity field. OT coupling (right): shorter, straighter paths — simpler velocity field to learn. Same bridge formula, different source-target pairings.

4 — Latent Flow Matching and Conditional Generation

Latent Flow Matching (LFM)

Instead of learning transport in high-dimensional pixel space, LFM first trains an encoder-decoder (autoencoder) and then learns a velocity field in the latent space:

$$z = E_{\psi}(x), \hat{x} = D_{\psi}(z) \rightarrow \text{learn } v_{\theta}(z, t) \text{ in latent space}$$

Advantages: latent space is lower-dimensional, semantically cleaner, and transport paths are often simpler. Disadvantage: sample quality is capped by the autoencoder — if it discards important information, the decoder becomes a bottleneck.

Conditional generation and CFG in flow matching

Conditional flow matching simply conditions the velocity field on y : $dx/dt = v_{\theta}(x, t, y)$. CFG in flow matching works analogously to diffusion CFG — the model is trained with random condition dropping, and sampling uses a guided velocity:

$$v_{\text{cfg}}(x, t, y) = v_{\theta}(x, t, \emptyset) + s * (v_{\theta}(x, t, y) - v_{\theta}(x, t, \emptyset))$$

The guided object is a velocity field rather than a score estimate, but the conceptual logic is identical: extrapolate beyond the conditional in the direction that increases class alignment.

Consistency models

Consistency models learn a map $f_{\theta}(x_t, t)$ that is approximately constant along the flow trajectory — mapping any noisy state to the corresponding clean sample:

$$f_{\theta}(x_s, s) \approx f_{\theta}(x_t, t) \text{ for any } s < t \text{ on the same trajectory}$$

This enables one-step or few-step generation. However, consistency is a fragile objective — a trivial constant function satisfies it. Teacher-student training (with a diffusion teacher) is essential to prevent collapse.

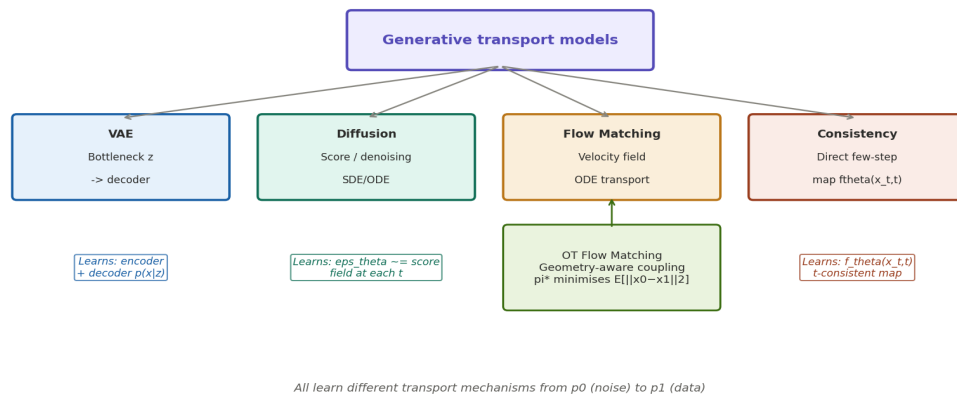


Figure 4 — The generative model family. All methods learn different transport mechanisms from p_0 (noise) to p_1 (data): VAE via bottleneck, diffusion via score/denoising, flow matching via velocity field, OT-FM via geometry-aware coupling, consistency models via direct few-step map.

Sample Exam Questions

Q1 — Why is flow matching best described as a transport model rather than just a regression model?

At the surface, flow matching minimises $L = E[\|v_{\theta} - u_t\|_2^2]$, which looks like simple regression. But the deeper object is the induced dynamics $dx/dt = v_{\theta}(x, t)$, which transports an entire probability distribution from p_0 to p_1 . The governing population-level equation is the continuity equation: $dp_t/dt + \nabla \cdot (p_t v) = 0$. The regression target is only the training mechanism; the learned object is a transport field that moves probability mass. Flow matching is about distribution transport, not just pointwise vector fitting.

Q2 — What is the role of the interpolation bridge and why is it not a trivial design choice?

The bridge $x_t = l(x_0, x_1, t)$ defines the intermediate states the model is trained on and the velocity target $u_t = dx_t/dt$. Different bridges change: (1) the distribution of training inputs x_t , (2) the target velocity at those inputs, (3) the complexity of the velocity field required for accurate transport. A linear bridge is simple but may produce curved-looking trajectories in data space; alternative bridges can make paths straighter and learning easier. The bridge is a modeling assumption about how mass should move, not a harmless implementation detail.

Q3 — Why can latent flow matching be preferable to pixel-space flow matching?

Pixel space is high-dimensional, contains much low-level variation, and requires the model to capture fine nuisance structure. Learning a velocity field directly in pixel space is computationally expensive and may be harder. Latent FM defines an encoder $z = E(x)$ and learns transport in the lower-dimensional latent space: $dz/dt = v_{\theta}(z, t)$. This can be preferable because dimensionality is reduced, semantic structure is more regular, and transport paths may be simpler. The gain depends on the quality of the autoencoder.

Q4 — Why do OT couplings help even when the bridge formula stays linear?

Even with the same linear bridge $x_t = (1-t)x_0 + tx_1$, the pairing (x_0, x_1) determines transport geometry. Random pairings can create crossing, unnecessarily long paths, forcing the model to learn a complex velocity field. OT couplings minimise $E[\|x_0 - x_1\|^2]$, yielding shorter, more coherent displacement directions. The bridge formula is unchanged — OT changes the supervision geometry by selecting better training pairs, making the velocity field simpler to learn and sampling more stable.

Q5 — Why are consistency models conceptually attractive but also fragile?

Consistency models are attractive because they aim for $f_{\theta}(x_t, t) \approx f_{\theta}(x_s, s)$ along the same trajectory, enabling one-step or few-step generation — much faster than diffusion or flow models. They are fragile because consistency alone does not guarantee correctness. A trivial constant function satisfies the consistency condition without generating meaningful samples. High-noise states can also drive the model toward conditional means, causing mode collapse. Teacher-student training with a strong diffusion or flow teacher is essential to provide a non-trivial anchor that forces the student to learn a meaningful canonicalisation map.

Key Concepts Summary

Key concepts

- Flow matching: learns velocity field $v_{\theta}(x, t)$ via ODE $dx/dt = v_{\theta}(x, t)$
- Linear bridge: $x_t = (1-t)x_0 + tx_1$, target velocity $u_t = x_1 - x_0$
- Training: $L = E[\|v_{\theta}(x_t, t) - u_t\|^2]$ — supervised regression on known velocity
- Continuity equation: $dp_t/dt + \nabla \cdot (p_t v) = 0$ — flow matching is mass transport
- Euler integration: $x_{n+1} = x_n + \Delta t v_{\theta}(x_n, t_n)$ for sampling
- OT coupling: minimise $E[\|x_0 - x_1\|^2]$ — shorter paths, simpler velocity field
- Latent FM: learn v_{θ} in autoencoder latent space z — lower dimension, cleaner structure

- CFG in FM: $v_{\text{cfg}} = v_{\text{uncond}} + s^*(v_{\text{cond}} - v_{\text{uncond}})$ — same idea as diffusion CFG
- Consistency models: $f_{\theta}(x_t, t) \approx f_{\theta}(x_s, s)$ — fast but fragile without teacher
- Key insight: VAE, diffusion, FM, consistency all learn different transport mechanisms from p_0 to p_1

Next: Chapter 06 — Transformers and LLMs: autoregressive sequence modelling.

Transformers & Large Language Models

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Learning Objectives (from lecture)

- Understand what an autoregressive model is
- Describe how RNNs work and their limitations
- Understand LSTM and how it fixes the vanishing gradient problem
- Describe the basic ideas behind the Transformer architecture
- Understand Q, K, V projections and multi-head attention
- Understand causal masking and why it is essential
- Understand sinusoidal positional encoding
- Understand Byte-level BPE tokenisation

1 — Autoregressive Models: The Core Idea

An autoregressive model defines a joint probability over a sequence by the chain rule of probability. This is simultaneously a probabilistic statement (valid likelihood) and a practical training strategy (each position is a supervised next-token prediction problem):

$$p(x_1, x_2, \dots, x_T) = \text{PRODUCT}_{t=1}^T p(x_t | x_{\text{prev}}) = \text{PRODUCT}_{t=1}^T p(x_t | x_{1:t})$$

Generation follows the same factorisation: sample x_1 , condition on it to sample x_2 , etc. Training maximises the log-likelihood, which decomposes into a sum of per-position cross-entropy terms:

$$L = -\sum_t \log p_{\theta}(x_t | x_{1:t}) \text{ (cross-entropy at each position)}$$

2 — RNNs and Their Bottleneck

Before Transformers, sequence modelling used Recurrent Neural Networks (RNNs). An RNN processes one token at a time, maintaining a hidden state h_t :

$$h_t = f(W_h * h_{t-1} + W_x * x_t + b)$$

$$p(x_{t+1} | x_{1:t}) = \text{softmax}(W_y * h_t + b_y)$$

RNN limitations

1. Bottleneck: ALL past information must be compressed into one fixed-size vector h_t .
2. Sequential: cannot parallelise — step t depends on step $t-1$.
3. Vanishing gradients: BPTT (Backpropagation Through Time) propagates gradients through many steps -> they vanish or explode.

4. Long-range dependencies are unreliable — earlier tokens are increasingly forgotten.

3 — LSTM: Gated Memory to Fix Vanishing Gradients (Lecture Section)

The LSTM (Long Short-Term Memory) introduces a dedicated cell state c_t — a "memory highway" — with three gates that learn when to write, forget, and reveal information:

*Forget gate: $f_t = \text{sigma}(W_f * [h_{t-1}, x_t] + b_f)$ — what to erase from c_{t-1}*

*Input gate: $i_t = \text{sigma}(W_i * [h_{t-1}, x_t] + b_i)$ — what new info to write*

*Candidate: $g_t = \text{tanh}(W_g * [h_{t-1}, x_t] + b_g)$ — candidate content to write*

*Cell update: $c_t = f_t * c_{t-1} + i_t * g_t$*

*Output gate: $o_t = \text{sigma}(W_o * [h_{t-1}, x_t] + b_o)$ — what to reveal*

*Hidden state: $h_t = o_t * \text{tanh}(c_t)$*

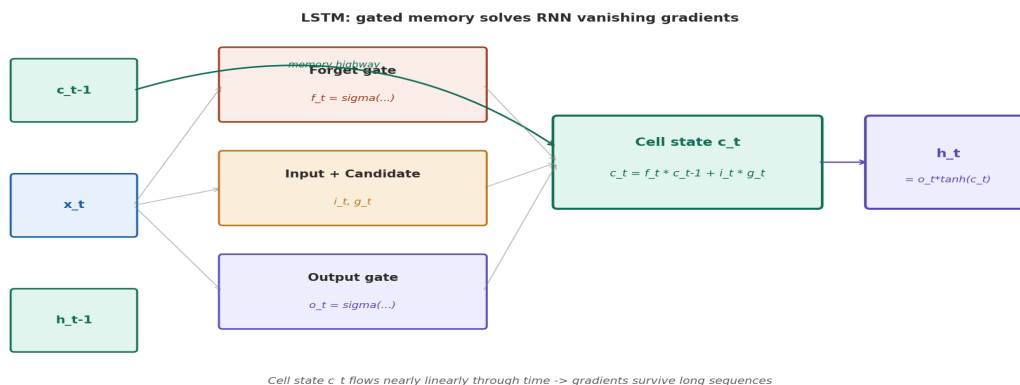


Figure 1 — LSTM architecture. Three gates (forget, input, output) control information flow. Cell state c_t is a "memory highway" — gradients flow through it with minimal decay. Gates are sigmoid units: output in $(0, 1)$, acting as soft on/off switches.

Why LSTM fixes vanishing gradients

The cell state $c_t = f_t * c_{t-1} + i_t * g_t$ has a gradient path that can be nearly linear.

If $f_t \approx 1$ (forget gate open): gradient flows back through c_{t-1} almost unchanged.

The gates learn WHEN to let gradients pass — unlike vanilla RNNs which always apply $\tanh$.

But: LSTMs still process sequentially, limiting parallelism during training.

4 — Why Transformers Replace RNNs

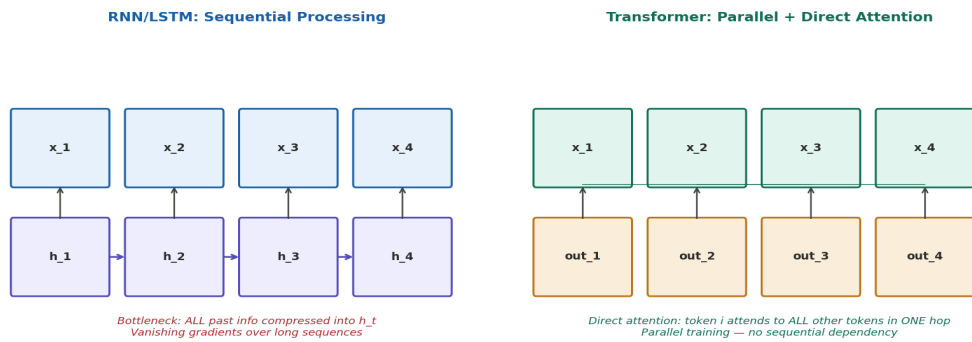


Figure 2 — RNN (left): sequential, bottleneck hidden state, limited parallelism. Transformer (right): every token attends directly to every other in one layer — no sequential bottleneck.

The Transformer's key insight: a token should attend directly to all relevant past tokens, not through a compressed hidden state. This requires **self-attention**.

5 — Self-Attention: Q, K, V Projections

Each token produces three vectors via learned projections W_Q, W_K, W_V :

Vector	Role	Question it answers
Q (Query)	What information am I looking for?	What does this position need?
K (Key)	What information do I offer for matching?	How should others find me?
V (Value)	What content do I contribute?	What do I actually pass forward?

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

The $\sqrt{d_k}$ scaling prevents the dot products from becoming too large at high dimensions, which would push softmax into saturation (near-zero gradients).

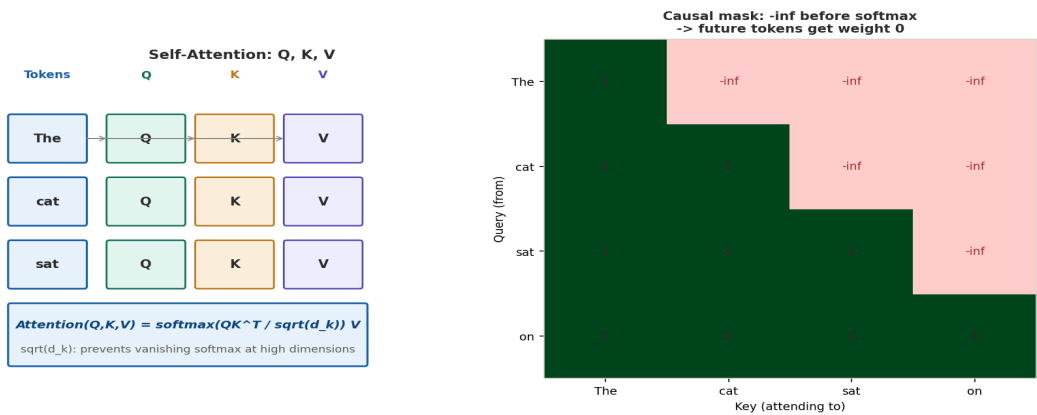


Figure 3 — Left: Q, K, V projections for each token; attention formula. Right: causal mask — upper triangle set to -inf before softmax, so future tokens get weight 0.

Why separate Q, K, V — not one vector?

If one vector served all roles, the model loses flexibility to distinguish matching from content.

Q/K learn a geometry for relevance matching; V learns a geometry for content to retrieve.

This makes attention a learned content-based retrieval mechanism, not just a similarity average.

Multi-head attention: run h attention heads in parallel with different W_Q^i , W_K^i , W_V^i projections.

Each head can specialise: one tracks syntax, another long-range semantics, another position.

6 — The Full Transformer Block

The Transformer block combines attention with a per-token FFN. The FFN expands then contracts: $d \rightarrow 4d \rightarrow d$, using GELU activation (not ReLU — smoother, empirically better in Transformers). **Residual connections** (skip connections) allow gradients to bypass layers and prevent information loss. **LayerNorm** normalises within each token independently, stabilising activations for deep stacks.

Pre-LN block order (used in modern GPT-style models):

$$x' = x + \text{MHA}(\text{LayerNorm}(x))$$

$$\text{output} = x' + \text{FFN}(\text{LayerNorm}(x'))$$

Without residual connections: gradients vanish in deep stacks. Without LayerNorm: activations drift to very large or very small scales, making attention/FFN sublayers unstable.

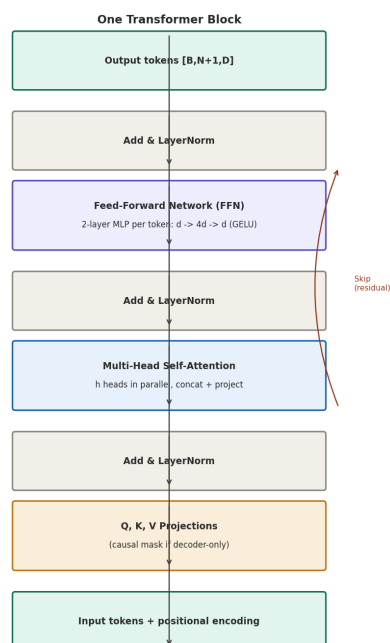


Figure 4 — One Transformer block. Layers bottom to top: input tokens, Q/K/V projections, MHA, Add+LN, FFN, Add+LN, output. Residual skip connections bypass each sublayer.

7 — Causal Masking: Enforcing the Autoregressive Constraint

In a decoder-only LM (GPT-style), token t must only attend to positions up to t . This is enforced by setting attention scores to $-\infty$ for future positions before the softmax:

```
causal_mask = torch.tril(torch.ones(T, T)).bool()
```

```
masked_scores = scores.masked_fill(~causal_mask, float("-inf"))
```

Why causal masking is essential

Without it: model sees the token it must predict during training $\rightarrow$ trivially easy, zero loss.

The model would NOT learn a valid autoregressive conditional distribution.

At generation time: no future tokens exist $\rightarrow$ the model would fail completely.

The mask ensures training objective matches generation-time constraints.

Encoder-only models (BERT): NO causal mask — bidirectional attention for understanding tasks.

8 — Positional Encodings: Sinusoidal (Lecture)

Self-attention is permutation-invariant — without positional information, the model cannot distinguish "The cat sat" from "sat cat The". Sinusoidal encodings provide a unique fingerprint for each position:

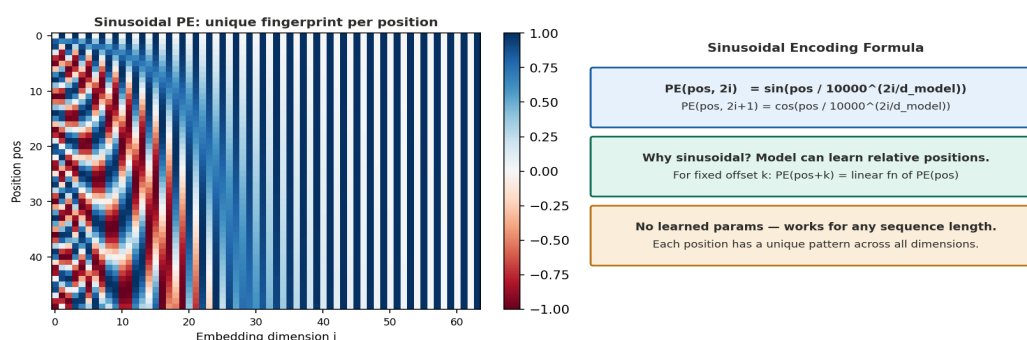


Figure 5 — Left: heatmap of sinusoidal PE — each position has a unique pattern across dimensions. Right: formula and key properties. Relative offsets are encoded: $PE(pos+k) = \text{linear fn of } PE(pos)$.

9 — BPE Tokenisation: Byte-Pair Encoding (Lecture Section 5)

Language models operate on tokens — subword units that balance vocabulary size against sequence length. **Byte-Pair Encoding (BPE)** builds a vocabulary by iteratively merging the most frequent adjacent symbol pairs:

Byte-Pair Encoding (BPE) — merge algorithm

Initial (chars)	l o w e r _ l o w e s t _
Count pairs	lo:2, ow:2, we:2, er:1, r_:1, es:1...
Merge "lo"	l o w e r _ l o w e s t _
Merge "low"	l o w e r _ l o w e s t _
Final vocab	low, er, est, _ , lower_, lowest_ ... GPT-2 uses Byte-level BPE: base vocab = 256 byte values (handles ANY Unicode)

Figure 6 — BPE algorithm. Starting from characters, the most frequent adjacent pairs are merged iteratively until reaching the target vocabulary size. GPT-2 uses byte-level BPE with 256 base byte values — handles all Unicode without unknown tokens.

Why tokenisation matters (not just preprocessing)

Sequence length: character-level = very long sequences; word-level = unknown words; BPE = balanced.

Granularity of meaning: tokens shape what patterns the model can learn.

Efficiency: shorter sequences -> cheaper attention ($O(N^2)$).

Byte-level BPE (GPT-2): base vocabulary = all 256 byte values -> handles ANY Unicode without OOV.

The merge vocabulary is learned from the training corpus — domain-specific merges emerge.

10 — Transformer Family

Architecture	Attention type	Used for	Examples
Encoder-only	Bidirectional (no mask)	Understanding: classification, tagging, retrieval	BERT, RoBERTa
Decoder-only	Causal (autoregressive)	Generation: LLMs, completion	GPT-2/3/4, LLaMA
Encoder-decoder	Enc: bidir; Dec: causal + cross-attn	Seq-to-seq: translation, summarisation	T5, BART

Sample Exam Questions — with Model Answers

Q1 — Explain why the autoregressive factorisation is both a probabilistic statement and a training strategy.

As a probabilistic statement: the chain rule of probability is always valid — it expresses the joint probability of the whole sequence in terms of conditional distributions. The model assigns well-defined likelihoods to complete sequences. As a training strategy: maximising the joint log-likelihood decomposes into maximising each conditional $\log p(x_{t+1}|x_{1:t})$ separately. This converts sequence modelling into many supervised next-token prediction problems, one at each position —

computationally tractable and naturally parallelisable within a forward pass.

Q2 — Why do LSTMs have three gates, and what does each one control?

The forget gate $f_t = \text{sigma}(W_f[h_{t-1}, x_t] + b_f)$ controls what to erase from the previous cell state c_{t-1} . The input gate i_t and candidate g_t together control what new information to write to the cell state. The output gate o_t controls how much of the cell state to reveal as the hidden state h_t . Together, they allow the cell state to act as a memory highway where gradients can flow with minimal decay — unlike vanilla RNNs where the tanh nonlinearity always compresses gradients.

Q3 — Why do we need separate Q, K, V projections in self-attention?

Q represents what a token is looking for, K represents what it offers for matching, and V represents the content to be retrieved. Separate projections allow the model to learn different geometric spaces for matching (Q/K) and for content retrieval (V). If one vector served all three roles, the model would have less freedom — it must simultaneously be a good query, a good key, and carry useful content in the same representation. Separation makes attention a flexible learned retrieval mechanism rather than simple similarity averaging.

Q4 — Why is causal masking essential? What breaks without it?

Causal masking prevents token t from attending to future positions, ..., T during training. Without it, the model can directly look at the token it must predict — training loss would be near-zero trivially, and the model would not learn a valid autoregressive conditional distribution. At generation time, future tokens are unavailable, so the model would fail entirely. The mask ensures the training objective exactly matches the generation-time setting.

Q5 — Why is BPE tokenisation not a minor preprocessing detail?

BPE determines sequence length (affects $O(N^2)$ attention cost), the granularity of meaning the model can represent, training efficiency, and how rare or unseen words are handled. Character-level: very long sequences; word-level: unknown words at test time; BPE: balanced tradeoff. GPT-2's byte-level BPE with 256 base tokens handles any Unicode input without out-of-vocabulary issues. The learned merges are corpus-dependent — domain-specific subwords emerge naturally. Different tokenisations produce different learned representations for the same text.

Key Concepts Summary

- Autoregressive: $p(x_{1:T}) = \text{PRODUCT } p(x_t | x_{1:t})$ — chain rule of probability
- RNN: $h_t = f(W_h h_{t-1} + W_x x_t)$ — bottleneck, sequential, vanishing gradients
- LSTM gates: forget (erase), input+candidate (write), output (reveal) — cell state is memory highway
- LSTM: $c_t = f_t * c_{t-1} + i_t * g_t$; $h_t = o_t * \tanh(c_t)$ — fixes RNN vanishing gradient
- Self-attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
- $\sqrt{d_k}$ scaling: prevents softmax saturation at high dimensions
- Q=what I seek, K=what I offer, V=what I carry — separate for flexible retrieval

- Multi-head: h parallel attention heads $\rightarrow$ captures diverse relationship types simultaneously
- FFN: per-token 2-layer MLP ($d \rightarrow 4d \rightarrow d$, GELU) — transforms info WITHIN each token
- Residual + LayerNorm: enable deep stacks — functional necessity, not cosmetic
- Causal mask: $-\infty$ for future positions $\rightarrow$ autoregressive constraint enforced
- Sinusoidal PE: $PE(pos, 2i) = \sin(pos/10000^{(2i/d)})$; no params; encodes relative positions
- BPE: merge most frequent adjacent pairs; byte-level in GPT-2 $\rightarrow$ no OOV, handles all Unicode
- Encoder-only=bidir; Decoder-only=causal; Enc-Dec=both

Vision Transformers (ViT)

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Core ideas — including flashcard topics

- ViT treats an image as a sequence of patch tokens: $N = HW/P^2$ patches, each projected to D -dim.
- [CLS] token + positional embeddings added before Transformer encoder.
- Bidirectional self-attention enables global interaction from layer 1.
- ViT evolution: ViT (2021) → DeiT → Swin → MAE → DiT. Each adds a different capability.
- DiT replaces the U-Net in diffusion — scales strongly with model size (larger > smaller + more steps).
- MaskGIT: bidirectional transformer for image synthesis via masked visual token prediction.
- Evaluation: FID (distribution), Precision (realism), Recall (diversity), IS (fidelity + diversity).
- Positional encodings: learned absolute, sinusoidal, relative bias (Swin), RoPE (rotation).

1 — From Image to Tokens: The Core ViT Pipeline

ViT converts an image into a sequence of patch tokens, adds a class token and positional embeddings, passes through L encoder blocks, and reads the CLS token for classification.

$$N = HW/P^2 \mid X_{\text{patch}} \text{ in } \mathbb{R}^{N \times D} \mid X_0 = [x_{\text{cls}}; x_1 \dots x_N] + P \mid \text{logits} = W_{\text{cls}} * x_{\text{cls}}^{\wedge(L)}$$

Full ViT pipeline — tensor shapes at each stage

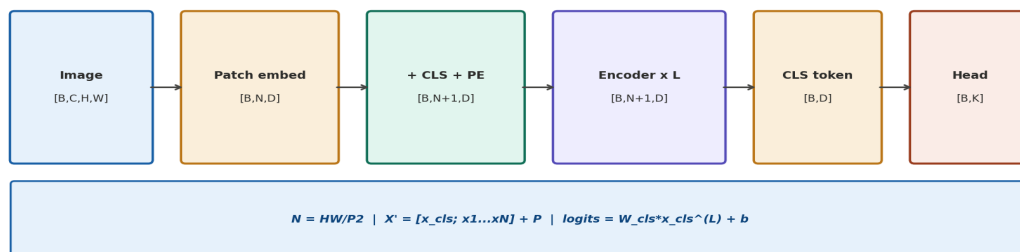


Figure 1 — Full ViT pipeline with tensor shapes at every stage. Patch embedding (Conv2D with kernel=stride=P) → prepend CLS → add PE → L encoder blocks → read CLS → classification head.

Flashcard: patch embedding as convolution

Patch embedding = $\text{Conv2d}(\text{in_channels}, \text{embed_dim}, \text{kernel_size}=P, \text{stride}=P)$.

This simultaneously extracts non-overlapping patches AND projects to D channels.

Tensor flow: $[B, C, H, W] \rightarrow [B, D, H/P, W/P] \rightarrow \text{flatten} \rightarrow \text{transpose} \rightarrow [B, N, D]$.

In code, after flattening: shape is $[B, D, N]$ — must transpose to $[B, N, D]$ for the transformer.

2 — Positional Encodings: Four Types You Must Know

Self-attention is permutation-invariant — without positional information, the model cannot distinguish spatial layout. Four main approaches are tested in the flashcards:

Learned Absolute	Sinusoidal Absolute	Relative (Swin bias)	RoPE (rotation)
Fixed lookup table	sin/cos formula	Bias = $f(i-j, i-j)$	Rotate Q and K by position angle
per position	position-invariant	encodes relative offset	Encodes relative position via inner product
Fails if resolution changes	Cannot adapt	Used in Swin Transformer	Used in LLMs and modern ViTs

Figure 2 — Four positional encoding types. Learned absolute: lookup table per position (fails if resolution changes). Sinusoidal: fixed formula, position-invariant. Relative bias (Swin): encodes offset between tokens. RoPE: rotates Q and K vectors by position angle before dot product.

Type	Formula / key idea	Limitation	Used in
Learned absolute	Fixed lookup table $\text{PE}[\text{pos}]$	Fails if resolution changes — interpolation needed	Original ViT
Sinusoidal	$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)})$	Cannot adapt to task	Early transformers
Relative bias	Bias added to attn logits = $f(\Delta \text{tarow}, \Delta \text{tacol})$	Larger memory	Swin Transformer
RoPE	Rotate Q, K by angle $\theta * \text{pos}$ before dot product	More complex impl.	LLaMA, modern ViTs

Why 2D-aware encodings matter

Tasks like detection and segmentation need strong spatial reasoning.

Flattened 1D positions lose row/column structure.

Relative or 2D-aware encodings (Swin bias, RoPE) handle resolution changes and spatial relations better.

`trunc_normal_` initialisation is standard for CLS token and positional embeddings — stabilises early training.

3 — ViT History and the Modern Family

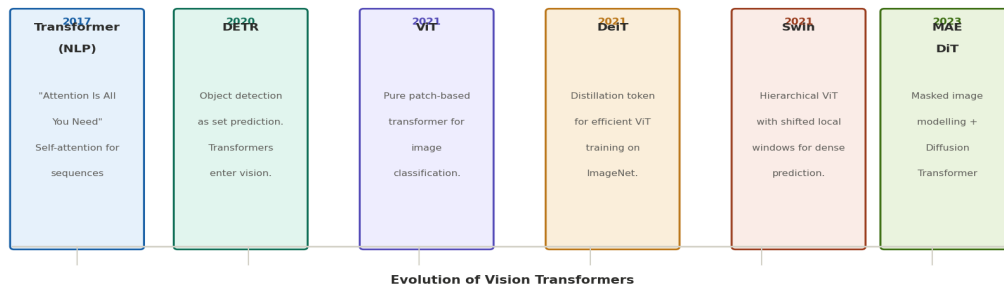


Figure 3 — ViT evolution timeline. Each model adds a key capability: DeiT adds training efficiency, Swin adds hierarchical structure for dense tasks, MAE adds self-supervised pretraining, DiT moves ViT into generative diffusion.

The modern ViT family — key distinctions

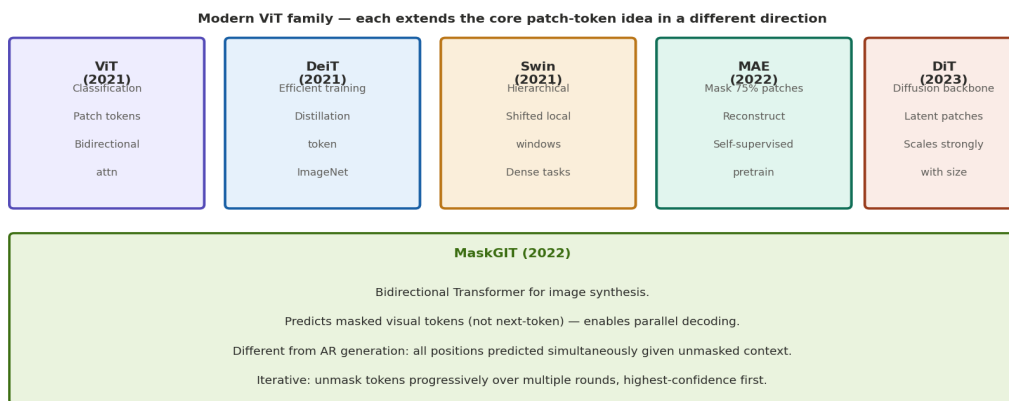


Figure 4 — The five key ViT-family models. MaskGIT (bottom) deserves special attention: unlike autoregressive models, it predicts all masked tokens in parallel and iteratively refines them — a fundamentally different generation paradigm.

4 — DiT: Diffusion Transformer (Flashcard Focus)

DiT replaces the U-Net backbone in latent diffusion models with a pure Transformer operating on latent patches. The key flashcard question is about its scaling behaviour:

DiT scaling — the key flashcard answer

DiT operates on latent patches (encoded by a VAE), using full self-attention over all patches.

Conditioning on timestep t uses adaLN (Adaptive Layer Norm) — shifts and scales features based on t and class label.

Scaling result: larger DiT models consistently achieve better FID than smaller DiT models, even if smaller models use more sampling steps.

Why? Smaller models lack the capacity to learn the complex score/denoising function. More sampling steps cannot substitute for model capacity.

U-ViT comparison: the ViT-style backbone (U-ViT) quantitatively outperforms CNN-style U-Net on both sFID and FID metrics.

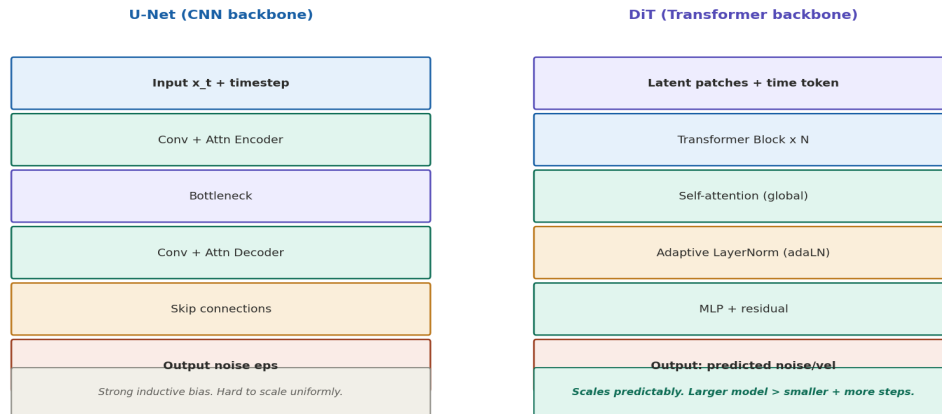


Figure 5 — U-Net (left) vs DiT (right). U-Net mixes convolution and attention with hard-wired skip connections. DiT is a pure Transformer over latent patches with adaLN conditioning. DiT scales more predictably.

5 — Evaluation Metrics: FID, Precision, Recall, IS

The flashcards test these four metrics explicitly. Understand each conceptually:

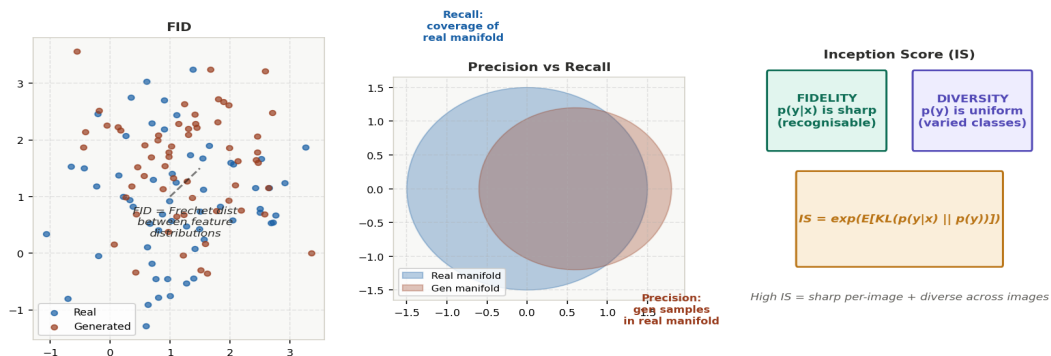


Figure 6 — Three evaluation metrics. FID: Fréchet distance in Inception feature space (lower = better). Precision vs Recall: precision = fraction of generated samples inside real manifold (realism); recall = fraction of real manifold covered by generated samples (diversity). IS: rewards both per-image clarity and cross-image diversity.

Metric	Measures	Formula / key idea	Limitation
FID	Distribution closeness	$\ \mu_r - \mu_g \ _2 + \text{Tr}(\dots)$	Inception features, Gaussian approx, sample size sensitive
Precision	Realism	Fraction of gen samples in real manifold	Does not capture diversity
Recall	Diversity / coverage	Fraction of real manifold covered by gen	Does not capture realism
IS (Inception Score)	Fidelity + diversity	$\exp(E[KL(p(y x) p(y))])$	Does not compare to real images directly

Sample Exam Questions (from professor)

Q1 — Consider a ViT without any class token. Is classification still possible? Discuss one tradeoff.

Yes. The model can aggregate final patch tokens via mean pooling, attention pooling, or another global summarisation method, then apply a classifier head. The tradeoff: without a CLS token, there is no dedicated position whose sole purpose is accumulating task-specific global information. The CLS token learns to specialise for this role through backpropagation. Pooling is simpler and more symmetric but may be less explicitly optimised for classification.

Q2 — Why do residual connections and LayerNorm play an important role in a deep ViT, even though neither directly adds visual knowledge?

Residual connections allow gradients to flow directly to earlier layers without passing through the full stack, preventing vanishing gradients and preserving earlier representations. LayerNorm stabilises the scale of activations within each token at each layer, making optimisation more predictable. Neither encodes image structure, but both are essential to turn the conceptual architecture into a trainable deep system.

Q3 — A student claims ViTs are simply better than CNNs because they are newer and more general. Critically evaluate.

The claim is too simplistic. ViTs provide flexible global interaction and scale well with data and compute, making them dominant in large-scale pretraining and multimodal systems. But CNNs have strong inductive biases well-matched to many visual problems, are more data-efficient at smaller scales, and remain highly competitive when modernised. Modern practice mixes both: convolutional stems + transformer blocks. The better conclusion is that ViTs and CNNs offer different advantages depending on data scale, compute, and task.

Q4 — Why is the rise of transformer backbones in generative image models (DiT, U-ViT) conceptually significant?

It suggests the Transformer is a general-purpose representation mechanism, not a language-only architecture. If the same backbone supports understanding (classification, retrieval) and generation (diffusion backbone), then visual perception and synthesis share deep architectural commonality. This changes model design: classification encoders and generative backbones are now variations of related token-based architectures rather than fundamentally separate families. DiT also shows that scaling laws from language models (bigger model = better) transfer to image generation.

Q5 — Summarise the full ViT pipeline from raw image to output, identifying main design choices.

Step 1: Image split into $N = HW/P^2$ non-overlapping patches (design choice: patch size P). Step 2: Each patch flattened and linearly projected to D -dim token (= Conv2d with kernel=stride= P). Step 3: Learnable CLS token prepended. Step 4: Positional embeddings added (design choice: learned absolute, sinusoidal, relative, or RoPE). Step 5: L transformer encoder blocks (design choices: depth L , heads, D). Step 6: CLS token state extracted $\rightarrow$ classification head (design choice: CLS or pooling). Key tradeoff: smaller $P \rightarrow$ finer resolution but N grows quadratically $\rightarrow O(N^2)$ attention cost.

Key Concepts Summary — Including Flashcard Topics

Key concepts

- $N = HW/P^2$ patches; tensor flow $[B,C,H,W] \rightarrow [B,N,D] \rightarrow [B,N+1,D] \rightarrow [B,D] \rightarrow [B,K]$
- Patch embedding = Conv2d(kernel=P, stride=P) — efficient implementation
- trunc_normal_ init for CLS token and positional embeddings (stabilises training)
- CLS token: learnable, updated via backprop, aggregates global info via bidirectional attention
- Positional encodings: learned absolute (resolution-sensitive), sinusoidal (fixed), relative bias (Swin), RoPE (rotates Q/K)
- GELU is the standard MLP activation in Transformer blocks (not ReLU)
- DeiT: adds distillation token for efficient ImageNet-scale ViT training
- Swin Transformer: hierarchical + shifted local windows -> better for dense prediction
- MAE: mask 75% of patches, reconstruct -> powerful self-supervised ViT pretraining
- MaskGIT: bidirectional transformer, predicts masked visual tokens in parallel (not AR)
- DiT: replaces U-Net in diffusion with Transformer on latent patches; uses adaLN conditioning
- DiT scaling law: larger model > smaller model + more sampling steps (capacity cannot be substituted)
- U-ViT outperforms CNN U-Net on sFID and FID — ViT backbone is better for diffusion
- FID: distribution distance in Inception space (lower=better). Precision=realism. Recall=diversity. IS=fidelity+diversity
- $O(N^2)$ attention cost — patch size tradeoff: smaller P = finer detail but much higher compute

Next: Chapter 08 — Vision-Language Models: CLIP, contrastive learning, fusion decoders.

Vision-Language Models (VLMs)

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Two dominant families

- Dual-encoder (CLIP-style): separate image + text encoders, contrastive training, shared embedding space.
- Fusion encoder-decoder: image encoder provides visual memory; text decoder uses cross-attention.
- Dual encoders excel at retrieval and zero-shot recognition. Fusion models excel at generation and grounding.
- CLIP temperature τ controls sharpness of contrastive discrimination.
- Zero-shot classification = similarity matching with text prompts — no trained classifier head needed.

1 — CLIP: Contrastive Language-Image Pre-Training

CLIP learns from matched image-text pairs by training image and text encoders so that matched pairs are close in embedding space and mismatched pairs are far apart. For a batch of N pairs, the similarity matrix has entries:

$$s_{ij} = \hat{v}_i^T \hat{t}_j / \tau \text{ where } \hat{v} = \text{L2-normalised image embedding, } \hat{t} = \text{L2-normalised text embedding}$$

The contrastive loss maximises s_{ii} (matched pairs) relative to all s_{ij} for $i \neq j$ (mismatched pairs, the negatives). This is applied symmetrically for both image-to-text and text-to-image directions.

Temperature τ — more than a cosmetic constant

Temperature τ controls how sharply the model separates the best match from the rest. Lower τ -> sharper discrimination (stronger gradient on near-misses, harder optimisation). Higher τ -> smoother distribution (weaker separation, easier training but poorer discrimination). τ directly shapes the contrastive geometry the model learns.

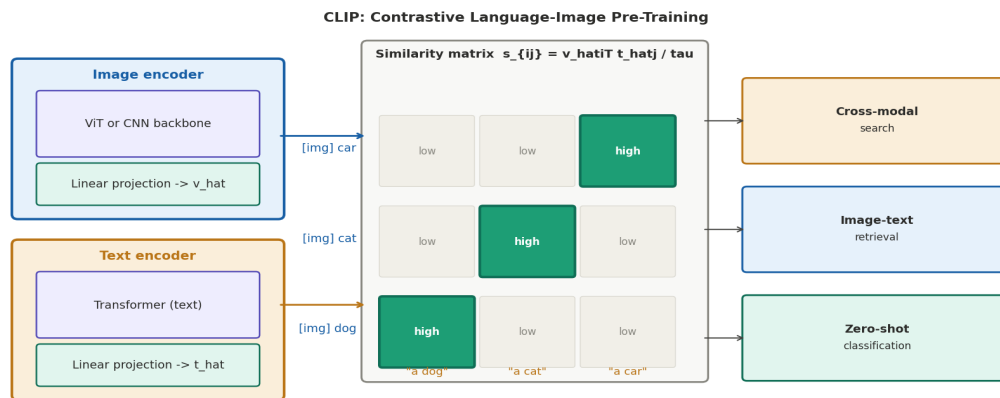


Figure 1 — CLIP architecture and training. Image encoder produces $\mathbf{v_hat}$; text encoder produces $\mathbf{t_hat}$. Similarity matrix $s_{ij} = \mathbf{v_hat}_i^T \mathbf{t_hat}_j / \tau$. Green diagonal = matched pairs (maximised). Off-diagonal = negatives (minimised). Right: downstream applications.

Why both positive AND negative pairs are essential

If the model only increased similarity for matched pairs, embeddings could collapse to a trivial solution where all pairs appear similar.

Negative pairs define the geometry of separation — they teach the model what distinctions matter.

The batch structure is critical: each batch provides the correct match AND a set of competing alternatives.

Larger batches provide more negative examples -> stronger contrastive signal (used to scale CLIP training).

2 — Zero-Shot Classification

Standard supervised classifiers have a fixed output layer trained on a specific class set. CLIP reframes classification as similarity matching: embed the image, embed one or more text prompts per class, choose the class whose prompt has the highest cosine similarity to the image.

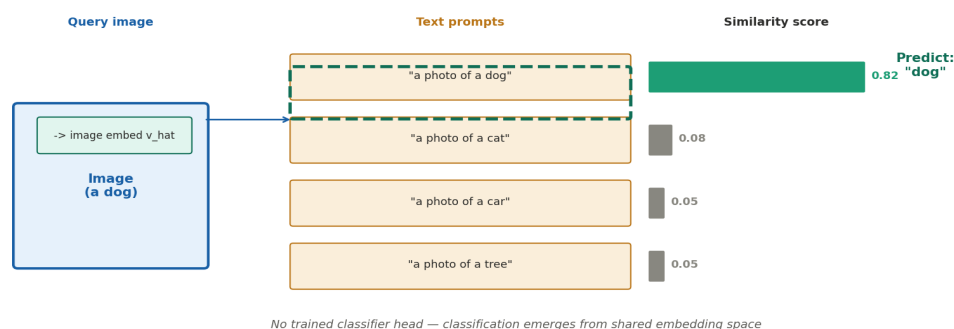


Figure 2 — Zero-shot classification. Image embedding is compared to embeddings of text prompts for each class. The highest-similarity prompt wins — no trained classifier head needed. Prompt wording matters: different templates change performance.

Why cosine similarity and normalisation are important

When embeddings are L2-normalised, the dot product equals cosine similarity. This focuses comparison on direction in embedding space rather than raw magnitude — magnitude can be affected by factors unrelated to semantic content. Normalisation stabilises geometry and makes comparisons meaningful across examples.

3 — Fusion Encoder-Decoder VLMs

For tasks requiring generation or detailed grounding (captioning, VQA, spatial reasoning), a fusion model is needed. The image encoder (e.g. ViT) produces a set of visual memory tokens. The text decoder uses two attention mechanisms:

Attention type	What it attends to	Role
Masked self-attention	Previous text tokens only (causal)	Builds text context, respects autoregressive constraint
Cross-attention	Visual memory (image encoder output)	Injects visual evidence into text generation

The generative distribution becomes:

$$p(y_t | y_{1:t}, image) = \text{softmax}(W * \text{decoder_state}_t)$$

Each generated token is conditioned on both the image (via cross-attention) and all previously generated tokens (via masked self-attention). This makes the model genuinely generative and visually grounded.

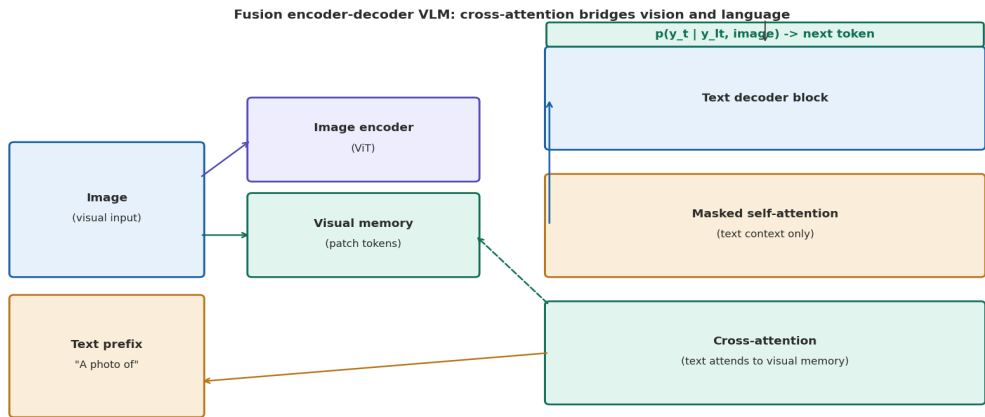


Figure 3 — Fusion encoder-decoder VLM. Image tokens enter the encoder (top left); visual memory is produced. The text decoder uses cross-attention to attend to visual memory and masked self-attention over the text prefix.

Teacher forcing and exposure bias

Teacher forcing: during training, the decoder receives correct previous tokens (not its own predictions). Makes training stable.

Exposure bias: at test time, the decoder conditions on its own (possibly wrong) outputs. Errors accumulate.

This train-test mismatch is a known limitation of all autoregressive models trained with teacher forcing.

4 — VLM Failure Modes

Failure mode	Description	Why it happens
Bias / spurious correlations	Associates concepts with backgrounds or demographics	Training data shortcuts
Weak spatial grounding	Fails on counting, spatial relations, attributes	Global embeddings lose local detail
Distribution shift	Performance drops in new domains (medical, aerial)	Training distribution mismatch
Hallucination	Generates plausible but visually unsupported text	Decoder relies on language priors over visual evidence

Sample Exam Questions

Q1 — Why is the CLIP training objective fundamentally about both positive AND negative pairs?

CLIP must learn not only that matched pairs are similar, but that mismatched pairs are dissimilar. If the model only increased similarity for positive pairs without contrasting them against negatives, the embedding space could collapse — all embeddings become similar to each other, satisfying the positive objective trivially. Negative pairs define the geometry of separation: they teach the model what distinctions matter and force the embedding space to encode semantically meaningful differences.

Q2 — In CLIP, why is temperature tau more than a cosmetic scaling constant?

Temperature controls how sharply the model distinguishes the best match from alternatives in the softmax. Lower tau makes similarity differences more consequential: near-misses (similar but incorrect pairs) receive strong gradients, which strengthens discrimination but makes optimisation harder. Higher tau smooths the distribution, weakening the pressure to separate close competitors. tau directly shapes the contrastive geometry the model learns — it is a meaningful hyperparameter, not just scaling.

Q3 — Why does zero-shot classification in CLIP not require a conventional classifier head?

CLIP reframes classification as language-conditioned similarity matching. Instead of a fixed weight vector per class, the model compares image embeddings to text prompt embeddings (e.g. "a photo of a cat"). The class whose prompt is most similar to the image wins. Class knowledge is mediated through the shared embedding space learned during contrastive training — not through a task-specific output layer. New classes can be added at test time simply by writing new prompts.

Q4 — What is the conceptual difference between masked self-attention and cross-attention in a fusion decoder?

Masked self-attention lets the decoder build a contextual representation of the text prefix already generated, enforcing the autoregressive constraint (no future tokens visible). It answers: "what has already been said?" Cross-attention lets the decoder consult the visual memory from the image

encoder. It answers: "what in the image is relevant to the current word being generated?" Self-attention organises linguistic context; cross-attention injects visual evidence.

Q5 — Why does a fusion decoder excel at detailed grounding while a dual encoder like CLIP struggles?

CLIP compresses both image and text into global vectors — powerful for retrieval but loses fine-grained spatial and relational detail. A fusion decoder uses cross-attention, allowing each generated text token to attend directly to specific visual memory vectors (patch tokens). This means the model can condition each word on localised evidence: counting objects, describing spatial relations, or answering questions about specific image regions. Dual encoders sacrifice this granularity for retrieval efficiency.

Key Concepts Summary

Key concepts

- CLIP: dual encoders, contrastive loss, shared normalised embedding space
- Score $s_{ij} = v_i^T t_j / \tau$ — matched pairs maximised, mismatched pairs minimised
- Temperature τ : lower = sharper discrimination, stronger gradients, harder optimisation
- Zero-shot classification: image embed vs text prompt embeds — no trained head needed
- L2 normalisation: focus on direction (cosine similarity), not magnitude
- Fusion VLM: masked self-attention (text) + cross-attention (image) in decoder
- Cross-attention: text tokens attend to visual memory — enables detailed grounding
- Teacher forcing: correct tokens during training -> exposure bias at test time
- Hallucination: decoder relies on language priors instead of visual evidence
- Failure modes: bias, weak grounding, distribution shift, hallucination

Next: Chapter 09 — Training and Adapting VLMs: from zero-shot prompting to RL-based alignment.

Training and Adapting VLMs

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Central conceptual distinction

- Methods that change only the conditioning context: zero-shot, few-shot, RAG.
- Methods that change the policy parameters: SFT, LoRA, DPO, RL-based methods.
- The adaptation ladder is not a ranking from "weak to strong" — it is a map of what is being modified.
- Methods differ in: context vs weights, knowledge vs labels vs preferences vs rewards.
- They are often complementary, not mutually exclusive.

1 — The Adaptation Ladder

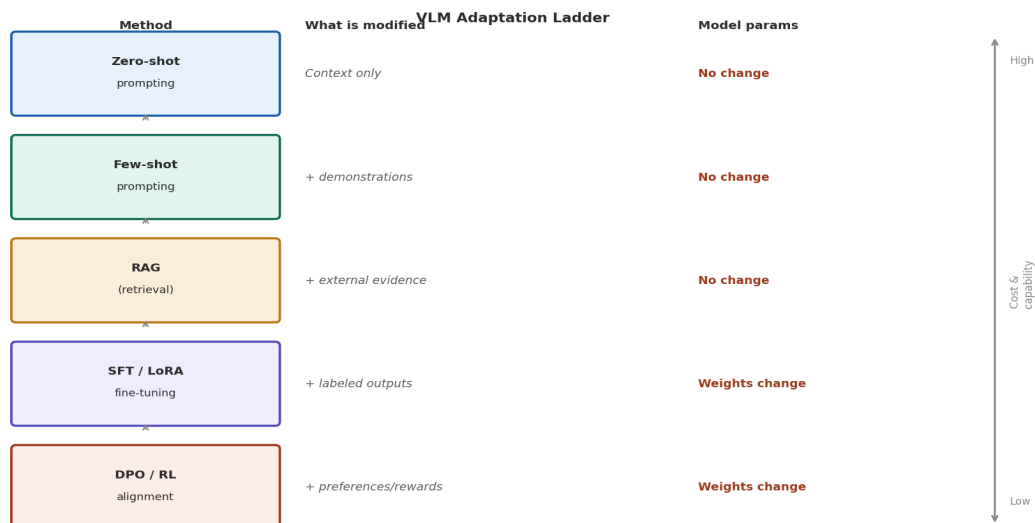


Figure 1 — The VLM adaptation ladder. Bottom to top: increasing cost and capability. Methods differ in what they modify: prompt context, retrieved evidence, labeled outputs, parameterisation, preferences, or reward signals.

Zero-shot, few-shot, and RAG — context-only adaptation

Zero-shot changes only the instruction; few-shot adds demonstration examples to the context window. Neither requires gradient descent — behaviour changes because the Transformer's hidden states depend on the full context. RAG introduces an external memory, retrieving documents and conditioning generation on them:

$$p_{\theta}(y|x,v) = \sum_z p_{\eta}(z|q(x,v)) p_{\theta}(y|x,v,z)$$

RAG is a latent-variable model over retrieved evidence z . The key object is the posterior responsibility weight $w(z)$ — which document actually explains the gold answer. When the primary failure is missing factual knowledge, RAG is often the most principled first intervention.

2 — SFT and LoRA: Parameter Learning

Supervised fine-tuning (SFT) optimises next-token log-likelihood on labeled triples (x, v, y^*) :

$$L_{SFT} = -(1/N) \sum_i \sum_t m_{\{i,t\}} \log p_{\theta}(y^*_{\{i,t\}} | y_{\{i,t\}}, x_i, v_i)$$

The token mask $m_{\{i,t\}}$ is crucial: in chat-style instruction tuning, only the assistant response tokens should contribute to the loss — not the user prompt or system text. This is a practical but important design choice.

LoRA keeps the pretrained weight matrix W frozen and learns a low-rank update:

$$W' = W + \Delta W = W + BA \text{ where } B \in \mathbb{R}^{d_{out} \times r}, A \in \mathbb{R}^{r \times d_{in}}, r \ll d$$

This reduces trainable parameters from $d_{out} \times d_{in}$ to $r \times (d_{in} + d_{out})$. SFT defines the objective; LoRA defines the parameterisation — they are at different levels, not competing alternatives.

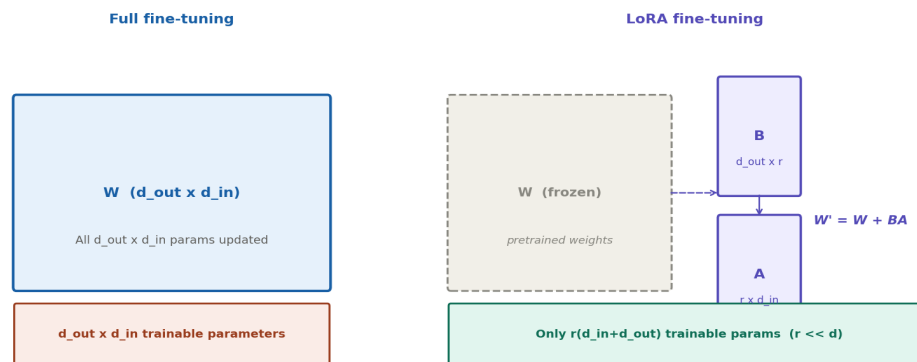


Figure 2 — Full fine-tuning (left): all $d_{out} \times d_{in}$ parameters updated. LoRA (right): W is frozen; only low-rank matrices A and B are trained. $W' = W + BA$ at inference time (no extra latency).

Which projections to adapt with LoRA?

Attention projections (q_{proj} , k_{proj} , v_{proj} , o_{proj}): change information ROUTING.

MLP projections ($gate_{proj}$, up_{proj} , $down_{proj}$): change nonlinear TRANSFORMATION.

Many downstream tasks need both: new visual-semantic associations AND new feature transformations.

Adapting only attention changes how info flows; adapting only MLP changes how info is reshaped.

3 — DPO and RL-Based Alignment

DPO (Direct Preference Optimisation) aligns model behaviour with human preferences by comparing a preferred response y_w against a rejected response y_l , relative to a frozen reference policy π_{ref} :

$$L_{DPO} = -\log \sigma(\beta[\log \pi_{\theta}(y_w|x, v) - \log \pi_{\theta}(y_l|x, v) - \log \pi_{ref}(y_w|x, v) + \log \pi_{ref}(y_l|x, v)])$$

DPO performs preference alignment without explicit RL rollouts or PPO-style advantage estimation. Warning: relative preference is not identical to factual correctness — if preference pairs reward style over truth, the model may become better aligned to human taste while becoming less scientifically grounded.

RLOO: leave-one-out RL baseline

RLOO generates G completions per prompt, scores each with a reward function, and uses the average reward of the other completions as a baseline:

$$A_i = r_i - (1/(G-1)) * \sum_{j \neq i} r_j, L_{RLOO} = -(1/G) \sum_i A_i \log \pi_{\theta}(o_{ilq})$$

This provides a relative baseline for the same prompt, avoiding some of the complexity of PPO while still enabling reward-based training.

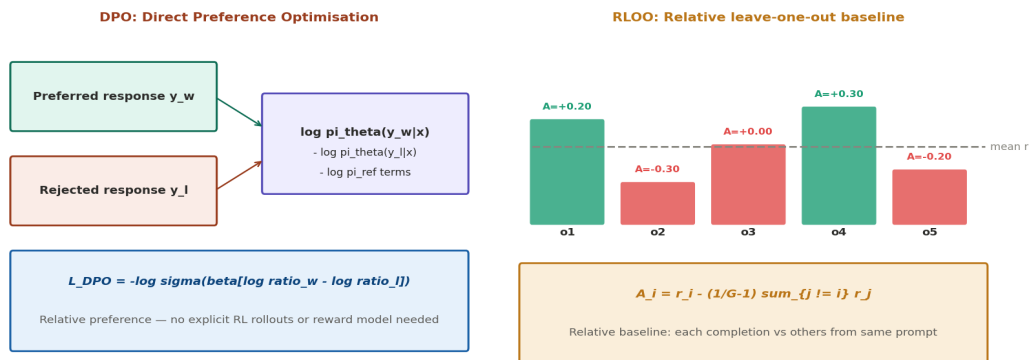


Figure 3 — DPO (left): optimises relative log-probability of preferred vs rejected response against a reference policy. RLOO (right): G completions per prompt, advantage = reward - leave-one-out average reward from same prompt.

4 — RL Reinterpretation of Token Generation

RL concept	VLM interpretation
State s_t	Prompt, image, and generated prefix so far
Action a_t	Next token generated
Policy $\pi_{\theta}(a_t s_t)$	Next-token distribution from the LM
Trajectory τ	Full generated answer o
Reward r	Scalar score for the complete answer (preference, verifier, etc.)

Layered view: RAG + LoRA + RL alignment as three complementary layers

RAG acts on the evidence layer: what external information the generator sees at inference time.

LoRA/SFT acts on the policy-parameter layer: how the model maps context to answer distribution.

RL alignment acts on the behavioural objective layer: what output behaviour is incentivised.

These layers compose — RAG grounds knowledge, SFT specialises the policy, RL aligns preferences.

Sample Exam Questions

Q1 — Why is the adaptation ladder best described as a map of intervention types rather than a ranking from weak to strong?

The methods differ along multiple conceptual dimensions simultaneously, not just "strength." Zero-shot, few-shot, and RAG change the conditioning information at inference time — they modify context, not parameters. SFT, LoRA, DPO, and RL-based methods change the policy itself. Moreover, the modified object differs across methods: prompt, memory, labels, parameterisation, preferences, rewards. This is why the methods are often complementary rather than mutually exclusive — each targets a different layer of the system.

Q2 — When would RAG be a more principled first intervention than SFT or DPO?

When the primary failure is missing factual knowledge. If a VLM confuses closely related species because it lacks domain knowledge, the bottleneck is not style or preference alignment but missing evidence. RAG directly addresses this by injecting external evidence at inference time. SFT teaches the model to imitate desired outputs but does not give it new, explicit evidence at inference time. DPO adjusts preferences among answers — it is even further from solving a knowledge gap. For grounding failures and missing task knowledge, RAG is often the cleanest first upgrade.

Q3 — Explain LoRA mathematically. Why can a low-rank update be sufficient?

LoRA freezes the pretrained weight W and learns $W' = W + BA$, where B is $d_{\text{out}} \times r$ and A is $r \times d_{\text{in}}$. This reduces trainable parameters from $d_{\text{out}} \times d_{\text{in}}$ to $r \times (d_{\text{in}} + d_{\text{out}})$ — far smaller when $r \ll d$. The hypothesis is that meaningful updates for downstream tasks lie in a low-dimensional subspace. Pretrained models already encode rich representations; fine-tuning typically requires small, structured adjustments rather than relearning from scratch. The low-rank structure captures this efficiently.

Q4 — What is the reference policy π_{ref} in DPO and why is it needed?

The reference policy π_{ref} is a frozen copy of the pre-DPO model (typically the SFT-trained model). It acts as an anchor that prevents the model from drifting too far from its original behaviour during preference training. Without it, optimising the preference objective alone could collapse the model to degenerate outputs that "win" the preference comparison for superficial reasons. The KL penalty against π_{ref} (implicit in the DPO objective) regularises the update, preserving general language quality while shifting preference alignment.

Q5 — How does RLOO interpret token generation as RL and what advantage does the leave-one-out baseline provide?

RLOO reinterprets generation as policy optimisation: state = prompt + prefix, action = next token, trajectory = full answer, reward = scalar score. G completions are generated for the same prompt; each is scored. The advantage for completion i is $A_i = r_i - \text{average reward of all other completions from the same prompt}$. This leave-one-out baseline is useful because it is prompt-specific — it removes the average effect of the prompt, isolating the contribution of each specific completion. This reduces variance compared to a global baseline and avoids some PPO complexity.

Key Concepts Summary

Key concepts

- VLM adaptation: context-only (zero-shot, few-shot, RAG) vs parameter-changing (SFT, LoRA, DPO, RL)
- Zero-shot: prompt only. Few-shot: +demonstrations in context. Neither uses gradients.
- RAG: latent-variable model over retrieved evidence z ; best for missing knowledge failures
- SFT: next-token MLE with token mask $m_{\{i,t\}}$ — only assistant spans counted
- LoRA: $W' = W + BA$, $r \ll d$; SFT defines objective, LoRA defines parameterisation
- LoRA adapts: attention (q/k/v/o projections) AND MLP (gate/up/down) for full adaptation
- DPO: maximises log-ratio of preferred vs rejected relative to reference policy π_{ref}
- DPO warning: relative preference $\neq$ factual correctness; style may dominate truth
- RLOO: $A_i = r_i - \text{leave-one-out mean}$; prompt-specific baseline reduces variance
- RL reinterpretation: state=prefix, action=token, reward=completion score
- Three complementary layers: RAG (evidence) + LoRA (parameters) + RL (objective)

Next: Chapter 10 — Quantization: deploying models under severe numerical constraints.

Quantization

ECE786 — Generative Models & Vision-Language Systems * Spring 2026

Core framing

- Quantization is a numerical approximation problem, not just a deployment trick.
- The uniform quantizer $\hat{x} = s * \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$ has one structural tension: scale s controls both precision AND range simultaneously.
- Three phases: pre-quantization (calibration), during-training (QAT), post-training (PTQ).
- The STE is essential for QAT: forward pass uses fake-quantized values; backward uses surrogate gradient.
- Best PTQ shifts from "approximate weights well" to "approximate the function well" (AdaRound, GPTQ).

1 — The Uniform Quantizer and the Core Tradeoff

A uniform quantizer maps floating-point values to integer codes:

$$\hat{x} = s * \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$$

The scale s creates a structural tension: small s gives fine resolution (low rounding error) but a narrow range (high clipping error). Large s widens the range but coarsens the grid. For fixed bit-width, one cannot improve both simultaneously — this is the fundamental precision-range tradeoff.

Sensitivity and Hessian-based analysis

Near a trained solution, the loss increase from a quantization perturbation e is:

$$\Delta L \approx -L'(w) * e + (1/2) * L''(w) * e^2$$

Since $L'(w)$ is near zero at a trained minimum, the second derivative (Hessian) becomes the dominant predictor of damage. High curvature (high Hessian diagonal) = high sensitivity. This justifies keeping sensitive layers at higher precision.

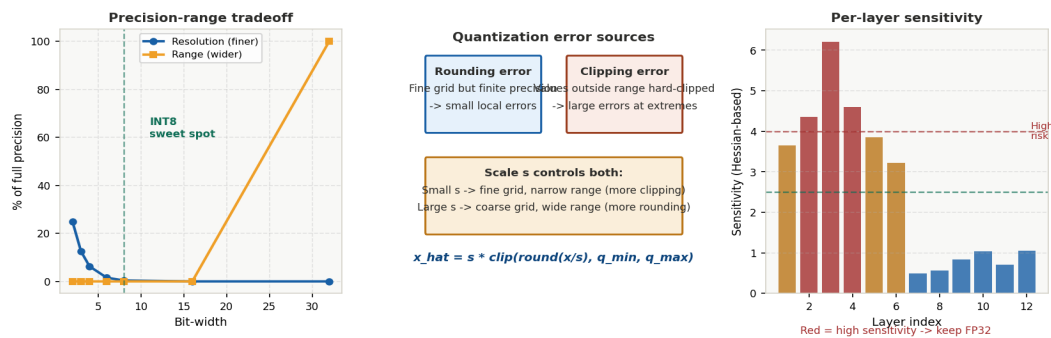


Figure 1 — Left: precision-range tradeoff across bit-widths — higher bits resolve both, INT8 is often a sweet spot. Middle: rounding vs clipping error sources and the role of scale s . Right: per-layer Hessian sensitivity — red layers should stay in FP32.

2 — Quantization-Aware Training (QAT) and the STE

QAT moves the quantizer inside the training loop. The model learns with fake-quantized weights and activations, so it becomes robust to quantization distortions before deployment. The objective:

$$\min_{\{W, \Theta\}} L(f(x; \{Q_W(W_l)\}, \{Q_a(a_l)\}))$$

The challenge: rounding and clipping are not differentiable. The Straight-Through Estimator (STE) solves this by separating forward fidelity from backward usability:

Straight-Through Estimator (STE)

Forward pass: use fake-quantized values $\hat{W} = Q(W)$ — the model experiences quantization.

Backward pass: replace the true gradient (which is 0 for rounding) with a surrogate: pass $dL/d\hat{W}$ directly through as if the quantizer were the identity inside the clip range.

STE is not the exact gradient — it is a useful approximation that points optimization in a productive direction.

Without STE: gradients stall because $d(\text{round}(x))/dx = 0$ almost everywhere.

PACT and LSQ: learnable quantizer parameters

PACT makes the clipping threshold α a learnable parameter (instead of a fixed heuristic). LSQ makes the step size s itself learnable. Both illustrate the broader principle: better quantization often comes from making previously hand-chosen numerical decisions adaptive and part of the optimisation.

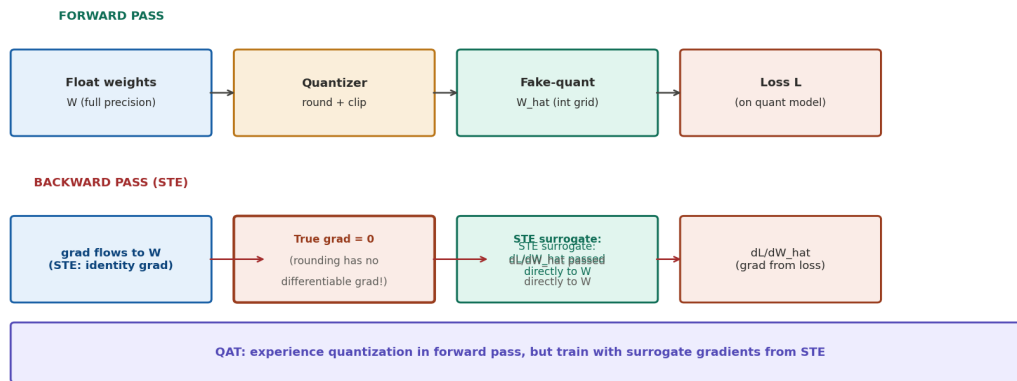
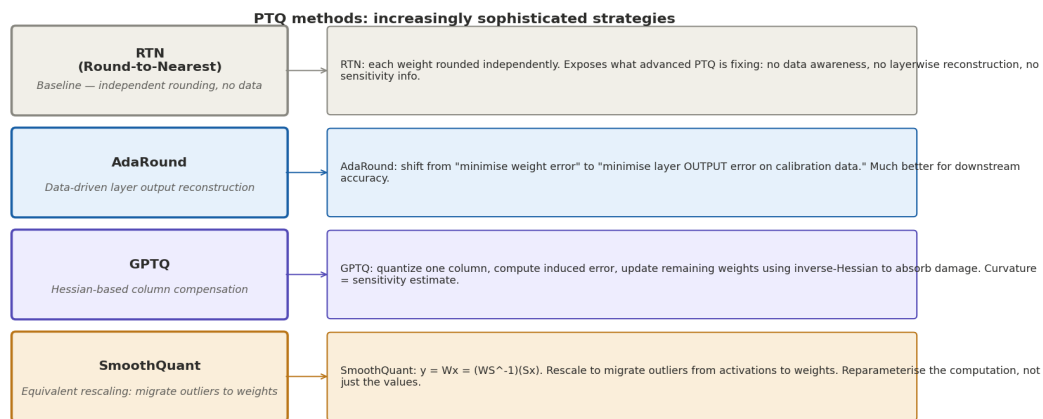


Figure 2 — QAT with STE. Forward pass (top): float weights are fake-quantized to W_{hat} , used for forward computation. Backward pass (bottom): true gradient through $\text{round}()$ is 0 (problem!); STE passes surrogate gradient directly from W_{hat} to W .

3 — Post-Training Quantization (PTQ): RTN to GPTQ

PTQ quantizes a pretrained model without reopening full training. The key insight: more advanced methods shift from "approximate each weight well" to "preserve the function of the layer well."



Key principle: best PTQ shifts from approximating WEIGHTS well to approximating FUNCTION well

Figure 3 — PTQ methods progression. RTN (baseline) -> AdaRound (layer-output reconstruction) -> GPTQ (Hessian compensation after each column) -> SmoothQuant (equivalent rescaling to migrate activation outliers to weights).

4 — Quantization for Diffusion Models and Multimodal VLMs

General model classes present unique quantization challenges:

Model type	Unique challenge	Key approach
Autoregressive LLMs	Activation outliers, KV-cache growth	SmoothQuant, AWQ, group-wise quantization
Diffusion models	Error compounds across timesteps, timestep-varying distributions	Timestep-aware calibration

Model type	Unique challenge	Key approach
Diffusion transformers	Both outlier issues AND timestep dependence	Combined transformer PTQ + timestep scheduling
CLIP dual encoders	Must preserve embedding geometry, not just task accuracy	Joint image-text calibration
Fusion VLMs	Unequal sensitivity: encoder vs projector vs decoder	Mixed-precision per-module

I-BERT: quantizing the full computation graph

Quantizing only matrix multiplications is insufficient if GELU, Softmax, and LayerNorm remain in FP32.

I-BERT uses integer polynomial approximations for nonlinear operators.

The real deployment goal: a fully quantized computation graph, not just quantized weight matrices.

This reframes quantization as a property of the entire executed graph, not just stored parameters.

Sample Exam Questions

Q1 — What is the central precision-range tradeoff in uniform quantization, and why is it structural?

The scale s in $\hat{x} = s * \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$ simultaneously controls grid spacing and representable range. Small s : fine resolution (low rounding error), narrow range (high clipping error). Large s : wide range (low clipping error), coarse grid (high rounding error). This is structural because the same parameter controls two competing objectives. For a fixed bit-width (fixed number of integer codes), one cannot simultaneously improve local resolution and global coverage.

Q2 — Why is the STE conceptually necessary for quantization-aware training?

Rounding and clipping have zero or undefined true derivatives almost everywhere — rounding has zero derivative everywhere except at discontinuities. If one backpropagated through the true quantizer, gradients would be zero and learning would stall. The STE replaces the true gradient with a surrogate (typically the identity inside the clip range, zero outside), allowing gradients to flow to the float weights while the forward pass still uses fake-quantized values. QAT therefore trains under quantized forward behaviour but with approximate, useful gradients. The STE is not exact — it is a practical approximation that makes QAT tractable.

Q3 — Why do PACT and LSQ both illustrate the same broader principle about quantization?

PACT learns the clipping threshold; LSQ learns the step size. Both convert what were previously fixed, hand-chosen numerical parameters into learnable quantities optimised by the task loss. The broader principle: quantization improves when the network adapts not just its weights but also the numerical rules that govern the quantizer. Early methods treated clipping thresholds and scales as engineering constants; PACT and LSQ show these are part of the optimisation problem.

Q4 — Why does GPTQ use Hessian-based compensation after quantizing each column?

If each weight column is rounded independently and the error accepted as final, quantization errors accumulate through the rest of the layer. GPTQ recognises that after quantizing one column, the remaining unquantized columns are still free to adjust. Under a second-order (Hessian) approximation, those remaining weights can be updated to compensate for the damage introduced by the quantized column. The inverse Hessian provides the locally optimal compensating update. GPTQ therefore iterates: quantize column -> estimate damage -> compensate remaining weights -> continue, rather than accepting each rounding decision as irreversible.

Q5 — Why are diffusion models harder to quantize post-training than feedforward classifiers?

A feedforward classifier sees quantization error only within one forward pass. A diffusion model runs the same denoiser network hundreds or thousands of times across the reverse process. Small quantization errors from each call can compound over the denoising trajectory — an error at $t=T$ affects the input to the denoiser at $t=T-1$, and so on. A second complication is that the activation distribution of the denoiser changes with timestep: calibration data from a single noise level may not be representative of the full reverse process. Effective diffusion PTQ therefore requires timestep-aware calibration and careful error analysis across the full trajectory.

Key Concepts Summary

Key concepts

- Uniform quantizer: $\hat{x} = s * \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$
- Structural tension: same scale s controls precision AND range — cannot optimise both for fixed bit-width
- Two error sources: rounding error (too coarse grid) + clipping error (too narrow range)
- Hessian sensitivity: high curvature = high sensitivity to quantization perturbation
- QAT: quantizer inside training loop; model experiences int-precision in forward pass
- STE: forward = fake-quant, backward = surrogate gradient (identity inside clip range)
- PACT: learnable clip threshold α . LSQ: learnable step size s . Both make fixed params adaptive.
- PTQ phase: RTN (baseline) -> AdaRound (output reconstruction) -> GPTQ (Hessian compensation)
- SmoothQuant: $y = (WS^{-1})(Sx)$ — migrate outliers from activations to weights equivalently
- I-BERT: quantize full graph including GELU, Softmax, LayerNorm — not just matmuls
- Diffusion PTQ harder: errors compound across timesteps + timestep-varying distributions
- CLIP quantization: must preserve embedding geometry, not just classification accuracy

This completes all 10 chapters. Good luck on your exam!

ECE786 Spring 2026

Code Concepts

Likely Exam Questions with Sample Answers

Prof. Zualkernan · Open-Book Exam Reference

About this document

These questions are written in exactly the same style as the professor's sample exam questions:

- Conceptual and reasoning-based — not syntax recall or code memorisation.
- "Explain why", "what would happen if", "what is the role of" style.
- Full prose answers that demonstrate understanding of the notebook code.

Chapters covered: Ch4 (Diffusion), Ch7 (ViT), Ch8 (CLIP / VLMs), Ch9 (Adapting VLMs).

These are the four chapters where the professor's lecture notebooks contain runnable code.

Chapter 4 — Advanced Diffusion Concepts

The Chapter 4 notebook implements score-based diffusion on MNIST, including a TinyUNet with FiLM-style time injection, the forward diffusion schedule, the reverse sampling loop, classifier-guided diffusion with a noise-aware classifier, and classifier-free guidance. The questions below follow the same conceptual style as the sample exam questions: they ask for reasoning and understanding, not syntax.

Question 1

The lecture code computes the score of a standard Gaussian as $\text{score} = -x$. Explain conceptually why this formula is correct, and what it means geometrically in the context of diffusion sampling.

Sample Answer

For a standard Gaussian, the log-density is a downward parabola with its peak at the mean. The score is defined as the derivative of the log-density, which equals $-x$ for $N(0,1)$. Geometrically, this means the score always points toward the mean: when x is positive the score is negative, pulling the sample left; when x is negative the score is positive, pulling right. The farther a sample is from the mean, the stronger the pull. In diffusion sampling, following the score field at each timestep moves a noisy sample toward the most typical region of the data distribution, which is what denoising means geometrically.

Question 2

The TinyUNet injects the timestep as an additive bias: the feature map at each layer receives a time-dependent offset computed from a sinusoidal time embedding. Why is this injection necessary, and what would happen if the network received only the noisy image without any time information?

Sample Answer

The denoising task is fundamentally different at different noise levels. At small t , the input is nearly clean and denoising should be subtle and detail-preserving. At large t , the input is almost pure noise and denoising requires a global, uncertain inference. Without time information, the same network would face an ambiguous mapping: the same visual pattern can mean very different things at $t=10$ versus $t=900$. The network would have no way to decide how aggressively to denoise. By adding a time-dependent bias at every level of the network, the model learns a different effective processing pipeline for each noise level, which is essential for the reverse process to work correctly across the full trajectory.

Question 3

The training loop normalises MNIST images to the range $[-1, 1]$ before diffusion training. Explain why this normalisation is important for diffusion specifically, rather than being a minor preprocessing detail.

Sample Answer

Diffusion starts the reverse process from samples drawn from a standard Gaussian $N(0, I)$, which is symmetric around zero. If the clean data x_0 were in $[0, 1]$, the signal would be systematically positive while the noise is centred at zero. This asymmetry would create a mismatch between the prior used at sampling time and the distribution the model was trained to denoise toward. By normalising to $[-1, 1]$, the clean data is approximately zero-centred, which aligns with the Gaussian noise prior. The reverse process then recovers samples in $[-1, 1]$, which must be mapped back to $[0, 1]$ for visualisation. This is why the lecture code clips and rescales: $(\text{samples.clamp}(-1, 1) + 1) / 2$.

Question 4

In classifier-guided diffusion, the classifier is evaluated on the reverse mean rather than on the raw noisy sample. Why does it make sense to differentiate the classifier with respect to this mean, and what would be lost if the classifier were trained only on clean images?

Sample Answer

The reverse mean is the model's current best proposal for where the next denoised sample should go. Taking the gradient of the classifier with respect to this point tells us how to shift that proposal so that it becomes more consistent with the desired class. This is meaningful because the mean lies in a semantically interpretable position: it is partially denoised and closer to the data manifold than the raw noisy sample. A classifier trained only on clean images would produce unreliable gradients when evaluated on heavily corrupted inputs at large timesteps. Its decision boundary is shaped for clean data, so its gradients at noisy states may point in the wrong direction or be numerically unstable. A noise-aware classifier is trained on noisy inputs at every timestep, so its gradients remain meaningful throughout the entire reverse trajectory.

Question 5

Classifier-free guidance trains a single model on both conditional and unconditional inputs by randomly replacing the label with a null token during training. Explain why this random dropout step is conceptually necessary rather than optional, and what would fail if it were removed.

Sample Answer

The CFG sampling formula combines a conditional prediction and an unconditional prediction to amplify the conditioning signal: $\text{eps_cfg} = (1+w)*\text{eps_cond} - w*\text{eps_uncond}$. This formula only makes sense if the model has learned both branches. If label dropout were removed during training, the model would only ever see real labels and would never learn how to operate without a condition. The unconditional branch eps_uncond would have no meaningful reference point, so the subtraction in the guidance formula would have no useful effect. The random dropout step is therefore not optional: it is the mechanism that teaches a single network to function as both a conditional and an unconditional model, which is exactly what CFG needs to combine at sampling time.

Chapter 7 — Vision Transformers

The Chapter 7 notebook implements a Vision Transformer for CIFAR-10 classification, covering patch embedding, positional encodings, the class token, transformer encoder blocks with LayerNorm and residual connections, and the classification head. The questions follow the same reasoning-based style.

Question 1

In the ViT implementation, patch embedding is implemented using a Conv2d layer with kernel size equal to the patch size and stride equal to the patch size. Explain why this convolutional implementation is equivalent to flattening each patch and applying a linear projection, and why it is the preferred implementation.

Sample Answer

A convolution with kernel size P and stride P slides a non-overlapping P -by- P window across the image, applying the same learned linear transformation to each patch. Each output spatial position corresponds to one patch and produces a D -dimensional vector, which is exactly what flattening each patch and multiplying by a weight matrix of shape $(P \times P \times C, D)$ would produce. The two operations are mathematically equivalent. The convolutional implementation is preferred because it handles the extraction and projection in a single efficient GPU operation, avoids explicitly constructing the full unfolded patch tensor, and integrates naturally with the rest of the PyTorch framework without additional reshaping steps.

Question 2

The class token is implemented as a learnable parameter initialised to zeros and prepended to the patch sequence before the transformer encoder. Explain the role of the class token, why it is useful for classification, and what alternative approach could be used instead.

Sample Answer

The class token is a learnable vector that is added to the sequence alongside the patch tokens. During the forward pass it can attend to every patch token through bidirectional self-attention, allowing it to accumulate global information from across the entire image. After passing through all encoder layers, the final state of the class token is used as a single image-level representation for the classification head. It is useful because it gives the model a dedicated position whose sole purpose is to aggregate task-relevant global context, updated through backpropagation on the task loss. The alternative is mean pooling: averaging the final states of all patch tokens to produce a global representation. Mean pooling is simpler and treats all patches symmetrically. The class token approach gives the model a dedicated and specialised aggregation mechanism, but both approaches work well in practice.

Question 3

The ViT uses LayerNorm before each sublayer rather than after, a design sometimes called Pre-LN. Explain why this placement matters for training deep Vision Transformers, and what problem the alternative Post-LN design can create.

Sample Answer

In Pre-LN, each sublayer normalises its input before processing: $x' = x + \text{Sublayer}(\text{LN}(x))$. The residual stream carries unnormalised values, so gradients can flow directly back through the residual connection without passing through normalisation. This makes deep networks easier to train because gradient magnitudes remain better behaved. In Post-LN, normalisation happens after the residual: $x' = \text{LN}(x + \text{Sublayer}(x))$. In deep models this can cause vanishing gradients early in training because the output is normalised but the residual path is not explicitly protected. Post-LN models often require careful learning rate warmup to train stably, while Pre-LN models are generally more robust to initialisation and learning rate choice. The original ViT and GPT-style models both use Pre-LN for this reason.

Question 4

ViTs apply learned absolute positional embeddings that are added to the patch tokens before the encoder. Explain why positional information is necessary in a Vision Transformer, what limitation this particular encoding has, and what alternatives address that limitation.

Sample Answer

Self-attention is permutation-invariant: without positional information, the model cannot distinguish different spatial arrangements of the same set of patches. An image of a cat in the top-left would look identical to one in the bottom-right. Learned absolute positional embeddings assign a distinct learnable vector to each position and add it to the corresponding patch embedding before the encoder. The limitation is that these embeddings are tied to a fixed resolution: if the input resolution changes at test time, the learned table must be interpolated, which can hurt performance because the model never saw those positions during training. Alternatives include sinusoidal encodings, which are parameter-free and can generalise to different lengths; relative position biases, which encode pairwise offsets rather than absolute positions and are used in Swin Transformer; and RoPE, which rotates query and key vectors by a position-dependent angle and has strong resolution extrapolation properties.

Question 5

The lecture argues that Vision Transformers matter not just for image classification but as a foundation for modern generative and multimodal systems. Explain the conceptual argument for why the same patch-token abstraction that works for classification also works as a backbone for diffusion models such as DiT.

Sample Answer

The patch-token abstraction converts an image into a sequence of vectors that a transformer can process with self-attention. For classification, the encoder contextualises this sequence and a head reads the class token. For diffusion, the same idea applies: noisy latent patches are treated as tokens, the transformer processes them with self-attention to model long-range spatial dependencies, and the output is a prediction of the noise or velocity at each patch position. Conditioning on the timestep and class label can be handled by injecting additional embeddings, just as the class token is injected for classification. The key insight is that the transformer is not inherently a classifier or a denoiser; it is a general sequence processor. Once images are expressed as token sequences, the same architectural machinery can serve classification, generation, and cross-modal alignment by changing only the input, conditioning, and output head. DiT demonstrated that this approach scales

well and outperforms CNN-based U-Nets on diffusion.

Chapter 8 — Vision-Language Models and CLIP

The Chapter 8 notebook implements a CLIP-like model from scratch, including the similarity matrix, the symmetric contrastive loss, zero-shot classification, knowledge distillation from a teacher CLIP, and a fusion encoder-decoder VLM on MNIST. The questions follow the same conceptual, reasoning-based style.

Question 1

CLIP normalises image and text embeddings to unit length before computing similarity. Explain what the dot product between two normalised embeddings measures, and why this normalisation is important for the contrastive training objective.

Sample Answer

After L2 normalisation, each embedding vector has unit length. The dot product between two unit vectors equals their cosine similarity, which measures only the angle between them and ignores their magnitudes. This is important because embedding magnitudes can be influenced by factors unrelated to semantic content: training dynamics, batch effects, or the overall scale of the encoder outputs. If magnitudes were included, the model could trivially increase similarity between matched pairs by making all embeddings larger rather than by improving alignment. Normalisation forces the model to compete on the basis of direction in semantic space, which is the meaningful quantity for retrieval and zero-shot classification. It also stabilises the geometry of the embedding space and makes similarity comparisons consistent across examples.

Question 2

CLIP uses a symmetric contrastive loss that applies cross-entropy in both directions: image-to-text and text-to-image. Explain why both directions are used, and what would be missing if only the image-to-text direction were optimised.

Sample Answer

The symmetric loss ensures that the embedding space works well for retrieval in both directions. The image-to-text direction trains the model so that given an image, the correct text ranks highest among all texts in the batch. The text-to-image direction trains the model so that given a text, the correct image ranks highest among all images in the batch. If only the image-to-text direction were optimised, the model would learn to find texts for images reliably, but the reverse direction could be poor. In particular, the text encoder might not be pushed to produce embeddings that are discriminative enough to distinguish between similar images. The symmetric objective ensures both encoders are trained to produce high-quality representations for retrieval in both directions, which is essential since CLIP is used for both text queries over images and image queries over text.

Question 3

Zero-shot classification in CLIP uses text prompts of the form "a photo of a {class}" rather than just the class name. Explain why prompt wording matters, and what the conceptual mechanism is that allows zero-shot classification to work at all.

Sample Answer

Prompt wording matters because CLIP was trained on web image-text pairs where captions naturally include descriptive phrases rather than bare class names. A prompt like "a photo of a cat" is more similar to the kind of text the model encountered during training than the single word "cat". Using a matching template aligns the test query with the training distribution, which typically improves accuracy. Different templates can give meaningfully different results. The mechanism that allows zero-shot classification to work is the shared embedding space. CLIP has learned to place semantically related images and texts close together. At test time, a new class can be described through natural language without any retraining, because the class meaning is mediated through the text encoder and the learned joint space rather than through a fixed trained weight vector tied to a specific label.

Question 4

The lecture implements knowledge distillation from a large teacher CLIP to a small student CLIP using similarity-matrix KL distillation rather than standard contrastive training. Explain conceptually why distilling from the teacher's similarity matrix is more informative than training the student with a standard contrastive loss from scratch.

Sample Answer

Standard contrastive training treats each batch as a binary problem: the diagonal entries are positive and all off-diagonal entries are equally negative. This ignores soft relationships between examples: an image of a dog may be 80% similar to its paired caption but 60% similar to another caption about dogs. The teacher's similarity matrix captures these soft relationships. Distilling from this matrix means the student learns not just what matches, but the relative degrees of semantic proximity across all pairs in the batch. This is richer supervision than binary positive-negative labels. Additionally, the teacher already provides a well-structured embedding geometry developed through large-scale training. The student inherits this structure rather than having to rediscover it from scratch, which makes learning more sample-efficient and better preserves the semantic relationships that make CLIP useful.

Question 5

The lecture compares a dual-encoder CLIP model with a fusion encoder-decoder VLM. Explain the architectural difference between the two families, and describe one task for which each family is better suited and why.

Sample Answer

In a dual-encoder model, the image and text are encoded independently by separate networks into fixed-size global vectors, which are then compared using cosine similarity. The two modalities never directly interact within the network; alignment happens only at the level of the final embedding vectors. In a fusion encoder-decoder model, the image encoder produces a set of visual memory tokens, and the text decoder uses cross-attention to consult those tokens at each generation step. The two modalities interact directly inside the network. Dual encoders are well suited to large-scale retrieval: both modalities can be embedded independently and cached, enabling fast nearest-neighbour search over millions of examples. Fusion models are better suited to tasks that require detailed grounding, such as visual question answering or caption generation, because each generated token can attend directly to specific parts of the visual memory, allowing the model to condition its output on fine-grained visual evidence rather than a single global image vector.

Chapter 9 — Training and Adapting Vision-Language Models

The Chapter 9 notebook implements the full VLM adaptation ladder on a Qwen2.5-VL model: zero-shot, few-shot, RAG, SFT with LoRA, DPO preference tuning, and RLOO reinforcement learning. The questions follow the same conceptual style as the sample exam questions.

Question 1

LoRA introduces a low-rank update $W' = W + B \cdot A$, where W is frozen and B, A are learned. Explain conceptually why learning a low-rank update rather than updating the full matrix is a principled approach for adapting a pretrained language model, and not merely a parameter-saving trick.

Sample Answer

The hypothesis behind LoRA is that the meaningful changes needed to adapt a pretrained model to a new task lie in a low-dimensional subspace of the full weight space. A pretrained model has already developed rich, general representations through large-scale training. Fine-tuning requires only small, structured adjustments to redirect these representations toward the new task, rather than relearning everything from scratch. A low-rank matrix $B \cdot A$ can represent exactly such a structured adjustment: it captures the dominant directions of change without adding noise across all dimensions. Updating the full matrix allows any change, including ones that are harmful or redundant. The low-rank constraint acts as a form of regularisation that encourages compact, meaningful updates. In this sense, LoRA is principled because it encodes an inductive bias about what useful adaptation should look like, not just a method for reducing parameters.

Question 2

The SFT training loss includes a token mask that assigns zero weight to user and system tokens and nonzero weight only to the assistant response tokens. Explain why this masking is conceptually important, and what the model would learn without it.

Sample Answer

The mask ensures that the model is trained to predict the assistant's responses, not to imitate the user's questions or the system prompt. In a chat-style training format, the full sequence includes user instructions, system context, and assistant replies. If all tokens were included in the loss equally, the model would receive gradient signal for reproducing the user's queries and system text. This would not teach the model to generate good responses; it would instead teach it to echo its own input context. More subtly, the model would learn an entangled objective that mixes response generation with prompt reproduction, making it harder to learn what a good assistant answer looks like for a given question. The mask cleanly separates the learning signal: only the assistant turns matter, which is precisely what instruction tuning is designed to teach.

Question 3

DPO uses a frozen reference policy throughout training. Explain why this reference policy is conceptually necessary, and what would go wrong if it were updated jointly with the

trained model.

Sample Answer

The DPO loss measures how much more the trained model prefers the chosen response over the rejected one, relative to how the reference policy scores the same responses. This relative comparison is what provides a stable learning signal. If the reference policy were also updated, the baseline would shift as training progresses. The relative log-ratio would then be computed against a moving target, making the optimisation objective ill-defined: the model could satisfy the loss by moving in arbitrary directions that also happen to move the reference in the same direction, without actually improving preference alignment. The frozen reference also plays the role of a KL regulariser: it penalises the trained model from drifting too far from the pretrained behaviour, which helps prevent the model from becoming degenerate or losing general capabilities while optimising for preferences.

Question 4

The chapter presents retrieval-augmented generation as a form of latent-variable model over retrieved documents. Explain this probabilistic interpretation, and why RAG is often a more principled first intervention than SFT when a VLM fails due to missing domain knowledge.

Sample Answer

In the probabilistic view, RAG marginalises over possible retrieved documents: the model generates an answer by weighting different documents by how plausibly they explain the gold answer. Each document is a latent variable, and the responsibility weight captures not just retrieval probability but also explanatory usefulness. This framing matters because it clarifies what RAG is doing: it is conditioning generation on external evidence, not just searching for relevant text. When a model fails because it lacks factual knowledge, the bottleneck is the absence of evidence at inference time, not a learning failure. SFT addresses a different problem: it changes what the model imitates, but it does not give the model access to new evidence at test time. DPO is even further removed: it adjusts preferences between answers without adding any factual grounding. RAG directly attacks the missing-knowledge problem by supplying the relevant evidence at the moment of generation, which is why it is often the most principled first step when the failure is factual rather than stylistic.

Question 5

RLOO generates multiple completions for the same prompt and uses a leave-one-out baseline to compute advantages. Explain what a leave-one-out baseline is, why it reduces variance compared to a global baseline, and what conceptual problem it addresses that a simple global reward average cannot.

Sample Answer

In RLOO, for a batch of G completions of the same prompt, the advantage of completion i is its reward minus the average reward of all other completions for the same prompt: $A_i = r_i - (1/(G-1)) * \sum_{j \neq i} r_j$. The baseline is therefore prompt-specific rather than global. A global baseline averages rewards across all prompts and completions in the dataset. This fails to account for the fact that some prompts are inherently easier than others: if every completion of an easy prompt receives high rewards, a global baseline cannot separate prompt difficulty from completion quality. The leave-one-out baseline removes this confound by asking: is this particular completion better or worse than what the model typically produces for this specific prompt? This makes the gradient signal

cleaner because it isolates the quality of the completion relative to other completions of the same question, rather than relative to completions of unrelated prompts. The result is lower variance in the gradient estimate and more stable training.

ECE786

Master Exam Reference

Generative Models & Vision-Language Systems

Spring 2026 · Prof. Zualkernan · Open-Book Exam Document

What this document contains

- Tag **[Q]**: All sample exam questions with full model answers (from professor).
- Tag **[F]**: All flashcard Q&A (Ch 4, 5, 7 — 150 cards extracted).
- Tag **[X]**: Additional likely exam questions covering lecture objectives.
- Tag **[C]**: Code-focused questions from the actual lecture notebooks (Ch4, 7, 8, 9).
- Each chapter ends with a quick formula sheet.
- Concept index at the end for fast lookup during the exam.

How to use during exam

1. Scan the Table of Contents to jump to the relevant chapter.
2. The sample questions [Q] are the most likely formats.
3. The flashcards [F] cover finer details and specific terms.
4. The additional [X] questions cover learning objectives.
5. Use the concept index (last page) to find any topic by keyword.

Table of Contents

Chapter	Topic	Content
Chapter 02	Variational Autoencoders (VAE)	5 sample QA + 10 extra
Chapter 03	Diffusion Models	5 sample QA + 10 extra
Chapter 04	Advanced Diffusion	5 sample + 50 flash + 8 code QA
Chapter 05	Flow Matching	5 sample QA + 50 flashcards
Chapter 06	Transformers and LLMs	5 sample QA + 11 extra
Chapter 07	Vision Transformers (ViT)	5 sample + 50 flash + 7 code QA
Chapter 08	Vision-Language Models	5 sample + 9 extra + 6 code QA
Chapter 09	Training and Adapting VLMs	5 sample + 10 extra + 8 code QA
Chapter 10	Quantization	5 sample QA + 9 extra
Formula Sheet	Quick reference for all chapters	Last section
Concept Index	Alphabetical keyword lookup	Last page

Chapter 02 — Variational Autoencoders (VAE)

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 Why is the ELBO called a lower bound on $\mathbb{E}_{q(z)}[\log p(x|z)]$, and under what condition does it become exact?

The ELBO is called a lower bound because Since KL divergence is always nonnegative, it follows that for every datapoint. The bound becomes exact when the approximate posterior matches the true posterior: In that case the KL gap is zero. This shows that VAE learning is not only about learning a decoder; it is also about learning an inference model that makes the bound tight.

Q2 Interpret the two main terms in the ELBO, and explain the tension between them.

The expectation term rewards the model for explaining the observation well using latent values that are plausible for that same observation under $q(z|x)$. In practice, this plays the role of a probabilistic reconstruction objective. The KL term regularizes the approximate posterior so that it does not drift too far from the prior. This encourages a more organized latent space that supports sampling and reduces the risk that each datapoint receives an isolated, arbitrary code. The tension is that better reconstruction often pushes the encoder to store more information about in the latent variable, while stronger KL pressure pushes the latent representation toward a simpler distribution. A VAE must balance fidelity to the data against structure and regularity in latent space.

Q3 Explain the reparameterization trick conceptually. Why is it more than just a coding convenience?

The reparameterization trick rewrites a latent sample as $z = \mu + \sigma \epsilon$. This separates randomness from the learnable parameters. The noise is drawn from a fixed source, while the encoder controls how that noise is transformed into a latent sample. Conceptually, this matters because it makes the stochastic step behave like a differentiable computation. Gradients can then flow through the latent sample back into the encoder parameters. Without this idea, sampling would interrupt ordinary backpropagation and make learning much harder. So the trick is not merely implementation detail. It is one of the key ideas that makes variational autoencoders trainable with standard deep learning methods.

Q4 A standard VAE does not automatically guarantee disentangled latent variables. Why not, and how do ideas such as β -VAE or CVAE try to change this?

A standard VAE encourages reconstruction quality and approximate posterior regularity, but those pressures alone do not force separate latent coordinates to align with separate semantic factors. The model may still encode several factors together in entangled ways if that helps optimize the objective. A β -VAE modifies the tradeoff by increasing the weight on the KL term: When β is large, the model is pushed toward a more constrained latent representation, which can encourage factor separation, although often at the cost of reconstruction quality. A CVAE introduces side information such as labels or attributes. By explicitly conditioning on known factors, it reduces the burden on the latent variables and can produce a more structured representation. In both cases, the broader lesson is that disentanglement usually requires extra inductive bias rather than emerging automatically.

Q5 Suppose a VAE encoder learns an approximate posterior that is almost identical to the prior for every input, so that for all x . What does this suggest about how the model is using its latent variables, and what consequence would you expect for reconstruction quality?

If $q_\phi(z|x)$ is nearly the same as for every input, then the latent representation carries very little information about the specific datapoint x . In effect, the encoder is no longer distinguishing one input from another in a meaningful probabilistic way. This usually means the decoder is being forced to reconstruct without relying much on the latent code. If the decoder is powerful enough, it may still produce reasonable outputs by learning generic patterns in the data, but reconstruction quality for individual examples will typically suffer because the latent variable is not providing input-specific information. Conceptually, this reflects a failure to use the latent space as an informative hidden representation. The model may still optimize part of the objective, especially the KL term, but it is no longer achieving the intended balance between informative encoding and regularized generation.

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 Why does a deterministic autoencoder fail as a generative model?

Three reasons: (1) it does not define a probability model — no likelihood $p(x)$ can be computed; (2) its latent codes occupy arbitrary, disconnected regions of latent space — there are 'holes'; (3) sampling $z \sim N(0, I)$ and decoding produces nonsense because the AE was never trained for that. VAE fixes all three by making z probabilistic and regularising it toward $N(0, I)$.

X2 Write the standard VAE generative model. Why is the marginal $p_\theta(x)$ intractable?

Prior $p(z) = N(0, I)$, decoder $p_\theta(x|z)$ (neural network), joint $p_\theta(x, z) = p_\theta(x|z)p(z)$. Marginal: $p_\theta(x) = \int p_\theta(x|z)p(z) dz$. This integral is intractable because the decoder is a nonlinear neural network — there is no closed form for integrating over a high-dimensional continuous z .

X3 State Jensen's inequality and explain how it gives the ELBO.

For a concave function f and random Y : $f(E[Y]) \geq E[f(Y)]$. Applied to $\log$: $\log(E_q[p(x, z)/q(z|x)]) \geq E_q[\log p(x, z)/q(z|x)]$. The left side is $\log p(x)$ (intractable). The right side is the ELBO (tractable). Jensen guarantees $\text{ELBO} \leq \log p(x)$ always — so maximising it is a valid surrogate.

X4 Write the ELBO in its two-term form and explain each term.

$\text{ELBO} = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$. First term (reconstruction): rewards the decoder for explaining x from z . Second term (KL regularisation): pulls the encoder distribution toward the prior $N(0, I)$. If reconstruction term is missing: blurry outputs, z not used. If KL is missing: latent space becomes disconnected, sampling fails.

X5 Give the analytical KL formula between a diagonal Gaussian and $N(0, I)$.

$\text{KL}(N(\mu, \sigma^2 I) \parallel N(0, I)) = 0.5 * \sum (\sigma^2 + \mu^2 - 1 - \log(\sigma^2))$. This is computed analytically — no sampling required. Only the reconstruction term $E[\log p(x|z)]$ needs Monte Carlo. This is why Gaussians are special for VAEs.

X6 State the reparameterisation trick and explain why it is critical.

$z = \mu_\phi(x) + \sigma_\phi(x) * \epsilon$, where $\epsilon \sim N(0, I)$. The randomness lives only in ϵ (no learnable parameters); z is a deterministic, differentiable function of (μ, σ) . This lets gradients flow through the sampling step to the encoder. Without it, the ELBO gradient with respect to ϕ would require differentiating through a stochastic node — impossible with standard backprop.

X7 What is posterior collapse and what causes it?

Posterior collapse: $q_{\phi}(z|x)$ approximates $p(z) = N(0, I)$ for ALL inputs x . The latent variable z carries no information about x — the decoder ignores it. Causes: (1) decoder too powerful (can reconstruct without z); (2) KL pressure too strong early in training. Consequence: KL term is trivially minimised but the model fails as a structured generator.

X8 What is Beta-VAE? What does Beta > 1 do and what's the cost?

Beta-VAE modifies the loss: $L = E[\log p(x|z)] - \beta * KL(q||p)$. When $\beta > 1$, the model is forced toward more compressed, factorial latent codes — this can promote disentanglement. Cost: reconstruction quality degrades because the latent bottleneck becomes too restrictive. The objective trades fidelity for structure.

X9 What is CVAE? How does conditioning help with entanglement?

CVAE conditions both encoder and decoder on extra information y : $q_{\phi}(z|x, y)$, $p_{\theta}(x|z, y)$. The latent z only needs to model variation that y does NOT explain (residual variation). This reduces what z must capture, often producing more controllable and less entangled representations. Useful when part of the variation is known (e.g., class labels).

X10 Why is disentanglement not automatic in a standard VAE?

The ELBO rewards reconstruction and latent regularity (toward $N(0, I)$) but does not force separate latent dimensions to align with separate semantic factors. The model is free to entangle multiple factors if that optimises the ELBO. Disentanglement requires additional inductive bias (Beta-VAE) or conditioning (CVAE), or strong supervision.

Chapter 03 — Diffusion Models

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 What is the conceptual difference between $q(x_{1:T})$ and $q(x_t)$, and why is that distinction important in diffusion?

$q(x_{1:T})$ is the probability distribution over the entire noising trajectory starting from the clean sample x_0 . It is a distribution over a whole sequence of latent states. By contrast, $q(x_t)$ is only the marginal distribution of one particular noisy state at time t . This distinction matters because training usually needs only a randomly chosen noisy observation x_t , not the full path. The existence of the closed-form marginal lets us directly sample training pairs efficiently. The full trajectory is conceptually important for understanding diffusion as a Markov process, but the marginal is what makes practical optimization easy.

Q2 Why is the true reverse conditional not directly usable in realistic diffusion modeling, even though the forward process is known?

Knowing the forward conditional does not automatically give us a usable form for the reverse conditional on real data. The reverse distribution depends on the population-level distribution of x_0 induced by the data. In other words, Bayes' rule introduces terms involving the unknown data distribution. So although the local forward corruption rule is simple and Gaussian, the exact reverse direction is only analytically accessible in special cases. For natural images or other complex data, the reverse conditional is effectively intractable. This is why diffusion models learn a parametric approximation from data rather than derive it exactly.

Q3 Why does predicting the added noise make sense as a training target? Why is it not an arbitrary engineering trick?

Predicting ϵ_t is natural because under the forward marginal we can write $x_t = x_0 + \sum_{s=0}^{t-1} \sqrt{\alpha_s - \alpha_{s+1}} \epsilon_s$. Once x_0 and α_t are given, the unknown part is precisely the noise realization that corrupted the clean sample. Estimating that noise tells the model how far the sample has been pushed away from the underlying data structure. This target is also mathematically linked to denoising and score estimation. For Gaussian perturbations, the score can be written in a form proportional to the noise term. Therefore, predicting ϵ_t is a convenient surrogate for learning the local reverse direction. It is simple computationally, but it is also principled statistically.

Q4 Why must the denoising network know the timestep t ? Could one imagine a single denoiser that receives only x_t ?

In general, x_t alone is not sufficient because similar-looking noisy observations may correspond to very different denoising regimes. Early timesteps retain most of the original signal, so denoising should be gentle and detail-preserving. Late timesteps may be nearly pure noise, so denoising requires a much more global and uncertain inference. Without time information, the mapping from x_t to the appropriate denoising update becomes ambiguous. Conditioning on t resolves that ambiguity by telling the model the current signal-to-noise level. Conceptually, the timestep is part of the state of the reverse problem, not merely a bookkeeping variable.

Q5 Why are diffusion models often described as both latent-variable models and score-based models? Are these descriptions contradictory?

These descriptions are not contradictory; they emphasize different mathematical viewpoints. Diffusion models are latent-variable models because the trajectory acts as a collection of latent variables mediating the generation of x_0 . The full model can be written as a joint distribution over these hidden states. They are also score-based models because, under suitable parameterizations, the learned denoiser implicitly estimates score-like quantities such as gradients of log densities at noisy intermediate levels. This score information determines how to move samples from low-density noisy regions toward high-density data regions. Thus, one viewpoint emphasizes probabilistic graphical structure, while the other emphasizes geometric denoising directions in data space. The two are compatible descriptions of the same underlying process.

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 State the Markov property and the joint factorization for a diffusion chain.

Markov property: $P(x_t | x_{t-1}, x_{t-2}, \dots, x_0) = P(x_t | x_{t-1})$ — the future depends only on the present. Joint factorisation: $P(x_{0:T}) = P(x_0) * \text{PRODUCT}_{t=1..T} P(x_t | x_{t-1})$. The forward chain in diffusion uses Gaussian transitions $q(x_t | x_{t-1})$.

X2 Write the forward process $q(x_t | x_{t-1})$ and the closed-form marginal $q(x_t | x_0)$.

Forward step: $q(x_t | x_{t-1}) = N(x_t; \sqrt{\alpha_t} x_{t-1}, (1-\alpha_t)I)$. Closed-form marginal: $q(x_t | x_0) = N(\sqrt{\text{abar}_t} x_0, (1-\text{abar}_t)I)$ where $\text{abar}_t = \text{PRODUCT}_{s=1..t} \alpha_s$. Equivalent form: $x_t = \sqrt{\text{abar}_t} x_0 + \sqrt{1-\text{abar}_t} \epsilon$, $\epsilon \sim N(0, I)$.

X3 Write the DDPM training loss in full.

$L_{\text{simple}} = E_{\{x_0, \epsilon, t\}} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2]$. Procedure: sample x_0 from data, sample $t \sim \text{Uniform}\{1, T\}$, sample $\epsilon \sim N(0, I)$, compute $x_t = \sqrt{\text{abar}_t} x_0 + \sqrt{1-\text{abar}_t} \epsilon$, compute MSE between true ϵ and predicted $\epsilon_{\theta}(x_t, t)$. This is an MSE on noise prediction.

X4 Why must the denoising network be conditioned on the timestep t ?

The denoising task is fundamentally different at different noise levels. At small t : x_t retains strong semantic signal; denoising should be gentle. At large t : x_t is nearly pure noise; denoising requires global, uncertain inference. Without t , the same x_t value could correspond to different noise levels, creating an ambiguous mapping. Sinusoidal embeddings give each t a unique fingerprint.

X5 Why are U-Nets the standard architecture for diffusion?

Denoising requires both global context (scene understanding) and fine spatial detail (textures, edges). U-Net combines: (1) downsampling path = multi-scale context; (2) upsampling path = spatial detail reconstruction; (3) skip connections = preserve spatial information across scales; (4) timestep injection at every level via sinusoidal embedding. This matches the multi-scale nature of image generation.

X6 Explain the difference between DDPM and DDIM sampling.

Both use the SAME trained model. DDPM: stochastic reverse process — noise injected each step; ~1000 steps for quality; higher diversity. DDIM: deterministic reverse trajectory; 10-100 steps for same quality; lower diversity; much faster. Key insight: training and sampling are independent — one trained model supports multiple sampling schemes.

X7 List the diffusion model families from the lecture.

DDPM (canonical, stochastic), DDIM (deterministic, same network), Score-SDE (continuous-time SDE generalisation), LDM (latent diffusion, e.g., Stable Diffusion), Guided (classifier guidance or CFG), Consistency (one-step distillation from teacher), Flow Matching (deterministic ODE, learns velocity field).

X8 Why is the linear vs cosine noise schedule choice important?

The schedule β_t (equivalently $\bar{\alpha}_t$) controls how fast information is destroyed. Linear schedule destroys information too quickly early on. Cosine schedule allocates more steps to the semantically meaningful mid-range, often improving sample quality. The schedule shapes the SNR profile over time — it determines how semantics are distributed across timesteps.

X9 What's the relationship between predicting noise and estimating the score?

For Gaussian forward noise: $\nabla_{x_t} \log q(x_{t|x_0}) = -\epsilon / \sqrt{1-\bar{\alpha}_t}$. So predicting ϵ is equivalent to estimating the score up to a scaling factor. A noise-predicting network is implicitly learning the score field. This is why diffusion models can be described as either latent-variable models OR score-based models — same underlying object, two viewpoints.

X10 Why can't we directly use the true reverse $q(x_{t-1}|x_t)$?

By Bayes rule: $q(x_{t-1}|x_t) = q(x_{t|x_{t-1}})q(x_{t-1})/q(x_t)$. The marginals $q(x_t)$ and $q(x_{t-1})$ require integrating over the unknown data distribution — intractable for real data. So while the forward step $q(x_{t|x_{t-1}})$ is simple Gaussian, the reverse on real data is not analytically accessible. This is why $p_\theta(x_{t-1}|x_t)$ must be learned.

Chapter 04 — Advanced Diffusion (Score, Guidance, FID)

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 What is the theoretical basis of classifier guidance?

Classifier guidance follows from Bayes' rule. Since taking gradients with respect to gives The term is the unconditional score learned by the diffusion model, while comes from the classifier. Adding them produces a guided score that encourages samples to remain realistic while becoming more aligned with the desired condition.

Q2 Why is a noise-aware classifier preferable to a standard classifier in classifier-guided diffusion?

The reverse chain queries the classifier on noisy states , not just on clean data . A classifier trained only on clean inputs may behave unpredictably when asked to evaluate heavily corrupted samples. In that case, its gradient can be weak, unstable, or semantically meaningless. A noise-aware classifier is trained on pairs produced by the same forward diffusion corruption process. As a result, it learns how class evidence appears across noise levels and produces gradients that are relevant to the actual reverse trajectory. This makes guidance more stable and conceptually consistent with the diffusion framework.

Q3 What is the core idea behind classifier-free guidance, and why is random condition dropping essential?

Classifier-free guidance trains a single diffusion model to produce both conditional and unconditional predictions. During training, the condition is randomly removed for some examples and replaced by a null token. This forces the model to learn both and At sampling time, the two predictions are combined so that the conditional component is amplified relative to the unconditional one. The random dropping step is essential because without it the model would never learn a true unconditional branch, and the subtraction-based guidance mechanism would have no meaningful reference point.

Q4 Why does stronger guidance usually improve condition adherence but reduce diversity?

Guidance works by amplifying the component of the reverse update that favors the desired condition. As the guidance scale increases, the sampler is pushed more aggressively toward condition-consistent regions. This usually improves alignment with the class, text prompt, or other conditioning signal. However, the same amplification shrinks the effective set of plausible outputs, because trajectories that do not strongly support the condition are suppressed. The process becomes more concentrated, which often reduces diversity and can introduce artifacts if the guidance overwhelms the learned unconditional structure. Thus guidance strength controls a realism-versus-control-versus-diversity tradeoff rather than delivering a free improvement.

Q5 What does FID measure conceptually, and what are its limitations when evaluating diffusion models?

FID compares the distributions of real and generated samples in a deep feature space. It summarizes each feature distribution by a mean and covariance, then computes a Fréchet distance between the two Gaussians. Conceptually, it measures whether generated samples reproduce the central tendency and spread of real data in a semantically meaningful representation space. Its limitations are equally important. It depends on the chosen feature extractor, assumes a Gaussian approximation in feature space, can be sensitive to sample size and numerical instability, and may not fully capture controllability or prompt alignment. Therefore, FID is useful but incomplete; it should be interpreted alongside task-specific and qualitative evidence.

Flashcard Q&A (50 cards)

Quick-fire questions on specific concepts, terms, and details.

F1 Why is the Evidence Lower Bound (ELBO) optimized in diffusion models instead of the marginal likelihood?

The marginal likelihood is computationally intractable.

F2 In the diffusion ELBO, what does the 'prior matching' term $\text{att} = T$ represent?

How well the final noised distribution matches the Gaussian prior.

F3 What simplification allows the diffusion KL terms to reduce to a Mean Squared Error (MSE) objective?

Fixing the variance and assuming a Gaussian forward posterior.

F4 What is the mathematical definition of a 'score' $S(x)$ in diffusion modeling?

$S(x) = \nabla \log p(x)$

F5 Geometrically, what does the score function represent on a probability density curve?

The slope of the log-density curve pointing toward higher probability.

F6 For a standard Gaussian distribution $N(0, 1)$, what is the simplified score function?

$S(x) = -x$

F7 Gaussian distribution, what direction does a negative score at a positive x point toward?

Left, back toward the mean.

F8 Why does following the score field in a bimodal distribution steer samples toward modes?

The score always points toward the nearest peak/region of higher probability.

F9 According to the denoising-score identity, what does the network's noise prediction actually estimate?

The score field of the distribution at that specific noise level.

F10 What is the relationship between the score $S_t(x_t)$ and the predicted noise $\epsilon_{\theta}(x_t, t)$ at timestep t ?

$\epsilon_{\theta}(x_t, t) = \sigma_t^2 S_t(x_t)$

F11 What are the two primary components of the reverse diffusion sampling step?

Denoising drift (deterministic) and Sampling randomness (stochastic).

F12 the reverse process, what is the purpose of adding 'calibrated noise at each step?

To ensure the model samples from the full distribution rather than collapsing to a single mode.

F13 Gas) Hh Why is time embedding essential for the neural network in diffusion models?

The model must behave differently depending on whether the input is mostly noise or almost clean.

F14 FiLM-style time conditioning, how does the time embedding modify internal feature maps?

By adding a time-dependent, learned bias to each channel.

F15 What is the function of the 'reconstruction term' in the diffusion ELBO?

It measures the model's ability to predict the clean data x_0 from x_j .

F16 What is the core idea of 'Guided Diffusion'?

Modifying the reverse drift so samples flow toward images consistent with a specific condition.

F17 How does Classifier Guidance mathematically modify the unconditional score $S(x)$?

$S_{\text{guided}}(x) = S(x) + W \nabla \log p(y|x)$

F18 What does the parameter W represent in guided diffusion?

The guidance scale, balancing conditioning strength against sample diversity.

F19 Classifier-Free Guidance (CFG), how is the 'unconditional' behavior learned by the same model?

By randomly replacing class labels with a special 'null' token during training.

F20 What is the formula for the CFG noise prediction E_{cfg} combining conditional and unconditional outputs?

$E_{\text{cfg}} = (1 + W)E_{\text{cond}} - WE_{\text{uncond}}$

F21 What is a primary advantage of Classifier-Free Guidance over standard Classifier Guidance?

It does not require training an auxiliary classifier or backpropagating through one during sampling.

F22 How does increasing the guidance scale W typically affect the resulting samples?

It improves alignment with the condition but reduces the diversity of the samples.

F23 What does Fréchet Inception Distance (FID) measure to evaluate generative models?

The distance between the feature distributions of real images and generated images.

F24 Why is a 'noise-aware' classifier required for standard Classifier Guidance?

The classifier must be able to provide accurate gradients for noisy intermediate states x_z .

F25 Concept: Denoising drift

As a Concept: Denoising drift The deterministic part of the reverse step that pushes mass back toward regions of higher probability.

F26 Why does a very high guidance scale W sometimes lead to artifacts in generated images?

It over-amplifies specific features of the condition, potentially moving out of the natural data manifold.

F27 What is the 'null token' in the context of Classifier-Free Guidance?

A placeholder used during training to allow the model to learn an unconditional score field.

F28 the 1D bimodal example, what happens to the score between the two peaks?

It is a compromise weighted by the plausibility of each mode at that position.

F29 What is the mathematical 'Bayes' foundation for Classifier Guidance?

$V_x \log p(x|y) = V_x \log p(x) + V_x \log p(y|x)$

F30 How does 'Evaluation Mode' (e.g., .eval()) affect diffusion sampling in PyTorch?

It ensures stable, deterministic score fields by freezing Dropout and Batch Normalization layers.

F31 Why is the score function considered a 'pull' toward the expected value?

It points in the direction where probability increases most sharply, which leads to the mean in unimodal distributions.

F32 What is the relationship between noise prediction and variational inference in diffusion models?

Minimizing noise prediction error is equivalent to maximizing the ELBO.

F33 The standard sampling loop starts with X_T sampled from which distribution?

A pure standard Gaussian noise distribution $N(0, I)$.

F34 How does a 'deterministic' reverse process (no added noise) behave?

It acts like gradient ascent on the log-density, typically collapsing samples to modes.

F35 Why is it sensible to differentiate a classifier with respect to the 'reverse mean' in guided diffusion?

The mean is the model's current proposal for the next image state.

F36 What does a lower FID value signify regarding a generative model?

Higher realism and better diversity compared to the training set.

F37 Gas) ba In CFG training, if $D_{\text{drop}} = 0.1$, what percentage of the time does the model see the real label?

90 percent.

F38 What does the 'drift + diffusion' update rule $X_{t-1} = X_t + \text{BeS}(X_t) + \sqrt{\epsilon} \epsilon$ represent?

A discrete Langevin-style step combining the score-guided drift and stochastic noise.

F39 FiLM conditioning, why is the time embedding projected into different output sizes?

To match the varying channel widths of different convolution blocks in the U-Net.

F40 What is the role of the α_t term in the forward diffusion marginal $q(X_t|X_0)$?

It defines the schedule for how much of the original image X_0 is preserved at step t .

F41 How does the score function $S(X)$ for a Gaussian change as X moves further from the mean?

The magnitude of the score increases linearly.

F42 What is the primary trade-off when using a small number of diffusion steps L ?

Faster sampling at the cost of lower image fidelity and weaker guidance.

F43 the noise-form reverse update, what term represents the 'predicted error' that is subtracted?

$\epsilon - \mathbb{E}(\epsilon | X_t, C)$

F44 Why is normalization (e.g., to $[-1, 1]$) important for diffusion stability?

It ensures consistent scaling between the model's noise predictions and the actual added Gaussian noise.

F45 Gas) and What happens to the sampling process if the classifier and diffusion model use inconsistent normalization?

The guidance gradient will be in the wrong scale regime, making conditioning unreliable.

F46 ELBO decomposition, what do the intermediate KL terms ensure?

That the learned reverse step $P_\theta(x_{t-1} | x_t)$ matches the true forward posterior $P(x_{t-1} | x_t, x_0)$.

F47 How does Classifier-Free Guidance 'extrapolate' beyond the conditional model?

By pushing the noise prediction further away from the unconditional prediction toward the conditional one.

F48 What is the 'denoising score matching' objective basically doing?

Training a model to push noisy data points back toward the high-density regions of the original distribution.

F49 the bimodal score derivation, what are 'responsibilities' $r_k(x)$?

Weights that determine how much each mode contributes to the total score at a given point x .

F50 Why do diffusion models start with x_T as pure noise?

The forward process is designed to destroy all data information, leaving only a standard Gaussian distribution.

Code-Level Questions (from lecture notebooks)

Questions about the actual code in the lecture notebooks: implementation details, design choices, common patterns. Useful if the exam asks you to interpret or write code.

C1 The lecture code computes `score = -x` for a standard Gaussian. Derive this from first principles.

For $N(0,1)$: $\log p(x) = -x^2/2 + \text{const}$. The score is $d/dx \log p(x) = -x$. For $N(\mu, \sigma^2)$: $\text{score} = -(x-\mu)/\sigma^2$. This means the score always points toward the mean: if $x > \text{mean}$, score is negative (pull left); if $x < \text{mean}$, score is positive (pull right). The lecture visualises this with three plots: Gaussian density (bell curve), log-density (downward parabola), and score (straight line through zero with slope -1).

C2 In the lecture, `score_mix = np.gradient(logp_mix, x)` computes the bimodal score numerically. What is the analytical formula and why does the lecture use `np.gradient` instead?

Analytically, for $p(x) = 0.5 \cdot N(\mu_1, \sigma^2) + 0.5 \cdot N(\mu_2, \sigma^2)$: $\text{score} = r_1(x) \cdot (-(x-\mu_1)/\sigma^2) + r_2(x) \cdot (-(x-\mu_2)/\sigma^2)$ where $r_k(x) = p_k(x) / p(x)$ are the responsibilities (posterior mixing weights). Between the two modes the score is a weighted average pulling toward the nearest mode. `np.gradient` is used for simplicity in the lecture: it computes finite differences numerically. Both give the same result; the analytical form is used in the math derivation section.

C3 The lecture's `q_sample` function has this line: `sqrt_abar = extract(torch.sqrt(alphas_bar), t, x0.shape)`. What does `extract` do and why is it needed?

`extract(a, t, x_shape)` does two things: (1) `a.gather(0, t)` — selects the scalar `alpha_bar` value at each timestep index `t` for each sample in the batch, so if `t = [3, 7, 1]` it picks `a[3]`, `a[7]`, `a[1]`. (2) `.view(-1, *[1]*(len(x_shape)-1))` — reshapes from `(B,)` to `(B,1,1,1)` for broadcasting. Without this reshape, multiplying a `(B,)` tensor by a `(B,C,H,W)` tensor would fail or broadcast incorrectly. This pattern appears throughout the diffusion code everywhere schedule values need to be applied per-sample.

C4 The lecture training loop uses: `transforms.Normalize((0.5,), (0.5,))` to map images to `[-1, 1]`. Why does diffusion training specifically need `[-1, 1]` rather than `[0, 1]`?

Diffusion starts from $x_T \sim N(0, I)$ — zero-centered Gaussian noise. If `x_0` were in `[0, 1]`, the signal would be systematically positive while the noise is symmetric around zero. This asymmetry would make the denoising task harder: the network would need to learn a different effective prior. With `[-1, 1]` normalization, `x_0` is roughly zero-centered, matching the noise distribution. The reverse at `t=0` recovers values in `[-1, 1]` which must be clipped and mapped back: `(samples.clamp(-1, 1) + 1) / 2`.

C5 In the TinyUNet, time is injected as: `h1 = F.silu(self.conv1(x) + b1)` where `b1 = self.to_bias1(temb).unsqueeze(-1).unsqueeze(-1)`. Explain this FiLM-style injection line by line.

`self.to_bias1(temb)`: linear layer maps time embedding `(B, base*4)` → `(B, base)`. Each channel gets its own time-dependent bias. `.unsqueeze(-1).unsqueeze(-1)`: reshapes `(B, base)` → `(B, base, 1, 1)` for broadcasting over `H` and `W`. `self.conv1(x)`: produces `(B, base, H, W)` feature map. `+ b1`: broadcasts to add the same per-channel bias at every spatial location. `F.silu(...)`: Sigmoid Linear Unit activation (smoother than ReLU, empirically better for diffusion). This is bias-only FiLM (Feature-wise Linear Modulation): $h' = h + b(t)$. Full FiLM would also scale: $h' = \gamma(t)h + b(t)$. Without this injection, the network cannot distinguish noise levels and would learn a single average denoising function.

C6 The reverse sampling loop has `@torch.no_grad()` on `p_sample_loop` but the classifier-guided version `classifier_guided_sample` does NOT use `@torch.no_grad()`. Why?

Unconditional sampling (`p_sample_loop`) only needs forward passes through the diffusion model — no gradients required. `@torch.no_grad()` saves memory and speeds up the loop. Classifier-guided sampling needs `torch.autograd.grad(selected, x_in)` to compute the classifier gradient. This requires the autograd graph to be active for `x_in`. The solution in the lecture: `with torch.no_grad(): eps_pred = model(x, t)` disables grads for the expensive diffusion model, then `x_in = mean.detach().requires_grad_(True)` re-enables them ONLY for the classifier evaluation. This is the memory-efficient pattern: heavy diffusion network = no grad; lightweight classifier = grad enabled.

C7 In classifier-guided sampling, the lecture computes: `selected = logp[torch.arange(n, device=device), y].sum()` then `grad = torch.autograd.grad(selected, x_in)[0]`. Why `.sum()` before `.autograd.grad`? Would `.mean()` give the same gradient direction?

`.autograd.grad` requires a scalar output (single number) to differentiate. `logp[arange(n), y]` gives a vector of shape `(B,)` — one log-prob per sample. `.sum()` collapses to a scalar. Because `grad_i(sum(v)) = grad_i(v_i)` for each sample `i` independently, summing does not mix gradients across samples — each sample gets its own correct gradient. Using `.mean()` would give the same DIRECTION (same gradient direction for each sample) but scaled by `1/B`. The guidance formula `mean_guided = mean + w * beta_t * grad` absorbs the scale into `w`, so both work; `sum` is conventional.

C8 The lecture's CFG training uses: ``drop = (torch.rand_like(y.float()) < cfg_model.drop_prob)`` then ``y_in[drop] = null_token``. Why is the `null_token = 10` specifically (for 10-class MNIST)?

MNIST has classes 0-9, so valid labels occupy indices 0-9. The CFG model's label embedding is ``nn.Embedding(num_classes+1, base*4)`` — `num_classes+1 = 11` embeddings. `null_token = 10` uses the extra (11th) embedding slot as a dedicated 'no condition' token. This lets the model learn a distinct embedding for 'unconditional' that is separate from any real class. At sampling time, passing `y_null =` full tensor of 10s queries this unconditional branch. If we used an existing class index (e.g., `y=0` as null), the model could not distinguish 'class 0' from 'no class'.

C9 The TinyUNet uses ``nn.ConvTranspose2d(base*2, base, 4, stride=2, padding=1)`` for upsampling. Why `ConvTranspose2d` instead of bilinear interpolation + `Conv2d`?

`ConvTranspose2d` (transposed convolution / deconvolution) learns the upsampling weights — it is parametric. Bilinear+Conv is simpler and avoids checkerboard artifacts that transposed convolutions can produce. The lecture uses `ConvTranspose2d` as it is the classical U-Net upsampling choice and what most early diffusion papers used. `kernel_size=4, stride=2, padding=1` gives exact 2x upsampling: $\text{output_size} = (\text{input_size} - 1) * \text{stride} - 2 * \text{padding} + \text{kernel} = (n-1) * 2 - 2 + 4 = 2n$. Modern implementations often prefer bilinear+conv for training stability.

C10 The lecture trains a ``NoiseAwareClassifier`` that takes both ``x_t`` and ``t`` as inputs. Show the architecture difference from a standard image classifier and explain why ``t`` must be an input.

Standard classifier: `forward(x)` — single input, fixed decision boundary. `NoiseAwareClassifier`: `forward(x_t, t)` — two inputs: noisy image AND timestep. Time embedding is processed: `temb = sinusoidal_time_embedding(t) -> time_mlp(temb) -> biases b1, b2, b3`. These biases modulate the conv and FC layers via FiLM: `h = F.silu(conv(x) + b)`. Why `t` must be an input: the SAME visual pattern in `x_t` can mean completely different things at `t=10` (nearly clean) vs `t=900` (mostly noise). A time-unaware classifier would produce unreliable gradients at high noise levels — the lecture explicitly shows this is why 'noise-aware' classifiers are needed.

C11 In the final comparison, FID for CFG was -31.17. The lecture explains this as numerical instability. What specifically causes negative FID and how should you fix it in production code?

$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2 * \text{sqrtm}(\Sigma_r @ \Sigma_g))$. The matrix square root `sqrtm(Sigma_r @ Sigma_g)` is computed via eigendecomposition (`vals, vecs = eigh`; clip `vals >= 0`; reconstruct). With small sample size (`N=1000`), covariance estimates are noisy -> the matrix product `Sigma_r @ Sigma_g` may not be perfectly PSD. Clipping eigenvalues to `>= 0` (as the lecture does with ``np.clip(vals, 0, None)``) helps but small numerical errors can still make the trace negative. Production fix: ``fid = max(fid, 0.0)``. Also use larger `N` (`>= 10,000` samples) and add `epsilon*I` regularisation (the lecture adds `eps=1e-6` to both covariances). A negative FID means the two Gaussian approximations are extremely close — nearly perfect match.

Chapter 05 — Flow Matching

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 Why is flow matching best understood as a transport model rather than simply a regression model?

At the surface level, flow matching is trained with supervised regression: So one might think it is “just” vector regression. But this view is incomplete. The deeper object of interest is the induced dynamics which transports an entire probability distribution from to . The governing population-level equation is the continuity equation. Thus, the regression target is only the training mechanism. The learned object is a transport field that moves probability mass. The real conceptual content of the method lies in the induced distributional dynamics, not merely in minimizing an MSE loss.

Q2 What is the conceptual role of the interpolation path , and why is it not a trivial design choice?

The interpolation path defines what the model will regard as a plausible intermediate state between base and data samples. For a linear bridge, the target velocity is . But different bridges induce different intermediate geometries and different supervision signals. This changes: the distribution of training inputs , the target velocity seen at those inputs, the complexity of the velocity field required for accurate transport. So the bridge is not a harmless implementation detail. It expresses a modeling assumption about how probability mass should move through state space. Better bridges can make learning easier and sampling more stable.

Q3 Why can latent flow matching be preferable to pixel-space flow matching?

Pixel space is high-dimensional and contains large amounts of local detail, nuisance variation, and non-semantic structure. Learning a velocity field directly in this space can require the model to capture too much low-level complexity. Latent flow matching first defines a representation and then learns transport in the lower-dimensional latent space: This can help because: dimensionality is reduced, semantic structure is often more regular, transport trajectories may become simpler, computation can be cheaper. However, the gain depends on the latent representation being good. If the autoencoder discards important information, then the decoder becomes a bottleneck and sample fidelity may be capped.

Q4 Why do optimal-transport couplings matter even if the path formula remains linear?

Even if the bridge stays the choice of which is paired with which determines the geometry of the training trajectories. Random pairings can create unnecessarily long or crossing trajectories, which in turn force the model to learn a more complex vector field. OT couplings instead seek pairings that minimize expected transport cost: This often yields shorter, more coherent displacement directions and smoother supervision. So the path formula is only one ingredient; the coupling also shapes the difficulty of the learning problem.

Q5 Why are consistency models conceptually attractive, and why are they also fragile?

Consistency models are attractive because they aim to map multiple noisy versions of the same underlying sample to a shared canonical output: This suggests one-step or few-step generation, which is appealing from a sampling-speed perspective. However, the objective is fragile because consistency alone does not guarantee correctness. A trivial constant function can satisfy the consistency relation without preserving data structure. In addition, ambiguous high-noise states can drive the model toward conditional means, causing mode averaging or collapse. So consistency is powerful, but only when the objective is stabilized, usually by anchors, teacher guidance, or carefully designed training targets. $p_0 = p_{data}$ $v_\theta(x, t) = v_\theta(x, t)$, $t \in [0, 1]$.

Flashcard Q&A (50 cards)

Quick-fire questions on specific concepts, terms, and details.

F1 What is the role of the time-dependent velocity field $V_g(x, t)$ in flow matching?

It defines the direction and speed to move particles from a base distribution to a target distribution.

F2 What is the Ordinary Differential Equation (ODE) used for sampling in flow matching?

$\frac{dx}{dt} = v_\theta(x, t)$

F3 In the context of generative ODEs, what does the base distribution p_0 typically represent?

Simple Gaussian noise.

F4 In flow matching, what does the target distribution p_1 represent?

The complex target data distribution.

F5 Write the continuity equation that describes the evolution of probability density $p(x, t)$.

Write the continuity equation that describes the evolution of probability density $p(x, t)$: $\frac{\partial p}{\partial t} + \nabla \cdot (p v_\theta) = 0$

F6 In the continuity equation, what does the term $\nabla \cdot (p v_\theta)$ represent?

The probability flux divergence, measuring the net outflow of probability from a region.

F7 What is the intuitive meaning of the divergence $\nabla \cdot v_\theta$ at a point x ?

The vector field is expanding outward, creating a net outflow from that point.

F8 According to the continuity equation, why does density $p(x, t)$ increase in regions where the flow converges ($\nabla \cdot v_\theta < 0$)?

Probability mass accumulates because more flow enters the region than leaves it.

F9 Along a particle trajectory $x(t)$, how does the log-density change according to the continuity equation?

$\frac{d}{dt} \log p(x(t)) = -\nabla \cdot v_\theta(x(t), t)$

F10 What is a 'coupling' in the context of flow matching?

A rule or joint distribution used to pair samples from the base distribution p_0 with samples from the data distribution p_1 .

F11 Define the intermediate state x_t using the simplest linear interpolation bridge.

Define the intermediate state x_t using the simplest linear interpolation bridge. $x_t = (1 - t)x_0 + tx_1$

F12 What is the 'true velocity' target U_ϕ for a linear interpolation bridge $X_\phi = (1 - t)X_0 + tx_1$?

$U_\phi = X_1 - X_0$

F13 the flow matching training objective, what are the three components sampled to calculate the loss?

Base samples X_0 , data samples X_1 , and time $t \sim \text{Uniform}[0, 1]$

F14 Write the standard Flow Matching regression loss function $L(\theta)$.

Write the standard Flow Matching regression loss function $L(\theta)$. $L(\theta) = \mathbb{E}_{X_0, x_1, t} \mathbb{E}_{\phi \sim \text{Uniform}[0, 1]} \|V_\theta(x_\phi, t) - (X_1 - X_0)\|^2$ [17]

F15 Why is flow matching considered a supervised learning task?

The network V_θ is directly regressed against known target velocities U_t defined by the bridge.

F16 What is the simplest numerical method used to integrate the flow matching ODE during sampling?

The forward Euler method.

F17 Write the update rule for one step of Euler integration in flow matching sampling.

Write the update rule for one step of Euler integration in flow matching sampling. $X_{k+1} = X_k + \Delta t v_\theta(X_k, t_k)$

F18 Mathematically, the Euler update $X_{k+1} = X_k + \Delta t v_\theta(X_k, t_k)$ is derived from which expansion?

The first-order Taylor expansion.

F19 How does the Euler method approximate the integral solution $x(1) = x(0) + \int_0^1 v(x(t), t) dt$?

It replaces the continuous integral with a Riemann sum of instantaneous velocities.

F20 Gas) a In flow matching, $V(x, t)$ is a vector field. What does its magnitude $\|V\|$ represent?

The instantaneous speed of the particle at position x and time t .

F21 Gas) a How does Flow Matching avoid the 'score' bottleneck found in diffusion models?

It learns a deterministic velocity field via regression instead of estimating the gradient of the log-density (score).

F22 What is the primary difference between the sampling process of Flow Matching and standard Diffusion?

Flow Matching uses deterministic ODE integration, while Diffusion often uses stochastic SDE sampling.

F23 What is the purpose of using Optimal-Transport (OT) couplings in flow matching?

To pair X_0 and X_1 such that total transport cost is minimized, resulting in straighter, easier-to-learn trajectories.

F24 What is the cost function typically minimized by an OT coupling Π between P_0 and P_1 ?

$\mathbb{E}_{(X_0, x_1) \sim \Pi} \|X_0 - x_1\|^2$

F25 How does an OT coupling improve the 'geometry' of the learned velocity field?

By aligning X_0 with the closest x_1 , it reduces path crossing and ensures more coherent flow directions.

F26 What are the two learned modules in a Latent Flow Matching (LFM) architecture?

A latent representation model (AE or VAE) and a latent velocity field model $V_\theta(z, t)$.

F27 What is the sampling sequence for Latent Flow Matching?

Sample base latent Z_0 , integrate the latent ODE to get Z_1 , and decode Z_1 into the data space.

F28 Classifier-Free Guidance (CFG) for flow matching, what is the formula for the guided velocity V_{cfg} ?

$V_{\text{cfg}} = V_{\text{uncond}} + S(V_{\text{cond}} - V_{\text{uncond}})$

F29 What does the guidance scale S represent in the CFG velocity formula?

The strength of the push toward the specific class relative to the generic data manifold.

F30 How is the unconditional velocity V_{uncond} obtained in a model trained with label dropout?

By querying the model with a 'null' or empty label symbol $@$.

F31 In classifier-guided flow matching, how is the external classifier's gradient incorporated into the ODE?

$\dot{x} = v_\theta(x, t) + \lambda \nabla_x \log p(y|x)$

F32 What is the 'consistency condition' that defines a Consistency Model $f_\theta(x, t)$?

$f_\theta(x_s, t) = f_\theta(x_z, t)$ for any two points x_s, x_z on the same forward noising trajectory.

F33 What is the mapping target of a consistency model $f_\theta(x, t)$?

A canonical representation of the clean sample, usually x_0 .

F34 How do Consistency Models achieve one-step generation?

By learning a direct mapping from any noise level t to the clean data x_0 in a single function evaluation.

F35 In consistency training, what is the purpose of the 'boundary loss' at $t = 0$?

To anchor the model such that $f_\theta(x_0, 0) = x_0$, ensuring it acts as an identity at zero noise.

F36 What is 'Consistency Distillation'?

Training a consistency student model to match the multi-step trajectories produced by a pre-trained 'teacher' model.

F37 Write the general form of a continuous-time Diffusion SDE.

Write the general form of a continuous-time Diffusion SDE. $dx = f(x, t)dt + g(t)dW$;

F38 What defines the Variance-Preserving (VP) property in continuous diffusion schedules?

The scale $\alpha(t)$ and noise $O(t)$ satisfy $\alpha(t)^2 + O(t)^2 = 1$ for all t

F39 VP-style marginal $X_t = \alpha(t)x_0 + O(t)\epsilon$, what is the boundary condition for $\alpha(t)$ as $t \rightarrow 0$?

$\alpha(t) = 1$ (the signal vanishes into noise).

F40 How is the student model updated in 'stop-gradient' consistency training?

$\mathcal{L}_{\text{cons}} = \mathbb{E} \|f_\theta(x_s, t) - \text{sg}(f_\theta(x_z, t))\|^2$, where $\text{sg}(\cdot)$ prevents gradient flow through the target side.

F41 In the context of 2D image flow matching, what does a single pixel (i, j) in the velocity map represent?

The instantaneous rate of change of that pixel's brightness at time t .

F42 Why is the per-pixel velocity $V_0(X_x, t)$ not independent of other pixels?

The network uses the entire image context to determine the change for each specific pixel.

F43 What is the primary lever changed in 'Optimal Transport Flow' compared to baseline flow matching?

The coupling (X_0, X_1) is changed to minimize transport cost while keeping the linear bridge X_t the same.

F44 In flow matching, what is the effect of changing the 'bridge' function $l(X_0, X_1, t)$?

It changes both the intermediate points X_z where the model is queried and the target velocity $U_z; = \nabla l$.

F45 What is the relationship between discrete DDPM and continuous-time SDEs?

DDPM is a finite-step discretization of a continuous-time SDE process.

F46 teacher-guided distillation, what does the teacher posterior $Do(X_0 | x, t)$ represent?

The model's estimate of the clean data X_g given a noisy observation X_z .

F47 How is a teacher 'denoise step' from t_0 to $S (S < T_f)$ implemented using Monte Carlo sampling?

Sample a clean candidate X_0 from the teacher, then re-noise it to time S using the forward kernel $q(x_S | X_0, S)$.

F48 What is the benefit of sampling in 'log-SNR' Space during consistency training?

It avoids oversampling extremely noisy regions where the signal is too weak to provide useful gradients.

F49 Gas) Bad What does a negative velocity value ($[V_g]; < 0$) imply for a specific pixel in an image normalized to $[-1, 1]$?

The pixel should become darker (move toward the value -1).

F50 What is the 'deterministic transport' view of Flow Matching?

The velocity field defines a flow map that pushes the entire probability mass from P_0 to P_T through deterministic ODE paths.

Chapter 06 — Transformers and Large Language Models

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 **An autoregressive model writes Explain why this is both a probabilistic statement and a practical training strategy.**

It is a probabilistic statement because it expresses the joint probability of the whole sequence in terms of conditional distributions. The model is therefore assigning likelihood to complete sequences, not merely making isolated predictions. It is also a practical training strategy because maximizing the joint likelihood becomes equivalent to training the model to predict each next token from its prefix. This converts sequence modeling into a large collection of supervised prediction problems, one at each position in the sequence. The factorization also explains generation: once the conditionals are learned, the model can generate by repeatedly predicting and appending the next token.

Q2 **In self-attention, why do we need separate query, key, and value projections? Why not let one vector play all three roles?**

The query represents what a token is currently looking for, the key represents what each token offers for matching, and the value represents the content to be retrieved once a match is made. Keeping these roles separate gives the model more expressive flexibility. If one vector had to serve all three roles, the model would have less freedom to distinguish matching behavior from content storage. Separate projections allow the model to learn one geometry for deciding relevance and another for representing information to be passed forward. This separation is one reason attention can behave like learned retrieval rather than simple similarity averaging.

Q3 **The output of attention is a weighted sum of value vectors. Why is the value vector conceptually necessary once the model has already computed relevance weights?**

The relevance weights only indicate which positions matter and by how much. They do not specify what information should actually be extracted from those positions. The value vectors provide the content that is aggregated after the relevance pattern has been decided. This separation means a token can attend strongly to a position because it is relevant, while still retrieving a transformed representation of the information stored there. Without values, attention would know where to look but not have a flexible mechanism for deciding what representation to pull back from that location.

Q4 **Why can multi-head attention be more expressive than single-head attention even if both operate on the same input sequence?**

Different heads use different learned projections, so they can focus on different types of relationships at the same time. One head might track local syntactic structure, another might follow long-range references, and another might attend to positional or semantic patterns. A single head must compress all these possibilities into one attention map and one value aggregation process. Multi-head attention distributes the work across parallel subspaces, allowing the model to represent richer interaction patterns. The advantage is not just more parameters, but structured diversity in how context is analyzed.

Q5 Why is causal masking essential in a decoder-only language model? What would happen conceptually if it were removed during training?

A decoder-only language model is supposed to model Causal masking enforces this by preventing token from attending to future tokens. If the mask were removed during training, the model could directly use the answer token it is supposed to predict. This would make training deceptively easy but would destroy the intended probabilistic interpretation. The model would no longer be learning a valid autoregressive conditional distribution, and the resulting training objective would not match generation-time constraints.

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 Write the autoregressive factorisation.

$p(x_1, x_2, \dots, x_T) = \text{PRODUCT}_{t=1..T} p(x_t | x_1, \dots, x_{t-1}) = \text{PRODUCT}_{t=1..T} p(x_t | x_{\leq t})$. This is the chain rule of probability — always valid. It is simultaneously a probabilistic statement and a training strategy: maximising log-likelihood decomposes into per-position cross-entropy.

X2 What are the three main limitations of RNNs that motivated Transformers?

(1) Bottleneck: all past info compressed into one fixed-size hidden state h_t — limited capacity. (2) Sequential: cannot parallelise; step t requires step $t-1$. (3) Vanishing/exploding gradients: BPTT propagates gradients through many time steps. Long-range dependencies become unreliable. Earlier tokens are increasingly forgotten.

X3 List the LSTM equations in full.

Forget gate: $f_t = \text{sigma}(W_f[h_{t-1}, x_t] + b_f)$ — what to erase from c_{t-1} . Input gate: $i_t = \text{sigma}(W_i[h_{t-1}, x_t] + b_i)$. Candidate: $g_t = \text{tanh}(W_g[h_{t-1}, x_t] + b_g)$. Cell update: $c_t = f_t * c_{t-1} + i_t * g_t$. Output gate: $o_t = \text{sigma}(W_o[h_{t-1}, x_t] + b_o)$. Hidden state: $h_t = o_t * \text{tanh}(c_t)$.

X4 How does the LSTM cell state c_t fix vanishing gradients?

$c_t = f_t * c_{t-1} + i_t * g_t$ — a near-linear update path. If forget gate f_t is approximately 1, gradients flow back through c_{t-1} almost unchanged. The cell state acts as a 'memory highway' where gradients survive long sequences. Unlike vanilla RNN where $\tanh$ nonlinearity always compresses gradients, LSTM gates learn WHEN to let gradients pass.

X5 Write the self-attention formula with all components.

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$. Q (query): what am I looking for? K (key): what do I offer for matching? V (value): what content do I carry? Q and K are projected to compute compatibility scores; V provides the content that gets aggregated. The $\sqrt{d_k}$ scaling prevents dot products from growing too large at high dimensions (which would saturate softmax).

X6 Why are Q, K, V projected separately rather than using one vector?

Separate projections allow the model to learn different geometries for matching (Q/K) vs content (V). If one vector served all three roles, the model loses flexibility — it must simultaneously be a good query, a good key, AND carry useful content. Separation makes attention a learned content-based retrieval mechanism rather than just similarity-weighted averaging.

X7 How does causal masking work in practice?

Set scores to $-\infty$ for future positions BEFORE the softmax: `masked_scores = scores.masked_fill(causal_mask == 0, -inf)`. Then softmax gives exactly 0 weight to future positions. In PyTorch: `causal_mask = torch.tril(torch.ones(T, T)).bool()`. Without it, the model sees the very token it must predict — training trivially has zero loss but generation fails.

X8 Write the sinusoidal positional encoding formula.

$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$. $PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$. Properties: (1) no learned parameters; (2) unique pattern per position; (3) relative offsets encoded — $PE(pos+k)$ is a linear function of $PE(pos)$, so the model can learn relative positions. Used in original Transformer.

X9 Why do Transformer blocks need residual connections and LayerNorm?

Residual connections allow gradients to bypass each sublayer — gradients flow directly to earlier layers, preventing vanishing in deep stacks. They also preserve earlier representations. LayerNorm normalises within each token's representation, stabilising activation scales across layers. Without both, deep Transformer training becomes practically infeasible. They don't add domain knowledge but are functionally essential.

X10 Explain BPE tokenization briefly.

Byte-Pair Encoding builds vocabulary by iteratively merging the most frequent adjacent symbol pairs. Starting from characters, count pair frequencies, merge top pair, repeat until vocab size reached. Balances vocabulary size (word-level too rigid) against sequence length (char-level too long). GPT-2 uses byte-level BPE: base = 256 byte values, handles ANY Unicode with no OOV tokens.

X11 What is the difference between encoder-only, decoder-only, and encoder-decoder Transformers?

Encoder-only (BERT): bidirectional attention (no causal mask), strong for understanding tasks (classification, retrieval, tagging). Decoder-only (GPT): causal masked attention, autoregressive generation. Encoder-decoder (T5): bidirectional encoder + causal decoder with cross-attention to encoder output, for seq-to-seq (translation, summarisation).

Chapter 07 — Vision Transformers (ViT)

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 Consider a Vision Transformer without any class token. Is image classification still possible? Explain conceptually how and discuss one trade-off.

Yes, classification is still possible. The model can aggregate the final patch tokens using mean pooling, attention pooling, or another learned global summarization method, and then apply a classifier to that global representation. The trade-off is that without a class token, the model loses a dedicated sequence position whose role is to accumulate task-specific global information. Pooling may be simpler and more symmetric, but it may also be less explicitly specialized for classification than a learned class token pathway.

Q2 Why do residual connections and layer normalization play an important role in a deep Vision Transformer, even though neither one directly adds visual knowledge?

They play an optimization and stability role. Residual connections help preserve information across layers and make it easier to train deep stacks without destroying earlier representations. Layer normalization stabilizes the scale of activations, which makes learning more predictable. Although they do not encode image structure directly, they are crucial for turning the transformer from a conceptual architecture into a trainable deep system. Without such mechanisms, learning can become unstable or inefficient.

Q3 A student claims that ViTs are simply better than CNNs because they are newer and more general. Critically evaluate this claim.

The claim is too simplistic. ViTs are highly flexible and have become central in many modern systems, especially when large-scale pretraining or multimodal integration is involved. They can capture global interactions elegantly and transfer well across tasks. However, CNNs remain strong because their inductive biases are well matched to many visual problems. They can be more efficient, more data-frugal, and highly competitive when modernized. The better conclusion is that ViTs and CNNs offer different advantages, and performance depends on data scale, compute budget, and task structure.

Q4 Why is the rise of transformer backbones in generative image models conceptually significant, rather than just an engineering curiosity?

It is significant because it suggests that the transformer has become a general representation mechanism rather than a tool tied only to language. If transformer-style backbones can support both understanding and generation, then the same broad architectural principles may unify multiple kinds of perception and synthesis. This also changes how one thinks about model design. Instead of treating classification backbones and generative backbones as fundamentally separate families, one can view them as different uses of related token-based architectures.

Q5 Summarize the full conceptual flow of a Vision Transformer from raw image to output prediction, and identify the main design choices at each stage.

The image first enters as a tensor of pixels and is split into patches. Those patches are projected into token embeddings, often using a linear map or a convolutional implementation. Positional information is added so the model knows where tokens came from, and optionally a class token is prepended for image-level prediction. The sequence then passes through stacked transformer encoder blocks, where self-attention and MLP layers repeatedly refine the token representations. Finally, an aggregation choice is made, such as reading the class token or pooling patch tokens, and a task-specific head maps the representation to class logits or another target space. The main design choices include patch size, embedding dimension, positional encoding type, use of a class token, encoder depth, number of attention heads, and the form of the output head.

$$H \times W \times P \times P \times N = \dots H \times P \times W \times P \times RD \times \text{Xpatch} \in \mathbb{R}^{N \times D}$$
$$RD \times P \times D \times X' = X + P, P \times X_0 = [x_{cls}; x_1; x_2; \dots; x_N]$$
$$\text{logits} = W_{cls}x(L)_{cls} + b$$
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Flashcard Q&A (50 cards)

Quick-fire questions on specific concepts, terms, and details.

F1 What is the central architectural idea behind a Vision Transformer (ViT) regarding how it processes an image?

It splits an image into fixed-size patches and treats them as a sequence of visual tokens.

F2 Why is the 'token-based view' of images important for modern generative models?

It allows models to easily condition, scale, and mix image data across different modalities like text.

F3 the context of generative modeling, what advantage does 'global context' provide through self-attention?

Every token has a direct path to every other token, enabling coordination of structure across the whole image.

F4 What did the Diffusion Transformer (DiT) demonstrate regarding scaling compared to traditional U-Nets?

Replacing the diffusion U-Net with a transformer backbone is highly effective and scales predictably with model size.

F5 Why are Vision Transformers considered highly compatible with multimodal systems like text-to-image generators?

Both visual structure and language can be expressed and jointly processed in a unified token form.

F6 Gas) Fi Which model architecture uses a bidirectional transformer to synthesize images by predicting masked visual tokens?

MaskGIT

F7 How does the U-ViT architecture treat components like noisy image patches, time, and conditions?

It treats them all as tokens within a single transformer-style framework.

F8 The original 2017 Transformer paper 'Attention Is All You Need' showed that sequence modeling could be built entirely around

The original 2017 Transformer paper 'Attention Is All You Need' showed that sequence modeling could be built entirely around : Self-attention

F9 What significant signal did the DETR (2020) model provide to the field of computer vision?

It showed that transformer-style reasoning is useful for vision tasks like object detection by reframing them as set-prediction problems.

F10 What specific task did the original 2021 ViT paper focus on to show that patch-based transformers could work on images?

Image classification

F11 What was the primary contribution of the DeiT (2021) model regarding training efficiency?

It introduced a distillation token for task-specific supervision to train ViTs more efficiently on ImageNet-scale data.

F12 What architectural feature did the Swin Transformer introduce to address the needs of dense-prediction tasks like segmentation?

A hierarchical structure using shifted local windows.

F13 Gas) Hh The Masked Autoencoder (MAE) demonstrated that ViTs are strong encoders for _ by reconstructing masked image patches.

The Masked Autoencoder (MAE) demonstrated that ViTs are strong encoders for by reconstructing masked image patches. Self-supervised representation learning

F14 If an image has height H, width W, and the patch size is P, what is the formula for the number of patches N?

7H If an image has height H, width W, and the patch size is P, what is the formula for the number of patches N?

F15 What is the purpose of the 'linear projection' step in a ViT after the patches are flattened?

It maps each flattened patch vector into a learned embedding space of dimension D.

F16 Why is it necessary to add positional embeddings to patch tokens in a Transformer?

Transformers are permutation-invariant and do not automatically know the spatial location of tokens.

F17 What is the specific role of the 'class token' (CLS) in a Vision Transformer?

To collect global information from all other patches through self-attention for use in classification.

F18 a standard Transformer encoder block, which component typically follows the first Layer Normalization step?

Multi-Head Attention

F19 What is the function of the 'MLP head' in a Vision Transformer designed for classification?

It takes the final representation of the class token and produces the output logits for the predicted classes.

F20 How does a general-purpose encoder differ from a classifier in terms of the final image embedding?

The encoder produces a single feature vector (via the class token or mean pooling) rather than class logits.

F21 True or False: The class token is a fixed, non-trainable vector prepended to the patch sequence.

True or False: The class token is a fixed, non-trainable vector prepended to the patch sequence. False

F22 During training, how is the class token (Xcls) updated?

It is updated through backpropagation from the loss function, just like any other learnable parameter.

F23 What architectural condition allows the class token to gather information from the whole image in a ViT?

Bi-directional attention (lack of a causal mask).

F24 What is 'mean pooling' in the context of ViTs, and how does it relate to the class token?

It is an alternative to the class token where the final representation is the average of all patch tokens.

F25 What is the standard mathematical representation for adding absolute positional embeddings to patch tokens EH?

$Z_0 = [X_{cls}; E] + P$

F26 What is a major drawback of 'Learned Absolute Encodings' when the input image resolution changes?

The learned positional table usually has to be interpolated to match the new number of patches.

F27 How do 'Relative Positional Encodings' differ from absolute ones in vision tasks?

They encode how two patches are positioned relative to each other (e.g., 'above' or 'left of') rather than their fixed coordinates.

F28 What specific implementation does the Swin Transformer use to handle positional information?

Relative position bias added to the attention logits.

F29 What is the primary mechanism of 'Conditional Positional Encodings' (CPE)?

The positional signal is generated dynamically from the current token neighborhood using a Position Encoding Generator (PEG).

F30 How does 'Rotary Positional Embedding' (RoPE) apply position information to the Transformer?

By rotating the queries and keys in the attention mechanism before computing the scores.

F31 For which type of tasks is a 2D-aware positional encoding (like relative bias or RoPE) most beneficial?

Tasks requiring strong spatial reasoning or resolution transfer, such as detection and segmentation.

F32 What 'clever trick' is used to implement patch embedding efficiently in code?

Using a 2D convolution layer where the kernel size and stride both equal the patch size.

F33 the patch embedding convolution `~nn.Conv2d(in_chans, embed_dim, kernel_size=P, stride=P)`, what does "embed_dim" represent?

The number of filters, which corresponds to the output token embedding dimension D.

F34 the provided ViT implementation for CIFAR-10, what is the shape of the tensor after flattening the spatial grid but before the transpose?

[B, D, N]

F35 What is the formula for the attention scores in Multi-Head Self-Attention, where (x is the head dimension?

Attention(Q, K, V) = softmax_Q (QK^T/√d_k) V

F36 Which activation function is typically used in the MLP block of a Transformer?

Which activation function is typically used in the MLP block of a Transformer? GELU

F37 What is the sequence length seen by the transformer if the image is split into 64 patches and a class token is added?

What is the sequence length seen by the transformer if the image is split into 64 patches and a class token is added? 65

F38 the provided CIFAR-10 training loop, what is the purpose of 'weight decay' in the AdamW optimizer?

It serves as a regularization technique by adding a penalty based on the magnitude of the weights.

F39 What does the 'FID' (Fréchet Inception Distance) metric measure in generative modeling?

The fidelity and distributional closeness of generated images to real images (lower is better).

F40 How does 'Precision' differ from 'Recall' in the context of evaluating generative models?

Precision measures realism (fidelity to the real manifold), while Recall measures coverage (diversity of the distribution).

F41 What did the comparison between U-Net and U-ViT indicate regarding the 'sFID' and 'FID' metrics?

The ViT-style backbone (U-ViT) quantitatively outperformed the CNN-style U-Net baseline.

F42 the scaling analysis of DiT, why is it concluded that larger models are superior to smaller models with more sampling compute?

Smaller models cannot close the performance gap even if they use more computation steps during test-time sampling.

F43 Why is 'trunc_normal' initialization often used for the class token and positional embeddings in ViT?

It is a common practice to stabilize training and improve initial convergence.

F44 What is the purpose of the 'scale' factor (1/√d_k) in the attention mechanism?

To prevent the dot products from growing too large in magnitude, which would lead to vanishing gradients in the softmax.

F45 ViT for classification, which specific token's final hidden state is fed into the linear classification head?

The class token (index 0 of the output sequence).

F46 Concept: Patch Partitioning

Concept: Patch Partitioning Definition: The process of dividing an input image into non-overlapping spatial grids that serve as the fundamental input units for the transformer.

F47 What is the relationship between the number of heads and the embedding dimension D in Multi-Head Attention?

The embedding dimension D must be divisible by the number of heads.

F48 Why is bi-directional attention necessary for the class token to function effectively?

It allows the token to attend to all image patches simultaneously to form a global summary.

F49 What is the 'Inception Score' (IS) used to reward in generative modeling evaluation?

Images that are individually recognizable and collectively varied across classes.

F50 What is the input-to-output transformation of the MLP in a Transformer block regarding the shape of the tensor?

The MLP transforms each token independently and preserves the sequence shape $[B, N+1, D]$.

Code-Level Questions (from lecture notebooks)

Questions about the actual code in the lecture notebooks: implementation details, design choices, common patterns. Useful if the exam asks you to interpret or write code.

C1 The lecture splits CIFAR-10 images into patches using Conv2d. For a 32x32 image with patch_size=4, how many patches are produced and what is the tensor shape flow?

$N = (32/4)^2 = 64$ patches. Conv2d(3, embed_dim, kernel_size=4, stride=4): $[B, 3, 32, 32] \rightarrow [B, \text{embed_dim}, 8, 8]$. Flatten spatial: `.flatten(2)` $\rightarrow [B, \text{embed_dim}, 64]$. Transpose: `.transpose(1,2)` $\rightarrow [B, 64, \text{embed_dim}] = [B, N, D]$. This is equivalent to: for each 4x4 patch, flatten the $4*4*3=48$ pixels and apply a linear map W ($48 \rightarrow D$). Conv2d with kernel=stride=patch_size implements this efficiently without explicit loops over patches.

C2 The lecture uses `nn.Parameter(torch.zeros(1, 1, embed_dim))` for the CLS token and `nn.Parameter(torch.zeros(1, num_patches+1, embed_dim))` for positional embeddings. Why `torch.zeros` and not `torch.randn`? Does it matter?

Initialising to zeros is a common choice: the CLS token starts with no prior preference, and positional embeddings start as identity (add nothing). The model learns meaningful values through backpropagation during training. The lecture notes: 'It is initialized to a random value, is shared by all other tokens and gets updated based on the loss function.' Using `torch.randn` (small random init) is also valid and sometimes preferred to break symmetry. The key point is that both are `nn.Parameters` — they are trainable and will converge to useful values regardless of initialization, assuming training is stable. Large initial values (e.g., `randn * 0.02`) can cause attention collapse early in training.

C3 In the ViT classification head, the lecture reads `x[:, 0]` after the encoder. What is at position 0, why not position 1, and what would change if we used `x.mean(dim=1)` instead?

Position 0 is the CLS (class) token — prepended before patch tokens in `torch.cat([cls_tokens, x], dim=1)`. Positions 1 through N are the patch tokens. `x[:, 0]` extracts the CLS token state after L encoder blocks — it has attended to all patch tokens bidirectionally and accumulated global image info. `x.mean(dim=1)` (mean pooling over all N+1 positions including CLS) is an alternative the lecture explicitly discusses: 'Many vision models instead use mean pooling... This often works well, especially for smaller datasets.' The difference: CLS has a dedicated position with its own learned bias for classification; mean pooling treats all positions symmetrically. For CIFAR-10 scale, both work similarly.

C4 The lecture's ViT uses `nn.LayerNorm` before attention (Pre-LN). What is Pre-LN vs Post-LN, and why does it matter for deep ViTs?

Post-LN (original Transformer): $x' = \text{LayerNorm}(x + \text{Sublayer}(x))$. Normalises AFTER the residual. Pre-LN (modern standard, used in lecture): $x' = x + \text{Sublayer}(\text{LayerNorm}(x))$. Normalises BEFORE the sublayer. Why it matters: Post-LN suffers from gradient vanishing at initialization in deep networks — the output is normalised but the residual stream is not. Pre-LN keeps the residual stream unnormalised, allowing gradients to flow directly. ViT and GPT-2 both use Pre-LN. The lecture uses Pre-LN for stability. A practical consequence: Pre-LN models often do not need a warmup learning rate schedule as much as Post-LN models.

C5 For CIFAR-10 training, the lecture uses: `T.RandomCrop(32, padding=4)` and `T.RandomHorizontalFlip()` for training but NOT for testing. Why are these augmentations applied only during training, and what would happen if you also applied them at test time?

These are data augmentations that artificially expand the training distribution. RandomCrop with padding=4: pads the image to 40x40 then crops a random 32x32 region — simulates small translations. RandomHorizontalFlip: mirrors the image with 50% probability — simulates left-right symmetry. At test time, we want a DETERMINISTIC, reproducible evaluation. Applying random augmentations at test time would: (1) give different results each run (non-reproducible); (2) reduce test accuracy since we lose the 'canonical' view. Test-time augmentation (TTA) IS a valid technique where multiple augmented views are averaged — but it must be done deliberately, not accidentally.

C6 The lecture explains that the CLS token is updated via backpropagation. Write the gradient update equation for the CLS token and explain what `L_CLS` represents.

Forward: $z^{(0)} = [\text{CLS}; x_1, \dots, x_N]$; $z^{(L)} = \text{Transformer}_L(z^{(0)})$; logits = $\text{Head}(z^{(L)}_0)$; loss = $\text{CrossEntropy}(\text{logits}, y)$. Backward: $\text{grad_CLS} = d\text{Loss} / d(\text{CLS})$ — computed by autograd through Head, through L transformer blocks, back to the initial CLS token. Update: $\text{CLS} \leftarrow \text{CLS} - \text{lr} * \text{grad_CLS}$. The CLS token is an `nn.Parameter` so it participates in optimizer steps like any weight. The lecture states: 'the loss produces a gradient with respect to the initial CLS token... updated during training.' `L_CLS` is not a separate loss — the standard `CrossEntropyLoss` on the CLS token output trains the CLS token implicitly.

C7 The lecture discusses four types of positional encodings for ViT. Name them, give the key formula for each, and state when each is preferred.

(1) Learned absolute: $\text{PE}[\text{pos}]$ is a learned lookup table added to patch tokens. Preferred for fixed-resolution classification (original ViT). Fails when resolution changes. (2) Sinusoidal: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)})$. No parameters, works for any length. Used in NLP transformers. (3) Relative bias (Swin): attention logit $\pm = \text{bias}(\text{delta_row}, \text{delta_col})$ where delta = relative position. Encodes pairwise relations. Better for detection/segmentation (+2.9 mIoU over absolute). (4) RoPE: rotate Q and K by position angle $\theta * \text{pos}$ before dot product; attention uses $(R_{\text{pos}} q)^T (R_{\text{pos}} k)$. Strong for resolution extrapolation (+2.5 top-1 at 512x512). Rule of thumb from lecture: fixed-resolution classifier -> learned absolute; spatial tasks -> 2D relative; variable resolution -> RoPE or CPE.

Chapter 08 — Vision-Language Models (CLIP, VLMs)

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 Why is the CLIP training objective fundamentally about both positive pairs and negative pairs? Why would training only on matched pairs be insufficient?

CLIP must learn not only that a correct image-text pair should be similar, but also that incorrect pairs should be less similar. If the model only increased similarity for matched pairs without contrasting them against mismatched alternatives, the embedding space could collapse toward trivial solutions in which many unrelated examples also look compatible. The negative pairs define the geometry of separation. They teach the model what distinctions matter. This is why the batch structure is important: each batch provides both the correct match and a set of competing alternatives.

Q2 In a CLIP-like model, why is temperature more than a cosmetic scaling constant? Explain its conceptual effect on learning.

Temperature controls how sharply the model distinguishes the best match from the rest. A lower temperature makes similarity differences more consequential, which can strengthen discrimination but may also make optimization harsher or more sensitive. A higher temperature smooths the distribution and weakens the pressure to separate close competitors. Conceptually, temperature shapes how strongly the learning process emphasizes ranking precision among candidate pairs. So it directly affects the contrastive geometry the model learns.

Q3 Why does zero-shot classification in CLIP not require a conventional classifier head trained on the target classes?

Zero-shot classification works because CLIP reframes classification as language-conditioned similarity matching. Instead of learning a fixed classifier head with one weight vector per class, the model compares an image embedding to text embeddings built from prompts such as "a photo of a cat" or "a photo of a truck." The chosen class is the one whose prompt is most similar to the image. The knowledge of class meaning is therefore mediated through the text encoder and the learned joint embedding space, not through a special task-specific output layer.

Q4 In a fusion encoder-decoder VLM, what is the conceptual difference between masked self-attention in the text decoder and cross-attention to the image representation?

Masked self-attention lets the decoder build a contextual representation of the text prefix while respecting the autoregressive constraint that future tokens are not visible. It answers the question: What has already been said? Cross-attention, by contrast, lets the decoder consult the visual memory produced by the image encoder. It answers the question: What in the image is relevant to the current word being generated? So self-attention organizes linguistic context, while cross-attention injects visual evidence.

Q5 During caption training, teacher forcing is often used. Explain why teacher forcing is helpful during optimization, and also describe one conceptual weakness of relying on it.

Teacher forcing is helpful because, during training, the decoder receives the correct previous tokens rather than its own imperfect predictions. This makes optimization more stable and allows the model to learn the conditional next-token distribution efficiently. However, it creates a mismatch between training and inference. At test time, the model must condition on its own generated tokens, so earlier mistakes can accumulate. This gap is a conceptual weakness often described as exposure bias. $v_t s(x, y) = v^{\top} t$. $(x_i, y_i) s_{ij} = .$

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 Write the CLIP similarity score and explain what each part does.

$s_{\{ij\}} = v_{\text{hat}_i}^T * t_{\text{hat}_j} / \tau$. v_{hat} = L2-normalised image embedding; t_{hat} = L2-normalised text embedding; τ = learnable temperature. Normalisation makes the dot product equal cosine similarity, focusing comparison on direction not magnitude. Temperature τ controls sharpness of the discrimination.

X2 Explain the symmetric contrastive loss in CLIP.

For a batch of N pairs, build similarity matrix S ($N \times N$). Cross-entropy loss applied row-wise (image-to-text) AND column-wise (text-to-image), both targeting the diagonal as correct matches. $L = 0.5 * (CE_{\text{rows}} + CE_{\text{cols}})$. This is symmetric because retrieval should work both directions: given image, find best text; given text, find best image.

X3 Why does the temperature τ matter in CLIP?

Lower τ : sharper softmax — strong gradients on near-misses, stronger discrimination, harder optimisation. Higher τ : smoother distribution — weaker separation between matched and unmatched pairs, easier training but poorer final discrimination. τ directly shapes the contrastive geometry the model learns — it is a meaningful hyperparameter, not just scaling.

X4 How does CLIP do zero-shot classification without a classifier head?

(1) Embed test image: v_{hat} . (2) For each class c , write a text prompt (e.g., 'a photo of a {class}'); embed each: t_{hat_c} . (3) Predicted class = $\text{argmax}_c \cos(v_{\text{hat}}, t_{\text{hat}_c})$. No training on the new classes needed — the shared embedding space mediates classification via natural-language descriptions. New classes can be added simply by writing new prompts.

X5 What is the difference between dual-encoder and fusion VLMs?

Dual-encoder (CLIP): separate image and text encoders compress each modality into a single global vector; matched via cosine similarity. Strong for retrieval, classification. Weak for detailed grounding because global vectors lose local detail. Fusion VLM: image encoder produces visual MEMORY (many patch tokens); text decoder uses cross-attention to consult those memory tokens token-by-token. Stronger for captioning, VQA, grounding.

X6 What is masked self-attention vs cross-attention in a fusion decoder?

Masked self-attention: text decoder attends to PREVIOUS text tokens only (causal mask). Builds linguistic context — 'what has been said so far?'. Cross-attention: text tokens attend to ALL visual memory tokens from the image encoder. Injects visual evidence — 'what in the image is relevant to the current word?'. Self-attention organises text; cross-attention grounds it in the image.

X7 List the main VLM failure modes.

(1) Bias / spurious correlations — associates concepts with backgrounds, demographics, or co-occurring objects. (2) Weak spatial grounding — fails on counting, spatial relations, fine attributes (because global embeddings lose local detail). (3) Distribution shift — performance drops in new domains (medical, aerial, scientific). (4) Hallucination — generates plausible text not supported by the image, because the decoder relies on language priors over visual evidence.

X8 What is teacher forcing and what is exposure bias?

Teacher forcing: during training, decoder receives CORRECT previous tokens (from ground truth), not its own predictions. Makes training stable. Exposure bias: at test time, decoder must condition on its OWN previous predictions, which may be wrong. Errors accumulate. This train-test mismatch is a known limitation of all autoregressive models trained with teacher forcing.

X9 Why does L2 normalisation matter in CLIP embeddings?

Normalising v and t to unit length makes $v^T t = \cos(\angle(v, t))$. This focuses comparison on the DIRECTION of embeddings, not their magnitude. Without normalisation, magnitude can vary due to factors unrelated to semantic content (training dynamics, batch effects). Normalisation stabilises geometry and makes comparisons meaningful and consistent across examples.

Code-Level Questions (from lecture notebooks)

Questions about the actual code in the lecture notebooks: implementation details, design choices, common patterns. Useful if the exam asks you to interpret or write code.

C1 The lecture implements CLIP similarity as: `logits = logit_scale.exp() * F.normalize(image_emb, dim=-1) @ F.normalize(text_emb, dim=-1).T`. Explain each component and what removing `F.normalize` would do.

`F.normalize(v, dim=-1)`: L2-normalises each embedding vector to unit length. Makes dot product = cosine similarity. `.T`: transposes `text_emb` from (B, D) to (D, B) for matrix multiply. `@` operator: matrix multiply $(B, D) \times (D, B) \rightarrow (B, B)$ similarity matrix. Entry $[i, j] = \cos(\angle(\text{image}_i, \text{text}_j))$. `logit_scale.exp()`: scales all similarities by $1/\tau$ where τ is the learned temperature. Without `F.normalize`: dot product = $\|v\| \|t\| \cos(\angle)$. Norms vary by training dynamics — semantically similar pairs with larger norms would dominate. The contrastive loss would implicitly train the model to make norms large for everything, destabilising training. L2 normalisation forces focus on semantic direction, not magnitude.

C2 CLIP uses symmetric cross-entropy loss. The lecture shows: `loss = 0.5 * (F.cross_entropy(logits, targets) + F.cross_entropy(logits.T, targets))`. What are `logits` and `targets` here, and why two cross-entropies?

`logits`: the (B, B) cosine similarity matrix scaled by `logit_scale`. `logits[i, j]` = similarity of image i with text j . `targets`: `torch.arange(B) = [0, 1, 2, ..., B-1]`. For image i , the correct text is text i (same index in the batch). First CE (`logits, targets`): row-wise. For each image i , treats row i as a distribution over all texts. Maximises `logits[i, i] / sum_j logits[i, j]`. This is image-to-text retrieval. Second CE (`logits.T, targets`): column-wise. For each text j , maximises `logits[j, j] / sum_i logits[i, j]`. This is text-to-image retrieval. Averaging them ensures the embedding space works symmetrically in both retrieval directions.

C3 In the lecture's CLIP zero-shot code: ``text_features = encode_text(['a photo of a ' + c for c in classes])``. Why 'a photo of a' rather than just the class name?

CLIP is trained on web image-text pairs. Web captions naturally include phrases like 'a photo of a cat' — not just 'cat'. Using a matching template aligns the test distribution with the training distribution, improving zero-shot accuracy. This is called prompt engineering. The lecture notes it directly. Practical improvements: use multiple templates ('an image of a {c}', 'a photo of the {c}', 'a picture of a {c}'), encode all of them, and average the text embeddings — this ensemble of prompts often pushes accuracy by 1-3%. The class name alone ('cat') would be a valid query but less aligned to CLIP's pretraining distribution.

C4 The lecture trains a student CLIP via similarity-matrix KL distillation from a teacher. What is the loss and why is it better than just training the student with contrastive loss from scratch?

Similarity-matrix KL: for each batch, compute teacher similarity matrix S_{teacher} and student similarity matrix S_{student} . $\text{Loss} = \text{KL}(\text{softmax}(S_{\text{teacher}}/T_{\text{soft}}) \parallel \text{softmax}(S_{\text{student}}/T_{\text{soft}}))$ computed row-wise and column-wise. Why better than scratch contrastive: (1) The teacher's similarity matrix encodes soft relationships — image_i may be 90% similar to text_i and 60% similar to text_j . Pure contrastive (hard 0/1 labels) ignores these soft similarities. (2) Teacher guides the student toward a structured embedding space rather than learning from noisy random negatives. (3) More sample-efficient: the student learns from the teacher's knowledge across ALL batch pairs, not just the diagonal.

C5 The lecture implements a fusion encoder-decoder VLM on MNIST for comparison. What is the key architectural difference from CLIP, and when should you choose each?

CLIP (dual encoder): image encoder and text encoder run independently, producing global vectors. Compared via cosine similarity. No cross-attention between modalities — each modality is compressed to one vector before alignment. Fusion decoder: image encoder produces N patch tokens (visual memory). Text decoder uses CROSS-ATTENTION to attend to these tokens at each step. `self.cross_attn = nn.MultiheadAttention(d_model, n_heads)` with $Q=\text{text state}$, $K=V=\text{visual memory}$. Use CLIP when: retrieval, zero-shot classification, fast inference (cache embeddings). Use fusion decoder when: captioning, VQA, instruction following, tasks needing fine-grained spatial grounding. CLIP loses spatial detail in the global vector; fusion decoder preserves it through cross-attention.

C6 The lecture notes that the CLIP `logit_scale` starts as ``nn.Parameter(torch.ones(1) * np.log(1/0.07))``. What is the starting temperature and why parameterise as `log(1/tau)` rather than `tau` directly?

Starting value: $\log(1/0.07) = \log(14.28)$ approx 2.66. So initial $1/\tau = \exp(2.66)$ approx 14.3, meaning $\tau = 0.07$. Why log parameterisation: temperature τ must be strictly positive. If we parameterised τ directly and optimised with gradient descent, nothing prevents τ from going negative. Parameterising $\text{logit_scale} = \log(1/\tau)$ means $1/\tau = \exp(\text{logit_scale})$ which is always positive regardless of logit_scale value. This is a common trick: parameterise constrained quantities as unconstrained via $\exp/\log$ so gradient descent works without projection steps.

Chapter 09 — Training and Adapting VLMs

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 Why is the chapter's adaptation ladder best understood as a sequence of different intervention types rather than a simple ranking from weak to strong methods?

The methods differ along multiple conceptual dimensions, not just “strength.” Zero-shot, few-shot, and RAG mainly change the conditioning information supplied at inference time, while SFT, LoRA, DPO, and RL-based methods change the policy itself. This means the methods intervene at different levels: zero-shot changes only the instruction, few-shot changes the context by adding demonstrations, RAG changes the context by adding retrieved evidence, SFT changes the objective over labeled outputs, LoRA changes the parameterization of the update, DPO changes the supervision signal from demonstrations to preferences, RL-based methods optimize expected reward. So the ladder is not merely a scale of “more powerful.” It is a structured map of what is being modified: prompt, memory, labels, parameterization, preferences, or rewards. That is why the methods are often complementary rather than mutually exclusive.

Q2 Why does the chapter emphasize adapting both attention projections and MLP projections in transformer-based VLMs?

The chapter highlights that different sublayers play different conceptual roles. Attention projections such as `q_proj`, `k_proj`, and `v_proj` affect routing: they determine what information is queried, matched, attended, and reinserted into the residual stream. MLP projections such as `gate_proj`, `up_proj`, and `down_proj` affect transformation: they determine how information is nonlinearly processed after it arrives. So adapting only attention changes how information flows, while adapting only the MLP changes how information is reshaped. Many downstream tasks need both: new visual-semantic associations, new token-to-token interaction patterns, and new local feature transformations. That is why the chapter presents a broader adapter target set rather than treating LoRA as only an attention modification.

Q3 How does the chapter reinterpret classical reinforcement-learning notation for autoregressive VLM generation?

The chapter shows that RLHF does not require abandoning the language-model viewpoint. It simply reinterprets token generation in RL terms: state : prompt, image, and generated prefix, action : next token, policy : next-token distribution, trajectory : full generated answer, reward: scalar score assigned to the answer. This mapping matters because it explains why policy-gradient ideas apply naturally to VLMs. The model is still generating one token at a time, but now the sequence is judged by a reward function, human preference model, or verifier. So RLHF is best seen as policy optimization over token sequences conditioned on multimodal context.

Q4 Suppose a VLM gives well-formatted answers but repeatedly confuses closely related species because it lacks domain knowledge. Why would RAG often be a more principled first intervention than SFT or DPO?

If the primary failure is missing knowledge, then the problem is not yet one of answer style or preference alignment. RAG directly addresses the missing-knowledge bottleneck by injecting external evidence at inference time. Conceptually, RAG changes the conditional distribution from $p(y|x)$ to a retrieved-evidence version such as $p(y|x, e)$. This is often more principled than immediate SFT because SFT teaches the model to imitate desired outputs, but it does not necessarily give it access to new, explicit supporting evidence at inference time. DPO is even less direct for this failure mode because it adjusts preferences among answers, not the factual evidence base itself. So when the central issue is grounding and missing task knowledge, RAG is often the cleanest first upgrade.

Q5 How can RAG, LoRA-based SFT, and RL-based alignment be viewed as operating on three different layers of adaptation within one overall system?

These methods operate at different levels of the stack. RAG acts on the evidence layer. It decides what external information is available to the generator at inference time. LoRA-based SFT acts on the policy-parameter layer. It changes the model's internal mapping from multimodal context to answer distribution, but does so through a low-rank parameterization. RL-based alignment acts on the behavioral objective layer. It changes what the policy is rewarded for, typically using preference or reward signals rather than fixed gold outputs. So one can think of the three as: RAG: change what the model sees, LoRA/SFT: change how the model maps seen information to outputs, RL alignment: change what output behavior is incentivized. This layered view explains why the methods can be composed rather than treated as mutually exclusive alternatives.

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 List the methods on the VLM adaptation ladder from cheapest to most expressive.

Bottom (cheap, no weight change): (1) Zero-shot prompting — only the instruction. (2) Few-shot prompting — instruction + in-context demonstrations. (3) RAG (Retrieval-Augmented Generation) — instruction + retrieved external evidence. Middle (weight-changing): (4) SFT / LoRA fine-tuning — labeled outputs. Top (most expressive): (5) DPO / RL alignment — preferences or reward signals.

X2 Why is the adaptation ladder NOT just weak-to-strong ranking?

Methods differ along multiple dimensions, not just strength. Zero-shot/few-shot/RAG change the CONDITIONING (context) but not parameters. SFT/LoRA change PARAMETERS using labeled outputs. DPO/RL change BEHAVIOURAL OBJECTIVES using preferences or rewards. They are often COMPLEMENTARY rather than mutually exclusive — RAG can combine with LoRA can combine with DPO.

X3 When is RAG the most principled first intervention?

When the primary failure is missing factual knowledge. If a VLM confuses closely related species due to lack of domain knowledge, the bottleneck is not style or preference alignment but missing evidence. RAG directly addresses this by injecting external evidence at inference time. SFT teaches imitation; DPO adjusts preferences — neither solves a knowledge gap as cleanly as RAG.

X4 Write the SFT loss with the token mask.

$L_{\text{SFT}} = -(1/N) * \sum_i \sum_t m_{\{i,t\}} * \log p_{\theta}(y_{\{i,t\}} | y_{\{i,<t\}}, x_i, v_i)$. The token mask $m_{\{i,t\}}$ is crucial: in chat-style instruction tuning, only the assistant response tokens contribute to the loss — not the user prompt or system text. This is what makes the model learn to produce responses, not echo prompts.

X5 Explain LoRA mathematically.

Keep pretrained weight W frozen. Learn low-rank update: $W' = W + \Delta W = W + B * A$, where B is $(d_{\text{out}} \times r)$, A is $(r \times d_{\text{in}})$, and $r \ll d$. Reduces trainable parameters from $d_{\text{out}} * d_{\text{in}}$ to $r * (d_{\text{in}} + d_{\text{out}})$. Hypothesis: meaningful updates for downstream tasks lie in a low-dimensional subspace — pretrained models already encode rich representations, fine-tuning needs small structured adjustments.

X6 Which projections should LoRA adapt: attention or MLP?

Attention projections (q_proj, k_proj, v_proj, o_proj) change how information is ROUTED between tokens. MLP projections (gate_proj, up_proj, down_proj) change how features are TRANSFORMED within each token. Many downstream tasks need both — adapting only attention changes routing without enabling new feature transformations; adapting only MLP changes features but reuses original routing.

X7 Write the DPO loss.

$L_{DPO} = -\log \sigma(\beta * (\log \pi_{\theta}(y_w) - \log \pi_{\theta}(y_l) - \log \pi_{ref}(y_w) + \log \pi_{ref}(y_l)))$. Compares preferred y_w vs rejected y_l , relative to a frozen reference policy π_{ref} . Performs preference alignment WITHOUT explicit RL rollouts or PPO. Warning: relative preference is NOT factual correctness — model can become better aligned to taste while becoming less truthful.

X8 What role does the reference policy π_{ref} play in DPO?

π_{ref} is a frozen copy of the pre-DPO model (typically the SFT-trained model). It anchors the model to prevent drift during preference training. Without it, the preference objective alone could collapse to degenerate outputs that 'win' comparisons for superficial reasons. The KL penalty against π_{ref} (implicit in DPO) regularises the update — preserves general quality while shifting preference alignment.

X9 Map RL concepts onto LLM token generation.

State s_t = prompt + image + tokens generated so far. Action a_t = next token. Policy $\pi_{\theta}(a_t | s_t)$ = LM's next-token distribution. Trajectory τ = full generated answer. Reward r = scalar score for complete answer (preference rating, verifier output, task success). With this mapping, REINFORCE and PPO-style algorithms apply directly.

X10 Explain RLOO and its advantage formula.

RLOO = Reinforce Leave-One-Out. Generate G completions per prompt; score each. Advantage for completion i : $A_i = r_i - (1/(G-1)) * \sum_{j \neq i} r_j$. This is a PROMPT-SPECIFIC baseline — removes the average effect of the prompt, isolating each completion's contribution. Reduces variance vs global baseline. Simpler than PPO.

Code-Level Questions (from lecture notebooks)

Questions about the actual code in the lecture notebooks: implementation details, design choices, common patterns. Useful if the exam asks you to interpret or write code.

C1 The lecture loads the model with ``BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type='nf4', bnb_4bit_use_double_quant=True)``. Explain each parameter and the total memory saving compared to fp32.

`load_in_4bit=True`: store weights in 4-bit integers instead of 32-bit float. Raw saving: 8x.
`bnb_4bit_quant_type='nf4'`: Normal Float 4-bit — non-uniform quantization levels matched to the empirical (Gaussian) distribution of LLM weights. Lower quantization error than uniform INT4 for the same bit-width.
`bnb_4bit_use_double_quant=True`: also quantize the quantization constants (scale factors) themselves from fp32 to fp8. Saves an additional ~0.4 bits per parameter. Total: from fp16 (16 bits) to NF4 (4 bits) with double quant = approximately 4x reduction. A 7B model: fp16 = 14GB, NF4+DQ = approx 3.5-4GB. Fits on a single 8GB consumer GPU. This is the QLoRA setup.

C2 The lecture configures LoRA as: ``LoraConfig(r=16, lora_alpha=32, target_modules=['q_proj','v_proj'], lora_dropout=0.05)``. Write out the mathematical update and compute the number of new parameters for a 4096-dim attention layer.

LoRA: $W' = W + (\alpha/r) * B @ A$. W frozen. B : $(d_{out} \times r)$, A : $(r \times d_{in})$. For q_proj and v_proj with $d_{in}=d_{out}=4096$, $r=16$: Params per matrix: $r*(d_{in} + d_{out}) = 16*(4096+4096) = 131,072$. For both q_proj and v_proj : $2 * 131,072 = 262,144$ params per layer. In a 32-layer model: $32 * 262,144 = 8,388,608 \approx 8.4M$ trainable params. Original $q+v$ params: $2 * 4096^2 * 32 = 1.07B$. LoRA adds $<1\%$ of original params. $\alpha/r = 32/16 = 2.0$ is the effective learning rate scaling for the LoRA update.

C3 The lecture's SFT code uses a ``formatting_func`` that wraps examples in a chat template including ``<user!>``, ``<assistant!>`` tokens. Why does the format matter so much, and what is the token mask $m_{\{i,t\}}$ in the SFT loss?

Format matters: instruction-tuned models learn the chat format during pretraining. If SFT data uses a different template, the model sees unfamiliar structure — similar to code-switching. Mismatched format degrades performance often more than the actual content quality. Always use ``tokenizer.apply_chat_template()`` for the specific model's native template. Token mask $m_{\{i,t\}}$: in the SFT loss $L = -(1/N) \sum_t m_{\{i,t\}} * \log p(y_t | \dots)$, $m_{\{i,t\}} = 1$ only for ASSISTANT tokens (response), 0 for USER/SYSTEM tokens (prompt). Without the mask, the model would also be trained to reproduce the user prompt — learning to echo queries rather than respond. The mask is what makes SFT train response generation, not prompt imitation.

C4 The lecture uses ``DPOTrainer(model, ref_model=reference_model, ...)``. Write the DPO loss formula and explain why `ref_model` must be a FROZEN copy, not the same model.

DPO loss: $L = -E[\log \sigma(\beta * (\log \pi_{\theta}(y_w|x) - \log \pi_{ref}(y_w|x) - \log \pi_{\theta}(y_l|x) + \log \pi_{ref}(y_l|x)))]$. $= -E[\log \sigma(\beta * (\log_{ratio_w} - \log_{ratio_l}))]$. `ref_model` is frozen (no gradient updates) for two reasons: (1) It anchors the KL penalty: without it, the policy could drift arbitrarily to degenerate outputs that technically win comparisons. (2) It provides a stable baseline log-probability. If `ref_model` also updated, the relative log-ratio would be ill-defined (chasing a moving target). In code: `reference_model = copy.deepcopy(model); reference_model.eval(); for p in reference_model.parameters(): p.requires_grad_(False)`. The trained model updates; the reference stays frozen.

C5 The lecture shows RAG with a document store and embedding retrieval. Why does RAG chunk documents before embedding them, and what happens if `chunk_size` is too large vs too small?

RAG chunks because: (1) Dense embeddings of long documents become over-averaged — too much content compressed into one vector. A query about a specific paragraph cannot retrieve it if it is buried in a 50-page document's embedding. (2) Context window limits: LLMs have max input lengths; smaller chunks = more selective context injection. (3) Retrieval precision: smaller chunks = more specific match to the query. Too large (e.g., whole chapters): embeddings are too general; retrieval returns semi-relevant content; context is noisy. Too small (e.g., single sentences): context is fragmented; retrieved chunks lack surrounding context; answers may be incomplete. Typical sweet spot: 200-500 tokens with ~ 50 -token overlap between chunks to preserve sentence boundaries.

C6 The lecture's few-shot code formats examples as ``Q: ... \nA: ... \n\n`` repeated before the test query. What is the mechanism that makes few-shot work WITHOUT gradient updates?

In-context learning (ICL). The LLM's forward pass sees the entire prompt: examples + test query. The Transformer attention allows the test query token to attend to all previous example tokens. Mechanism: (1) pattern recognition — the model identifies the Q/A schema from the examples. (2) Implicit Bayesian inference — at each forward pass, the model conditions on the examples as part of the context. (3) The model was pretrained on enormous amounts of Q/A-style text, so the few-shot format activates learned behaviours. No weight update occurs — the model's parameters are unchanged. Inconsistent formatting (mixing Q: and Question:, varying separators) confuses the model's pattern matching and significantly hurts accuracy.

C7 The lecture explains KTO vs DPO. What is the data format difference and when would you choose KTO in practice?

DPO requires PAIRED data: (prompt, chosen_response, rejected_response). Must have explicit comparison for each prompt. KTO requires POINTWISE data: (prompt, response, is_good: bool). Each example is just labelled good or bad independently. KTO loss is inspired by Kahneman-Tversky prospect theory: weighs losses more than equivalent gains. Choose KTO when: (1) you have binary human ratings (thumbs up/down) but no comparative pairs; (2) your annotation pipeline cannot guarantee that paired responses are available for every prompt; (3) you have large-scale implicit feedback (e.g., user continued the conversation = good, user deleted = bad). Choose DPO when: you can afford to generate and label pairs (preferred/rejected) — DPO typically gives stronger preference alignment when pairs are available.

C8 The lecture maps RL concepts onto LLM generation: state = prompt + tokens so far, action = next token, policy = LM. What is the RLOO advantage formula and why is it better than a global baseline?

RLOO (Reinforce Leave-One-Out): Generate G completions per prompt. Advantage for completion i : $A_i = r_i - (1/(G-1)) * \sum_{j \neq i} r_j$. = reward of this completion minus average reward of ALL OTHER completions for the SAME prompt. Why better than global baseline: A global baseline averages across all prompts and completions. Some prompts are inherently easy (all completions score high), others hard (all score low). A global baseline cannot separate prompt difficulty from completion quality. RLOO's prompt-specific baseline removes this confound: $A_i > 0$ means 'this completion is BETTER than typical for this prompt.' This reduces variance dramatically, making gradient estimates cleaner and training more stable.

Chapter 10 — Quantization

Sample Exam Questions (from professor)

These are the questions the professor said will help you study. Master these first.

Q1 What is the central tradeoff between scale and range, and why does a single scale create a structural tension in uniform quantization?

In a uniform quantizer, the scale determines both the spacing between adjacent levels and the total real-valued range covered by the available integer codes. This means that the same parameter controls two competing objectives. If it is small, the levels are close together and rounding error is reduced, but the representable range becomes narrow and clipping error grows. If it is large, clipping is reduced because the range widens, but the grid becomes coarse and rounding error increases. So the tension is structural, not accidental. For a fixed bit-width, one cannot improve local resolution without shrinking global coverage, or expand coverage without coarsening the grid. Much of this can be read as a search for better ways to manage this precision-range tension, either by learning scales, changing granularity, using mixed precision, or restructuring the computation itself.

Q2 Why is the straight-through estimator conceptually necessary for quantization-aware training?

The problem is that the forward quantizer contains operations such as rounding and clipping, and their true derivatives are unhelpful for gradient descent. Rounding has zero derivative almost everywhere, and clipping suppresses gradients outside the allowed range. If one backpropagated through the true quantizer directly, learning would often stall or become numerically meaningless. The STE solves this by separating forward fidelity from backward usability. The forward pass still exposes the network to quantization distortions, but the backward pass uses a surrogate derivative that allows optimization to proceed. So the STE is conceptually necessary because QAT needs the model to experience low-precision behavior while still receiving gradients that are useful enough to train on.

Q3 Why are PACT and LSQ both examples of the broader idea that good quantization often comes from learning previously fixed numerical decisions?

PACT and LSQ both convert what might otherwise be hand-designed quantizer settings into trainable parameters. In PACT, the clipping threshold is learned from the task loss rather than fixed by heuristic range selection. In LSQ, the step size itself becomes a learned quantity rather than a predetermined scale chosen only from statistics. The broader conceptual theme is that quantization is improved when the network is allowed to adapt not only its weights but also the numerical rules that define the quantizer. This is a strong shift from early quantization schemes, where clipping and scale were often treated as engineering constants. PACT and LSQ show that these constants are actually part of the optimization problem.

Q4 Why does GPTQ use inverse-curvature compensation after quantizing a column or block, instead of simply rounding each block independently?

If one rounds each block independently, the induced error is accepted as final and may accumulate through the rest of the layer. GPTQ instead recognizes that once one column is quantized, the remaining unquantized columns are still free to move. Under a second-order approximation, those remaining degrees of freedom can be adjusted in a way that absorbs part of the damage caused by the newly quantized block. The inverse Hessian appears because it gives the locally optimal compensating update under the quadratic loss model. So GPTQ is conceptually important not just because it is a low-bit PTQ method, but because it turns quantization into a sequence of quantize, estimate damage, compensate, continue rather than a sequence of isolated rounding events.

Q5 Why are diffusion models harder to quantize post-training than ordinary feedforward classifiers?

A feedforward classifier usually experiences quantization error only across depth: a perturbed activation is propagated through later layers in one pass. A diffusion model is harder because the denoiser is called repeatedly across many timesteps. Small quantization errors can therefore compound over the denoising trajectory, not just inside one forward pass. A second complication is that diffusion statistics vary with timestep. This means one static calibration rule may be poor across the entire reverse process. Thus diffusion PTQ is hard for two linked reasons: the model is iterative, and its operating distribution shifts over time.

Additional Likely Exam Questions

Extra questions covering lecture learning objectives — likely topics that may appear.

X1 Write the uniform quantizer and explain the precision-range tradeoff.

$\hat{x} = s * \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$. The scale s controls BOTH resolution AND range: small s = fine grid + narrow range (more clipping); large s = coarse grid + wide range (more rounding). For fixed bit-width, one cannot improve both simultaneously — this is a structural tension, not an engineering inconvenience.

X2 Why is layer sensitivity to quantization related to the Hessian?

Near a trained minimum, the loss increase from a perturbation e is: $\Delta L \approx -L'(w)*e + 0.5*L''(w)*e^2$. At minimum $L'(w) \approx 0$, so the Hessian $L''(w)$ dominates. High Hessian curvature = high sensitivity to quantization. This justifies keeping sensitive layers at higher precision (FP32) and aggressively quantizing low-sensitivity layers.

X3 Explain QAT and why the STE is essential.

QAT moves the quantizer inside training: model trains with fake-quantized weights/activations. The challenge: $\text{round}()$ and $\text{clip}()$ are not differentiable (rounding has zero derivative almost everywhere). Straight-Through Estimator (STE) fix: forward uses fake-quantized values; backward replaces the true gradient (0) with the IDENTITY inside the clip range. Not exact, but useful — allows gradients to flow.

X4 What do PACT and LSQ have in common?

Both convert previously hand-chosen numerical parameters into LEARNABLE quantities optimised by the task loss. PACT learns the clipping threshold α . LSQ learns the step size s itself. The broader principle: better quantization comes from making the numerical rules of the quantizer adaptive, not just the weights they apply to.

X5 Explain GPTQ's Hessian-based compensation.

GPTQ recognises that after quantizing one column, the remaining unquantized columns are still free to adjust. Under a second-order (Hessian) approximation, those weights can be UPDATED to compensate for the damage. The inverse Hessian gives the locally optimal compensating update. Procedure: quantize column $\rightarrow$ estimate damage $\rightarrow$ compensate remaining weights $\rightarrow$ repeat. Better than RTN which accepts each rounding error as final.

X6 What is SmoothQuant and what does its equivalent rescaling exploit?

Observation: in LLMs, activations have large outliers while weights are well-behaved. SmoothQuant rewrites $Wx = (W*S^{-1})(S*x)$: scales activations down by S and scales weights up by S^{-1} . This MIGRATES the outlier difficulty from activations to weights — making both easier to quantize. Mathematically equivalent computation, but the rescaled form is much more quantization-friendly.

X7 Why are diffusion models harder to quantize post-training than feedforward classifiers?

(1) Errors compound: the denoiser runs hundreds or thousands of times across the reverse process — small per-call quantization errors accumulate over the trajectory. (2) Distribution shifts with timestep: activation distributions differ between $t=T$ and $t=0$; calibration data from one noise level isn't representative of the whole process. Effective diffusion PTQ requires timestep-aware calibration.

X8 List the PTQ method progression from simplest to most sophisticated.

RTN (Round-to-Nearest): each weight rounded independently — baseline, no data awareness. AdaRound: data-driven, minimises LAYER OUTPUT error on calibration data (not weight error). GPTQ: Hessian-based compensation after each column quantized. SmoothQuant: equivalent rescaling to migrate activation outliers to weights. Trend: shift from 'approximate weights well' to 'approximate the function well'.

X9 What does I-BERT contribute to quantization?

I-BERT quantizes the FULL computation graph, including nonlinear operators (GELU, Softmax, LayerNorm) using integer polynomial approximations. Quantizing only matrix multiplications is insufficient — nonlinear ops in FP32 break the integer-only pipeline. I-BERT reframes quantization as a property of the entire executed graph, not just stored parameters.

Quick Formula Sheet

All key formulas you might need, organised by chapter.

Chapter 02 — VAE

Joint: $p_{\theta}(x, z) = p_{\theta}(x|z) * p(z)$, $p(z) = N(0, I)$

Marginal: $p_{\theta}(x) = \int p_{\theta}(x|z) p(z) dz$ [INTRACTABLE]

ELBO: $L(x) = E_q[\log p_{\theta}(x|z)] - KL(q_{\phi}(z|x) || p(z))$

$\log p_{\theta}(x) = ELBO + KL(q_{\phi}(z|x) || p_{\theta}(z|x))$ [ELBO + gap]

Analytical KL: $KL(N(\mu, \sigma^2) || N(0, I)) = 0.5 * \sum(\sigma^2 + \mu^2 - 1 - \log(\sigma^2))$

Reparameterisation: $z = \mu_{\phi}(x) + \sigma_{\phi}(x) * \epsilon$, $\epsilon \sim N(0, I)$

Beta-VAE: $L = E[\log p(x|z)] - \beta * KL(q || p)$, $\beta > 1$ promotes disentanglement

CVAE: $q_{\phi}(z|x, y)$, $p_{\theta}(x|z, y)$

Chapter 03 — Diffusion

Markov property: $P(x_t | x_{\{t\}}) = P(x_t | x_{t-1})$

Joint: $P(x_{0:T}) = P(x_0) * \prod_t P(x_t | x_{t-1})$

Forward step: $q(x_t | x_{t-1}) = N(\sqrt{\alpha_t} * x_{t-1}, (1 - \alpha_t) * I)$

Closed-form marginal: $q(x_t | x_0) = N(\sqrt{\bar{\alpha}_t} * x_0, (1 - \bar{\alpha}_t) * I)$

$x_t = \sqrt{\bar{\alpha}_t} * x_0 + \sqrt{1 - \bar{\alpha}_t} * \epsilon$

where $\bar{\alpha}_t = \prod_{s=1..t} \alpha_s$, $\alpha_t = 1 - \beta_t$

DDPM loss: $L_{\text{simple}} = E[|| \epsilon - \epsilon_{\theta}(x_t, t) ||^2]$

Score relation: $\nabla_{x_t} \log q(x_t | x_0) = -\epsilon / \sqrt{1 - \bar{\alpha}_t}$

Sinusoidal time encoding: $PE(t, 2i) = \sin(t / 10000^{(2i/d)})$

$PE(t, 2i+1) = \cos(t / 10000^{(2i/d)})$

Chapter 04 — Advanced Diffusion

Score: $s_t(x) = \nabla_x \log p_t(x)$

Reverse SDE step: $x_{t-1} = x_t + \beta_t * s_t(x_t) + \sqrt{\beta_t} * z$

Classifier guidance: $s_{\text{guided}} = s_{\theta}(x_t) + w * \nabla_x \log p_{\phi}(y|x_t, t)$

derived from Bayes: $\log p(x|y) = \log p(x) + \log p(y|x) - \log p(y)$

CFG: $\epsilon_{\text{cfg}} = (1+w) * \epsilon_{\theta}(x_t, t, y) - w * \epsilon_{\theta}(x_t, t, \text{null})$

FID: $|| \mu_r - \mu_g ||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r * \Sigma_g)^{0.5})$

Chapter 05 — Flow Matching

ODE: $dx/dt = v_{\theta}(x, t)$, t in $[0, 1]$

Linear bridge: $x_t = (1-t) * x_0 + t * x_1$, target velocity $u_t = x_1 - x_0$

Training loss: $L = E[\| v_{\theta}(x_t, t) - u_t \|^2]$

Euler integration: $x_{n+1} = x_n + dt * v_{\theta}(x_n, t_n)$

Continuity equation: $d/dt p_t + \text{div}(p_t * v) = 0$

Log-density evolution: $d/dt \log p_t(x(t)) = -\text{div}(v(x, t))$

OT coupling: $\pi^* = \text{argmin } E[\| x_0 - x_1 \|^2]$ (better than random pairing)

CFG in FM: $v_{\text{cfg}} = v_{\text{uncond}} + s * (v_{\text{cond}} - v_{\text{uncond}})$

Consistency model: $f_{\theta}(x_s, s) = f_{\theta}(x_t, t)$ along same trajectory

Boundary: $f_{\theta}(x_0, 0) = x_0$

Loss: $L = \| f_{\theta}(x_s, s) - \text{sg}(f_{\theta}(x_t, t)) \|^2$

SDE form: $dx = f(x, t) dt + g(t) dW_t$

VP property: $\alpha(t)^2 + \sigma(t)^2 = 1$

Chapter 06 — Transformers & LLMs

Autoregressive: $p(x_{1:T}) = \text{PRODUCT}_t p(x_t | x_{\{1:t\}})$

RNN: $h_t = f(W_h * h_{t-1} + W_x * x_t + b)$

LSTM gates:

$f_t = \text{sigma}(W_f * [h_{t-1}, x_t] + b_f)$

$i_t = \text{sigma}(W_i * [h_{t-1}, x_t] + b_i)$

$g_t = \text{tanh}(W_g * [h_{t-1}, x_t] + b_g)$

$o_t = \text{sigma}(W_o * [h_{t-1}, x_t] + b_o)$

$c_t = f_t * c_{t-1} + i_t * g_t$

$h_t = o_t * \text{tanh}(c_t)$

Self-attention: $\text{Attention}(Q, K, V) = \text{softmax}(Q * K^T / \sqrt{d_k}) * V$

Causal mask: $\text{scores}[i, j] = -\text{inf}$ if $j > i$, then softmax

Sinusoidal PE: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d)})$

$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d)})$

BPE: iteratively merge most frequent adjacent symbol pairs

GPT-2: byte-level BPE, 256 base byte values, no OOV

Chapter 07 — Vision Transformers

Patch count: $N = H * W / P^2$

Patch embedding: $X_{\text{patch}} \in \mathbb{R}^{N \times D}$

Sequence with CLS + PE: $X_0 = [x_{\text{cls}}; x_1; \dots; x_N] + \text{PE}$

Classification head: $\text{logits} = W_{\text{cls}} * x_{\text{cls}}^{(L)} + b$

Tensor flow: $[B, C, H, W] \rightarrow [B, N, D] \rightarrow [B, N+1, D] \rightarrow [B, D] \rightarrow [B, K]$

Attention cost: $O(N^2)$ — quadratic in patch count

DiT scaling: larger DiT > smaller DiT + more sampling steps

FID, Precision (realism), Recall (diversity), IS (fidelity + diversity)

Chapter 08 — Vision-Language Models

CLIP similarity: $s_{ij} = v_{\text{hat}_i}^T \cdot t_{\text{hat}_j} / \tau$

L2 normalisation: $v_{\text{hat}} = v / \|v\|, t_{\text{hat}} = t / \|t\|$

CLIP loss: $L = 0.5 \cdot (CE_{\text{rows}} + CE_{\text{cols}})$ on diagonal targets

Zero-shot: $\text{predict} = \text{argmax}_c \cos(v_{\text{hat}}, t_{\text{hat}_c})$

Fusion decoder: $p(y_t | y_{\{t\}}, \text{image}) = \text{softmax}(W \cdot \text{decoder_state}_t)$

Cross-attention: text token attends to all visual memory tokens

Chapter 09 — Training and Adapting VLMs

SFT loss: $L = -(1/N) \cdot \sum \sum m_{\{i,t\}} \cdot \log p_{\theta}(y_{\{i,t\}}^* | y_{\{i,t\}}^*, x, v)$

LoRA: $W' = W + B \cdot A, B \in \mathbb{R}^{d_{\text{out}} \times r}, A \in \mathbb{R}^{r \times d_{\text{in}}}, r \ll d$

Trainable params: $d_{\text{out}} \cdot d_{\text{in}} \gg r \cdot (d_{\text{in}} + d_{\text{out}})$

DPO loss: $L = -\log \sigma(\beta \cdot (\log_{\text{ratio}_w} - \log_{\text{ratio}_l}))$

where $\log_{\text{ratio}} = \log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)$

RAG: $p(y|x, v) = \sum_z p_{\eta}(z|q(x, v)) \cdot p_{\theta}(y|x, v, z)$

RLOO advantage: $A_i = r_i - (1/(G-1)) \cdot \sum_{j \neq i} r_j$

Chapter 10 — Quantization

Uniform quantizer: $\hat{x} = s \cdot \text{clip}(\text{round}(x/s), q_{\text{min}}, q_{\text{max}})$

Tradeoff: small s = fine grid + narrow range; large s = coarse grid + wide range

STE backward: $dL/dW = dL/d\hat{W}$ (identity inside clip range)

QAT objective: $\min_{\{W, \theta\}} L(f(x; \{Q_W(W_L)\}, \{Q_a(a_L)\}))$

PACT: α (clip threshold) is learnable

LSQ: step size s is learnable

SmoothQuant: $W \cdot x = (W \cdot S^{-1}) \cdot (S \cdot x)$ — migrate outliers

GPTQ: Hessian-based compensation after each column quantization

I-BERT: integer polynomial approximations for GELU, Softmax, LayerNorm

Concept Index

Alphabetical keyword lookup. Use Ctrl+F in PDF reader for fast search.

Concept	Location	Concept	Location
adaLN	Ch07 — DiT conditioning mechanism (Adaptive LayerNorm)	LayerNorm	Ch06 — per-token normalisation
Adapter / LoRA	Ch09 — low-rank weight update for fine-tuning	Latent Flow Matching (LFM)	Ch05 — FM in autoencoder latent space
AdaRound	Ch10 — PTQ method, layer output reconstruction	Linear schedule	Ch03 — beta_t linearly increasing
Attention (self)	Ch06 — $\text{softmax}(\text{QK}^T/\text{sqrt}(d_k))V$	LoRA	Ch09 — $W' = W + BA$, low-rank update
Autoregressive	Ch06 — $p(x_{1:T}) = \text{product } p(x_{tlx_{\{t\}}})$	LSQ	Ch10 — learnable step size in quantization
AWQ	Ch10 — Activation-aware weight quantization	LSTM	Ch06 — gated recurrent network
Backpropagation Through Time (BPTT)	Ch06 — RNN training, vanishing gradients	MAE (Masked Autoencoder)	Ch07 — ViT self-supervised pretraining
Bayes rule for guidance	Ch04 — classifier guidance derivation	Marginal $q(x_{tlx_0})$	Ch03 — closed form, key for training
Beta-VAE	Ch02 — weighted KL, disentanglement	Markov chain	Ch03 — future depends only on present
Bottleneck (AE)	Ch02 — single fixed latent code	MaskGIT	Ch07 — bidirectional masked image transformer
BPE (Byte-Pair Encoding)	Ch06 — subword tokenisation	Masked self-attention	Ch06/08 — decoder causal attention
Causal mask	Ch06 — -inf future positions before softmax	Mode collapse	Ch02 — $q \approx p$ for all x, no info in z
Cell state	Ch06 — LSTM memory highway	Multi-head attention	Ch06 — h parallel attention heads
CFG (Classifier-Free Guidance)	Ch04 — single model, null token dropout	Noise schedule	Ch03 — beta_t / abar_t over t
Chain rule (probability)	Ch06 — gives autoregressive factorisation	Noise-aware classifier	Ch04 — trained on (x_t, t) pairs
CLIP	Ch08 — contrastive image-text pretraining	OT (Optimal Transport) coupling	Ch05 — minimise $E x_0 - x_{t1} ^2$
CLS token	Ch07 — ViT classification position	Output gate	Ch06 — LSTM, what to reveal as h_t
Consistency model	Ch05 — direct map from any noise level to x_0	PACT	Ch10 — learnable clipping threshold
Continuity equation	Ch05 — $d/dt p_t + \text{div}(p_t * v) = 0$	Patch embedding	Ch07 — Conv2d(kernel=P, stride=P) = linear projection
Cross-attention	Ch08 — text tokens attend to visual memory	Positional encoding	Ch06/07 — sinusoidal, learned, RoPE, relative
CVAE (Conditional VAE)	Ch02 — encoder/decoder conditioned on y	Posterior collapse	Ch02 — $q(z x) = p(z)$ for all x
DDIM	Ch03 — deterministic sampling, same network as DDPM	Precision (realism)	Ch07 — fraction of gen samples in real manifold

Concept	Location	Concept	Location
DDPM	Ch03 — canonical diffusion, noise prediction loss	Q, K, V projections	Ch06 — query, key, value in attention
DeiT	Ch07 — distillation token for ViT training	Quantization-Aware Training (QAT)	Ch10 — quantizer inside training loop
Denoising score identity	Ch04 — $\nabla \log p_t = -\epsilon / \sqrt{1 - \bar{\alpha}_t}$	RAG	Ch09 — retrieval-augmented generation
DiT (Diffusion Transformer)	Ch07 — Transformer backbone for diffusion	Recall (diversity)	Ch07 — fraction of real manifold covered by gen
DPO (Direct Preference Optimization)	Ch09 — preference alignment without RL	Reference policy π_{ref}	Ch09 — frozen anchor in DPO
ELBO	Ch02 — Evidence Lower Bound: reconstruction - KL	Reparameterisation trick	Ch02 — $z = \mu + \sigma * \epsilon$
Encoder-decoder (T5)	Ch06 — seq-to-seq Transformer	Residual connection	Ch06 — skip connection in Transformer
Encoder-only (BERT)	Ch06 — bidirectional, understanding tasks	Reverse process	Ch03 — learned $p_{\theta}(x_{t-1} x_t)$
Entanglement / Disentanglement	Ch02 — semantic factor separation in latent	RLOO	Ch09 — Reinforce Leave-One-Out, prompt-specific baseline
Euler integration	Ch05 — $x_{n+1} = x_n + dt * v_{\theta}$	RoPE	Ch07 — Rotary Position Embedding
Exposure bias	Ch08 — train-test mismatch in autoregressive models	Score function	Ch04 — $s(x) = \nabla_x \log p(x)$
FFN (Feed-Forward Network)	Ch06 — per-token MLP in Transformer	SFT	Ch09 — Supervised Fine-Tuning with token mask
FID	Ch04 — Frechet distance in Inception space	Sinusoidal embeddings	Ch03/06 — for timesteps and positions
FiLM	Ch03/04 — feature-wise linear modulation	SmoothQuant	Ch10 — migrate activation outliers to weights
Flashcards	Ch04/05/07 — 150 quick concept reviews	Stop-gradient (sg)	Ch05 — used in consistency loss
Flow matching	Ch05 — learns velocity field $v_{\theta}(x, t)$	STE (Straight-Through Estimator)	Ch10 — surrogate backward gradient
Forget gate	Ch06 — LSTM, what to erase from c_{t-1}	Swin Transformer	Ch07 — hierarchical + shifted local windows
Forward process	Ch03 — Gaussian Markov chain noising	Teacher distillation	Ch05/10 — student matches teacher trajectories
Gates (LSTM)	Ch06 — forget, input, output	Teacher forcing	Ch08 — train with correct previous tokens
GELU activation	Ch06/07 — smooth ReLU variant	Temperature τ	Ch08 — sharpens/softens CLIP discrimination
GPTQ	Ch10 — Hessian-based PTQ compensation	Timestep conditioning	Ch03 — denoiser must know t
Guidance scale w	Ch04 — controls condition adherence vs diversity	Trajectory vs marginal	Ch03 — $q(x_{1:T} x_0)$ vs $q(x_{t:T} x_0)$
Hallucination	Ch08 — VLM failure mode, language priors dominate	U-Net	Ch03 — downsample + upsample + skips
I-BERT	Ch10 — integer poly approximations for full graph	U-ViT	Ch07 — ViT-style diffusion backbone

Concept	Location	Concept	Location
Inception Score (IS)	Ch04/07 — rewards fidelity + diversity	Variance-Preserving (VP) SDE	Ch05 — $\alpha^2 + \sigma^2 = 1$
Inductive bias	Ch02/07 — model structure that promotes properties	ViT (Vision Transformer)	Ch07 — image as patch tokens
Jensen's inequality	Ch02 — $\log(E[Y]) \geq E[\log Y]$, gives ELBO	VP-SDE = DDPM discretisation	Ch05 — connection between continuous and discrete
KL divergence	Ch02 — measure between distributions	Zero-shot classification	Ch08 — similarity to text prompts, no head

Good luck on your exam!

Remember the open-book strategy:

1. Skim the question — identify the chapter and key concept.
2. Use Ctrl+F or the concept index to jump to the right Q&A.
3. Look for matching [Q] sample questions first — they often appear directly.
4. Read the model answer, then write in your own words.
5. Use the formula sheet for any quick math you need.